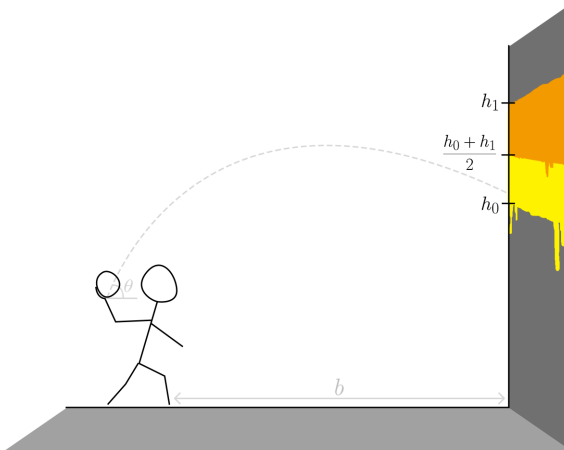


Matematikk i praksis

Tilleggshefte for programmeringsdel

Programmering med Python

Amir M. Hashemi



$$y(t) = -\frac{1}{2}gt^2 + v_0 t \sin \theta$$

$$T = \frac{b}{v_0 \cos \theta}$$

```

1 from math import cos,pi,sin
2
3 def height(t,v0,th):
4     g = 9.81
5     return -.5*g*t**2 + v0*t*sin(
6         th)
7
8 def find_points(h0,h1,y):
9     points = 0
10    mid_point = (h0 + h1)/2.
11    if (h0 <= y <= mid_point):
12        points = 1
13    elif (mid_point < y <= h1 ):
14        points = 2
15    return points
16
17 [h0, h1,g, th] =[3, 3.5, 9.81, pi
18 /4]
19
20 for v0 in [15.,16.,19.,22.]:
21     T = b/(v0*cos(th))
22     y = height(T,v0,th)

```



Forord

Dette veiledningsheftet er et supplement fo læreboken matematikk i praksis (7. *utgave*) [1]. Første del (kapittel 0) er mine forelesningsnotater for programmeringskurs jeg har hatt som jeg vil dele med leseren. I kapittel 0 tas en del med programmeringsdidaktikk med utgangspunkt i klassiske matematiske fag ved videregående skole som håper å hjelpe forelesere for å finne egen metode og pedagogisk opplegg for å tilrettelegge programmering.

Heftets oppbygning

For hvert kapittel gis noen eksempler, oppgaver, noen programmeringsoppgaver og i slutten av hver kapittel finner man fasit for programmeringsoppgaver for hvert kapittel i læreboken (hvis kapittelet hadde programmeringsoppgaver).

Anbefaling til leseren

Når du skal lære et nytt emne er det ikke nok å pugge eller lære seg å få tak i hvordan ting skal gjøres. Det er like viktig å spørre seg hvorfor metoden er som den er, og forstå hvordan ting henger sammen. Da blir det lettere å lære. Jo bedre du forstår hvert delemne, desto lettere er det å bruke den til å løse aktuelle problemer. Du får da god erfaring til å anvende den i andre fagfelt.

Takk

I forbindelse med de ulike utgavene av boken er det mange som har bidratt med hjelp og innspill, og som fortjener stor takk. Spesielt må nevnes Arne Strøm for computergrafikk og Arve Michaelsen for all teknisk hjelp med tidligere utgaver. Hans Foosnæs, Erik Plathe og Jan Søreng skal ha takk for konstruktive innspill til tredje utgave. Takk til Kari Grete N. Børve og Ragnar Soleng for innspill til 5. utgave. Takk også til studentene for deres tallrike tilbakemeldinger gjennom alle år.

Når det gjelder den foreliggende utgaven, vil vi først og fremst takke vår forlagsredaktør Jannicke Bærheim. En stor takk rettes også til Jonas Gahr Sturtzel Lunde og Kristine Baluka ved UiO, og Jarle Peder Berntsen, Anita Gjesteland og Erik Andreas Hanson ved UiB, som har gitt oss tilgang til modellerings og programmeringsoppgaver i oppgavesamlingene deres ved UiO og UiB. Sist og ikke minst vil vi gjerne takke Erik Andreas Hanson og Erlend Storvik, mine kollegaer ved HVL, for å lese gjennom dette heftet, og ha hjulpet å forbedre både teksten og innholdet.

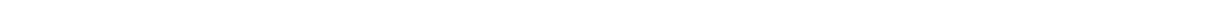
Oslo/Bergen, 21. desember 2022

Amir M. Hashemi og Arne Hole

Eventuelle kommentarer eller meldinger om feil i notatene tas imot med glede: ahas@hvl.no.

ISBN 978-82-94170-07-4

© Forfatterne 2025–2030



Innhold

0	Litt programmering med Python	1
0.1	Å komme i gang med Python	1
0.2	Noen viktige elementer i Python-programmering	2
0.3	Noen av Pythons innebygde funksjoner	3
0.4	Grunnleggende Python-programmering	4
0.5	Datatyper	8
0.6	Variabler	9
0.7	Å sende inn datasett i Python-program	10
0.8	Importeringspakker (numpy, math, pylab, sympy)	12
0.9	Indeksing	14
0.10	Løkker (betingelser og gjentakelser)	15
0.11	Funksjoner	21
0.12	Plotting i Python	28
0.13	Lister	30
0.14	Python-filhåndtering	32
0.15	Laste inn, håndtere og plote datasett	33
0.16	Modellering: tekopp-temperatur	33
0.17	Oppsummering	36
0.18	Fasit for programmeringsoppgaver i kapittel 1	39
1	Funksjoner	42
1.1	Programmeringsoppgaver i kapittel 2 (er ikke med i læreboken)	45
2	Trigonometri og periodiske fenomener	46
3	Trigonometri og periodiske fenomener	47
3.1	Fasit for programmeringsoppgaver i kapittel 3	47
4	Grenseverdi og kontinuitet	48
4.1	Fasit for programmeringsoppgaver i kapittel 4	48
5	Ekspensialfunksjoner	49
5.1	Fasit for programmeringsoppgaver i kapittel 5	49
6	Derivasjon	51
6.1	Numerisk derivasjon	51
6.2	Newtons metode	56
6.3	Fasit for programmeringsoppgaver i kapittel 6	60

7	Integrasjon og numerisk integrasjon	61
7.1	Fasit for programmeringsoppgaver i kapittel 7	64
8	Komplekse tall	66
9	Differensialligninger og anvendelser	67
9.1	Fasit for programmeringsoppgaver i kapittel 9	70
10	Lineær algebra	72
10.1	Vektorer	72
10.2	Matriser	72
10.3	Vektor-matriseoperasjoner	73
10.4	Fasit for programmeringsoppgaver i kapittel 11	77
11	Funksjoner av flere variabler	79
11.1	Optimering	79
11.2	Gradientnedstigningsmetoden (gradient descent)	79
12	Ekstra - typiske eksamensoppgaver	88
	Fasit	95

Kapittel 0

Litt programmering med Python

Python er et objektorientert programmeringsspråk, som ble laget av Guido van Rossum på 80-tallet, og ble først presentert i 1991. Python er et generell programmeringsspråk som kan brukes på alle moderne operativsystemer. Python har en god del fordeler, blant annet er det enkelt å bruke og har mange innebygde funksjoner. Python har en enkel syntaks som ligner på det engelske språket. Python er et ganske moderne programmeringsspråk som kan brukes i forskjellige vitenskapelig kontekster (f.eks. i maskinlæring). Den kan brukes til behandling av tekst, tall, bilder, vitenskapelige data og omtrent alt annet du kan bruke på en datamaskin. Den brukes daglig i driften av f.eks. Googles søkemotor, videodeling YouTube, NASA og New York Børs.

0.1 Å komme i gang med Python

Det er to hovedversjoner av Python, Python 2 og Python 3. Disse versjonene er ganske forskjellige. Her bruker vi Python 3, fordi den er mer semantisk korrekt og støtter nyere funksjoner. Python er et tekstbasert programmeringsspråk, og for å komme igang med å programmere, så har man flere alternativer blant annet:

- Programmere på nett: Det er flere tjenester på nettet som lar deg skrive og kjøre python-programmer, som for eksempel:

<https://trinket.io/python3>

<https://replit.com/new/python3>

- Installere en kildekode-plattform som både er en kode-editor, men som har innebygget en egen «Python-motor».

<https://www.spyder-ide.org/>

<https://jupyter.org/>

Eksempel på et slikt program er Spyder. Dette er en åpen kildekode-plattform integrert i utviklingsmiljø for vitenskapelig programmering med Python. Spyder integreres med en rekke fremtredende pakker i den vitenskapelige Python-pakken, inkludert numpy, scipy, Matplotlib, pandas, IPython, sympy og Cython, samt annen programvare med åpen kildekode. Det finnes også andre slike program:

<https://www.anaconda.com/distribution/>

<https://codewith.mu/en/download>

Konsollvinduet og script i editorvinduet

I dette heftet har vi valgt å bruke Spyder. Vinduet til venstre er editorvinduet og nederst på høyre side er konsollvinduet. I Spyder kan man kjøre Python-kode direkte i konsollvinduet. Dette passer for små og enkle beregninger, for eksempel hvis vi bare skal regne ut et regnestykke som $2 + 5$,

```
1 In [1]: 2+5
2 Out [2]: 7
```

Men når man skriver lengre programmer, er det enklere å skrive koden i et *script*. Et Python-script er en kjørbart fil som inneholder Python-kode. Det vil si at hvis man skriver inn $2+5$ i et script, så skjer det ikke noe før man kjører scriptet. Her er et eksempel på et script som skal beregne arealet til et rektangel:

```
1 a = 5
2 b = 3
3 # Regner ut de to rottene
4 A= a*b
5 print('Arealet til et rektangel med sidelengdene', 'a =', a, 'og b=
      ', b, ' er lik', A)
```

Dersom vi kjører dette scriptet, vil vi få:

Arealet til et rektangel med sidelengdene a = 5 og b= 3 er lik 15

online-programmering

Det er anbefalt av flere læresteder å bruke:

<https://trinket.io/python3>

når man skal programmere på nettet.

0.2 Noen viktige elementer i Python-programmering

De viktige elementene i Python-scriptet er:

- *Kommentarer*: Når man programmerer, så lønner det seg å forklare hva koden gjør. Det gjør man med tegnet *#kommentar* eller 'kommentar' eller kommentar". Dette tegnet tolker Python som «resten av denne linjen skal ikke leses av programmet». Kommentarer kan også settes mellom enten 'enkle apostrofer' eller doble apostrofer".
- *Streng*: En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.
- *Funksjoner*: en gruppe kodelinjer, som utfører en spesifikk oppgave, for eksempel beregner en matematisk uttrykk eller skriver ut en tekst. En funksjon kan brukes flere steder i ett program, med å kalle funksjonen.
- *Pakker*: For å utføre beregninger, er det nødvendig å importere en passende pakke. Her importerer vi pakken `numpy` som `np`, slik at vi kan bruke for eksempel funksjonen `sqrt` som tilhører pakken som `np.sqrt`.

- *Lister*: Når vi trenger å lagre flere verdier i samme variabel, kan vi benytte lister. Lister er samlinger med tall, strenger, variabler eller annet.
- *Datatypes*: En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype, som behandles (tekst (`str`), tall (`int`, `float`), vektor (`array`)).
- *Variabler*: Videre definerer vi de tre koeffisientene a, b og c . I Python kalles disse *variabler*, og likhetstegnet angir verdien deres. Også x_1 og x_2 som vi så regner ut, er variabler, og vi ser at i utregningen brukes a, b og c , som har verdiene vi allerede har gitt dem.
- *Indeksering*: Indeksering er prosessen som ordner et sett med data slik at det blir mulig å spore opp datene. Indeksering i Python starter ved indeksen 0.
- *Løkker*: Løkker brukes for å gjenta noe mange ganger etter hverandre

0.3 Noen av Pythons innebygde funksjoner

Python-interpreter har en rekke innebygde funksjoner som alltid kan kalles med å skrive deres navn med riktige input som inngår i funksjonene.

Funksjon	Beskrivelse
<code>help()</code>	Kaller det innebygde hjelpesystemet
<code>print()</code>	Skriver ut data
<code>abs()</code>	Returnerer absoluttverdi av et tall
<code>float()</code>	Returnerer et flyttall
<code>int()</code>	Returnerer et heltall
<code>input()</code>	For å få input fra bruker
<code>input(float())</code>	For å få input (et reelt tall) fra bruker
<code>input(int())</code>	For å få input (et heltall) fra bruker
<code>input(str())</code>	For å få input (tekst) fra bruker
<code>len()</code>	Returnerer lengde av et objekt
<code>list()</code>	Returnerer en liste
<code>range()</code>	Returnerer en sekvens av hele tall, som standard starter fra 0 og inkrementerer tallene med 1
<code>round()</code>	Returnerer rundverdi av et tall

0.4 Grunnleggende Python-programmering

Kommentere i koden med ' ' og

Kommentarer (en forklarende tekst) er det første vi vanligvis møter i koden. En kommentar starter enten med en hash, og varer ut linjen, eller skrives inne i triple anførselstegn, som første linje i eksempelkoden over. Ved triple anførselstegn kan kommentaren vare over flere linjer. Dette er tekst eller kode som ignoreres når koden kjøres. Det er lurt å ha med kommentarer underveis i koden, fordi det gjør den enklere å lese, og det kan være til hjelp når du senere ser tilbake på den.

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Fri Dec 16 12:39:51 2022
4 @author: ahas
5 """
6 print('Skriv ut noe')
```

Ved å kjøre programmet får vi:

```
1 Skriv ut noe
```

Strenger

En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.

Man kan skrive tekst (strenger) både med apostrof ('... ') og med anførselstegn ("..."):

```
1 print('Denne linjen blir skrevet ut')
2 print(''Denne linjen blir skrevet ut'')
```

Man får ikke feilmelding ved å bruke ekstra mellomrom:

```
1 print('Denne linjen blir skrevet ut')
2 print ('Denne linjen blir skrevet ut')
```

Man får feilmelding hvis man glemmer noe av syntaksen man skal ta med:

```
1 print(Denne linjen blir skrevet ut')
```

Vi skal se nærmere i notatene at print-funksjonens tolker følgende som streng og som tall:

```
1 print('3+4')
2 print(3+4)
```

Resultatet av den første programsetningen blir 3+4 og den andre 7.

Oppg. 0.4.1: a) Skriv en programkode som skriver ut: Denne setningen skrives ut!

b) Finn feilen i følgende programsetninger:

```
1 Print('Jeg liker programmering')
2 print('Jeg liker programmering!')
```

Oppg. 0.4.2: Hva blir resultat av følgende programsetninger:

a)

```
1 print('5+7')
```

b)

```
1 print(5+7)
```

c)

```
1 print('5+7')
```

d)

```
1 Print('5+7')
```

e)

```
1 print('5+7')
2 print()
```

Kalkulator

Vi ønsker at kalkulatoren skal kunne addere (+), subtrahere (-), multiplisere (*) og dividere (/). Vi kaller +, -, * og / for operatorer.

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
-	Subtraksjon	$5 - 2$	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	$5/2$	2.5
//	Heltalldivisjon	$13//3$	4
**	Eksponent	$2**3$	8
%	Modulo	$13 \div 4$	1

- a/b gir kvotienten når divisoren a deles på dividenden b .
- $a//b$ gir antall heltall som er i kvotienten (når man deler a på b) når divisoren a deles på dividenden b .
- $a\%b$ gir restleddet når divisoren a deles på dividenden b .

Heltalldivisjon er divisjon uten restledd. Dette betyr at for eksempel både $14//7$ og $15/7$ blir 2, siden $14 : 7 = 2$ og $15 : 7 = 2 + \frac{1}{7}$.

- $a \div b$ (modulo) gir restleddet når man deler a på b .

Eksempel 0.4.1: Her er et eksempel fra konsollvinduet:

```
1 > 2+3+4*5
2 25
3 >3/2
4 1.5
5 >2**3 +1
6 9
7 > 15/7
8 2.142857142857143
9 >15//7
10 2
```

Oppg. 0.4.3: Regn ut disse med Python

a) $3 \cdot 5 + \frac{4}{5}$

b) $2^3 + 3^2 + \frac{1.25}{5}$

c) Hva resultatet til $18\%4$ blir?

Oppg. 0.4.4: Løs disse oppgavene v.h.a. Python

a) Regn ut arealet til en trekant med høyden 5 cm og grunnlinjen 8 cm.

b) Regn ut arealet til et parallellogram som har høyden 7 cm og grunnlinjen 10 cm.

Arealet til et parallellogram er altså lik lengden (grunnlinjen) multiplisert med høyden.

- c) Regn ut arealet til en trapes som har høyden 6 cm og lengden til de to parallelle sidene er 8 cm og 12 cm.

Arealet av et trapes regnes ut ved å multiplisere høyden og den gjennomsnittlige lengden av de to parallelle sidene.

Å hente π fra numby-biblioteket

Eksempel 0.4.2: Vi ønsker å bestemme arealet til en sirkel med radius 5 cm.

Her har vi to muligheter Vi kan regne direkte med π som verdien 3.14, og regne direkte på kommandolinjen:

```
1 > 3.14*5**2
2 78.5
```

eller vi kan definere pi som 3.14 og regne ut arealet et skript:

```
1 pi=3.14
2 r=5
3 Areal = pi*r**2
4 print(Areal)
```

Ved å kjøre programkoden får man 78.5.

Vi lærer senere at vi kan importere pi fra *math*- eller *numpy*-biblioteket, der man enten installer hele *math*- eller *numpy*-biblioteket eller bare installerer det man trenger. Du kan bruke *numpy*-bibliotekets *numpy.pi* eller *math*.bibliotekets for *math.pi* og få verdien av den matematiske konstanten π i python.

```
1 import numpy
2 r=5
3 Areal = numpy.pi*r**2
4 print(Areal)
```

```
1 import numpy as np
2
3 r=5
4 Areal = np.pi*r**2
5 print(Areal)
```

Hvis man skal regne ut med kjente verdier som π eller kjente funksjoner som cos, kan man enten installere hele *numpy*-biblioteket:

```
1 from numpy import *
2 r=5
3 Areal = pi*r**2
4 print(Areal)
```

eller bare innstallere det man trenger:

```
1 from numpy import pi
2
3 r=5
4 Areal = pi*r**2
5 print(Areal)
```

Hvis man importerer fra numpy bare det man trenger, kan bare bruke dem uten å bruke np. Tilsvarende gjelder dette også for funksjoner som vises i det neste eksempelet.

0.5 Datatyper

En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype som behandles, og hvilke regler som skal brukes for hvordan dataene skal behandles. Her har vi fokus bare på enkle datatyper, datatyper som bare inneholder 'en verdi. De mest brukte er:

- int - integer (heltall som 1,2,3)
- float - desimaltall (eks 2.5 - OBS: her må man bruke punktum og ikke komma)
- str (string) - tekst, OBS: skrives mellom anførselstegn
- bool - boolean (True/False - OBS: når man skal gi en variabel verdien True eller False, må de skrives med stor forbokstav)

I tillegg har vi noen sammensatte datatyper, hvor hver datatype kan inneholde flere verdier, f.eks.

- list - lister
- tuple - tupler/tuppel (ligner på lister, men er en ikke-muterbar sekvens - den kan ikke endres og brukes først og fremst på data som ikke skal endres, slik som ukedager og måneder)
- set - mengder
- dict - dictionary (oppslagsliste)

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	{1,3,5}
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Eksempel 0.5.1: Her vises et eksempel, hvordan strenger oppfører seg med matematiske operasjoner:

```
1 print('tre' + 'ti')
2 print('to' * 3)
```

Utskriften blir da:

```
1 treti
2 tototo
```

0.6 Variabler

Som i matematikk brukes variabler i programmering. Variabler brukes til å huske data til bruk senere i programmet, for eksempel til utregninger, beslutninger og for å gjøre endringer på informasjon. Variabler kan innta verdier eller tekst, og for tekst skal ha med apostrof ('... ') eller anførselstegn ("... ") rundt seg. Man kan opprette en variabel i et program ved å skrive en bokstav eller en tekst etterfulgt av et liketstegn og

- verdien som variabelen skal innta hvis variabelen skal være *et tall*
- apostrof ('... ') eller anførselstegn ("... ") rundt seg (åpnes og avsluttes) hvis variabel skal være *tekst/streng*.

Som vist nedenfor her:

```
1 x= 2
2 y= 'Tekst'
3 z= "Tekst"
```

I løkkene blir ofte variablene oppdatert ved å legge til et tall til variabelen eller multiplisere variabelen med et tall, for eksempel:

Eksempel 0.6.1: Her settes variabelen n til verdien 2 og blir deretter oppdatert:

```
1 n = 2
2 # det legges til 4
3 n = n+4
4 # Resultatet ganges med 5
5 n=5*n
6 # Resultatet legges til 4
7 n += 4
8 # Resultatet ganges med 5
9 n* = 5
10
11 print('n blir da',n)
```

Her blir verdien til variabelen n satt sammen med 4 og ganget med 5 på to ulike måter, før resultatet blir multiplisert med 5 og blir vist. Svaret er da $((2 + 4) * 5 + 4) * 5 = 170$. $\diamond$

Oppg. 0.6.1: Skriv en programkode som utfører følgende regneoperasjoner ved hjelp av variabler:

- Skriv en programkode som leser inn to tall m og n som tar verdiene 7 og 1 henholdsvis, deretter blir m summert med 3 og n ganges med 5, og til slutt multipliseres m og n sammen og resultatet blir skrevet ut på skjermen som: *Produktet er* og resultatet.
- Skriv en programkode som leser inn to tall a og b som tar tallet 5 og teksten *TALL*, deretter multipliseres a og b med henholdsvis 2 og 3 og de oppdaterte variabelene a og b blir vist på skjermen

Oppg. 0.6.2: Skriv en programkode som beregner volum og overflateareal til en rektangulær rett prisme med lengde 5 meter, bredde 4 meter og høyde 3 meter.

Oppg. 0.6.3: Skrive en programkode som regner ut $y = h - \frac{1}{2}gt^2 + v_0t$ for $h = 3.5$, $g = 9.81$, $v_0 = 20$ og $t = 5$, og skriver på skjermen.

Avrunding

F-literalene bruker vi når vi vil runde av et tall i en *print-kommando*. Rett før høyre mengdeparentes legger vi inn et kolon. Da kan vi styre hvordan utskriften formateres. Vi runder av til to desimaler med formatdefinisjonen «.3f». Her betyr bokstaven «f» at variabelen skal skrives ut med et fiksert antall desimaler, og tallet betyr at det skal være tre desimaler. Hvis du i stedet skriver «.2f», får du to desimaler.

Eksempel 0.6.2: Her er et eksempel, hvordan to tall og deres sum blir avrundet:

```
1 a = 25/7
2 b = 32
3 print(f"Her er a {a:.3f} og b {b:.1f}. Summen er {a+b:.3f}.")
```

Det er vanlig at resultatet fra programmet som skrives til skjermen, inneholder både tekst og verdier av variabler. I programmet bruker vi det som kalles f-literal. Vi setter en «f» rett etter venstreparentesen for å fortelle at det kommer en f-literal. Alt som står inni mengdeparenteser i en f-literal, blir tolket som Python-kode. Det vil si at a blir tolket som verdien av variabelen a.

0.7 Å sende inn datasett i Python-program

1. I dataprogram kan variablene deklarerer med å definere variablene som får inndelt en verdi:

```
1 r=2
2 n=5
```

2. Et dataprogram kan ta imot data fra brukeren hver gang man prøver å kjøre programmet:

```
1 alder=input("Skriv inn alderen din:")
2 print(f''Du er {alder} år.'')
```

Her kunne man angi alderen som tall eller tekst. Hvis man ønsker seg å presisere hvilken datatype (heltall, flyttall og tekst), vil man ta i bruk videre i programmet følgende:

```
1 alder=int(input("Skriv inn alderen din:"))
2 r = float(input("Skriv inn radiusen til sirkelen (et reelt tall)
  her: "))
3 n = int(input("Skriv inn a (et heltall) her: "))
```

3. Når vi skal behandle datafil i Python, er det første vi må gjøre å laste ned en fil med dataene og lagre den i et hensiktsmessig format. De vanlige formatene er .txt og .csv. Når vi laster ned filer med stort tallmateriale, er det ofte lurt å bruke .csv (se avsnitt 0.14).

I neste eksempel vises hvordan `float(input())` og `float(input())` for å lese inn tall og hvordan `print(f"tekst og {tall}")` kan brukes for å formatere det som skrives ut på skjermen.

Formatering av tekst og tall

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra brukeren, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```

1 print('Her leses inn variablene')
2 a = int(input("Skriv inn a (et heltall) her: "))
3 b = float(input("Skriv inn b (et reelt tall) her: "))
4
5 c= a+b
6 print()
7 print(f'Summen til {a} og {b} er: {c} ')
8 print() # lager en linje mellomrom
9 print(f'Summen til {a} og {b} er:c=', c)
10 print()
11 print('formatering:')
12 print(f'Her er a {a:.2f} og b {b:.1f}. Summen er {c:.3f}.')
13 print(f'Summen til {a} og {b} er: {c:.3e} ')

```

Ved å kjøre programmet får vi da på skjermen

```

1 # Her leses inn variablene
2
3 Skriv inn a (et heltall) her: 11
4 Skriv inn b (et reelt tall) her: 2.35
5
6 Summen til 11 og 2.35 er: 13.35
7 Summen til 11 og 2.35 er: c =13.35
8
9 formatering:
10 Her er a 11.00 og b 2.4. Summen er 13.350.
11 Summen til 11 og 2.35 er: 1.335e+01

```

Hvis man skulle lese tekst fra brukeren, kunne man bruke `str(input())`.

Oppg. 0.7.1: Skrive en programkode som oppretter tre variabler a , b og c der a blir gitt som input av brukeren, og $b = 3$ og $c = 5$. Programmet skal regne ut $d = \frac{1}{3}ab + c^2$, og skrive resultatet på skjermen.

Oppg. 0.7.2: Skrive en programkode som oppretter to variabler r og h som blir gitt som input av brukeren (radius og høyden til en rett sylinder), og som regner ut $V = \pi r^2 h$ (volumet til sylindere, $\pi = 3.14$), og skriver resultatet på skjermen.

0.8 Importering av pakker (numpy, math, pylab, sympy)

En pakke eller et bibliotek i Python er en samling av moduler, hvor en modul er en fil som inneholder Python-definisjoner (funksjonsdefinisjoner), men inneholder ikke kall til. Modul kan være en en samling av funksjoner og klasser som kan gjenbrukes i mange ulike programmer. Når man bruker en modul fra en pakke, skrives: `pakke.modul`, henter modulen `modul` fra pakken `pakke`. Det finnes mange ulike pakker man kan benytte i Python, og vi skal hovedsakelig bruke pakkene `numpy` (numerisk Python) og `matplotlib` her.

Disse to kommandoene for å importere hele pakken er ekvivalente:

```
1 import numpy
```

```
1 from numpy import *
```

Eller man kan bare importere det man trenger:

```
1 from numpy import pi
```

Noen foretrekker å bruke:

```
1 import numpy as np
```

der for eksempel for å bruke π skriver man i programmet `np.pi`.

Under følger et kort eksempel der vi importerer noen pakker, og bruker dem til en liten beregning av $y(x)$ for en gitt x -verdi.

Eksempel 0.8.1: Vi ønsker å beregne $y(x) = e^{-\pi x}$ for $x = 0.2$.

```
1 import numpy as np
2 x= 0.2
3 y = np.exp(-np.pi*x)
4
5 print('y er lik',y)
```

Eksempel 0.8.2: Man kan importere hele pakken eller deler av pakken:

```
1 import numpy as np          # Importerer hele pakken som får navnet np
2 # Vi vil bruke pakkens cosinus- og pi-funksjoner:
3
4 print("sinus av ",np.pi," er ", np.sin(np.pi))
5 print() # oppretter ekstra linje
6
7 # Vi kan ogsaa bare importere enkeltfunksjoner. Da kan vi kalle paa
  funksjonene uten aa angi pakkens navn.
8
9 from numpy import sin, pi    # importerer vi bare delene vi trenger.
10
11 print("sinus av ",pi," er ", sin(pi))    # Merk at vi ikke trenger
  pakkens adresse.
```

Oppg. 0.8.1: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

Eksempel 0.8.3: Vi vil regne ut $\sin(1) + \cos(4) + e^{-2}$.

```
1 import numpy as np
2 svar=np.sin(1)+np.cos(4)+np.exp(-2)
3 print(svar)
```

```
1 from numpy import exp, sin, cos
2 svar=sin(1)+cos(4)+exp(4)
3 print(svar)
```

numpy eller *math*-biblioteket?

numpy-biblioteket inneholder flerdimensjonale array- og matrise-datastrukturer, mens *math*-biblioteket brukes til å utføre matematikk på skalardata, for eksempel trigonometriske beregninger. Hvis man skal regne ut med kjente funksjoner som *cos*, kan man enten importere hele *numpy*-biblioteket eller bare importere det man trenger:

```
1 from numpy import exp, sin, cos, tan, arcsin, arccos, arctan, exp
2 svar=sin(1)+cos(4)+exp(4)
3 print(svar)
```

Husk at de trigonometriske funksjonene (*sin*, *cos* og *tan*) tar inn en vinkel i radianer, og inversfunksjonene (*arcsin*, *arccos* og *arctan*) returnerer en vinkel i radianer.

Oppg. 0.8.2: Regn ut disse v.h.a Python

- a) $\cos(2) + \tan(1.5)$
- b) $e^4 + \log(2) + \ln(0.1)$

pylab-biblioteket

I de nye matematikk-lærebøkene for videregående-trinnet brukes *pylab*-biblioteket, som har mange innbygde funksjoner. *pylab*-pakken inneholder både *numpy* og *matplotlib*.

```
1 from pylab import *
2 print(cos(1))
```

Noen foretrekker *numpy* og at man bruker

```
1 import numpy as np
```

der man for eksempel for å bruke *sin*-funksjonen skal skrive *np.sin(x)*. Fordi man har da et bedre oversikt hvor i programmet har brukt pakken.

Oppg. 0.8.3: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

numpy- eller *sympy*-biblioteket?

numpy-biblioteket gjør beregningsmatematikk, mens *sympy*-biblioteket gjør symbolsk matematikk. For eksempel for å derivere eller å integrere:

```
1 from sympy import *
2
3 x = Symbol('x')
4 svar1= diff(x**3, x)
5 svar2= integrate(svar1, x)
6 print(svar2)
```

Andre pakker (*Matplotlib*)

Vi vil ofte ha behov for å visualisere data. Da kan biblioteket *Matplotlib* hjelpe oss. Her skal vi bruke modulen *pyplot* (som er en del av biblioteket *Matplotlib*) for å tegne en graf:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3,3, num = 101)
5 f = x**3-3*x
6
7 plt.plot(x,f, '--')
8 plt.grid()
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'])
12 plt.savefig('MinPythonFigMappe/eksplot.png')
```

0.9 Indeksering

Vi lager indeksmengder med kommandoen `range`. Kommandoen `range(2, 5)` betyr heltallene fra og med 2 til, men ikke med, 5 (til og med 4).

<code>range(1,n+1)</code>	alle hele tall fra og med 1 til n
<code>range(1,n+1,m)</code>	alle hele tall fra og med 1 til n med m steglengde
<code>range(1,n+1,m)</code>	alle hele tall fra og med 1 til n med m steglengde

Hvis du utelater første argument i `range`, antar Python at første argument er 0. Dersom du legger til et tredje argument, angir dette steglengden i intervallet.

Kommando	Verdi
<code>range(0, 5)</code>	0, 1, 2, 3, 4
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(2, 5)</code>	2, 3, 4
<code>range(0, 5, 2)</code>	0, 2, 4
<code>range(10, 5, -1)</code>	10, 9, 8, 7, 6

0.10 Løkker (betingelser og gjentagelser)

En løkke brukes for beregninger eller handlinger som skal gjentas flere ganger. Man kan velge et gitt antall ganger eller at det skal gjentas inntil et mål (noe) er nådd. Fordelen med å bruke en løkke er at vi slipper å skrive mange setninger med programkode. Da blir koden enklere å feilsøke og rette, men den blir også mer kompakt, noe som gjør at det tar kortere tid å kjøre gjennom programmet.

if/elif/else-løkke

En if/elif/else-setning brukes når vi ønsker å utføre en blokk med kode når den gitte tilstanden er sant eller usant. Her er et eksempel:

```
1 a = float(input("Skriv inn a (et reelt tall) her: "))
2 if a > 2:
3     print("a er mer enn 2")
4
5 elif a < 2:
6     print("a er mindre enn 2")
7
8 else:
9     print("a er lik 2")
```

Ved å kjøre programmet og taste inn forskjellige verdier for a får vi:

```
1 Skriv inn a (et reelt tall) her: 4
2 a er mer enn 2
3 Skriv inn a (et reelt tall) her: 1
4 a er mindre enn 2
5 Skriv inn a (et reelt tall) her: 2
6 a er lik 2
```

Oppg. 0.10.1: Hvis -7 skrives inn når følgende programkoden kjøres, hva blir resultatet?

```
1 print('Skriv et tall')
2 a = int(input())
3 x=a
4 if x < 0:
5     x = -x
6 print('Absluttverdien til',a,' er', x)
```

Oppg. 0.10.2: Skriv en programkode for følgende:

- Les et tall fra skjermen, og sjekk om tallet er positivt, negativt eller null, og skriv resultatet på skjermen.
- Les et tall fra skjermen, og sjekk om tallet er partall eller oddetall, og skriv resultatet på skjermen.
- Les to tall fra skjermen og si om disse tallene har samme eller motsatt fortegn.

while-løkke

While-løkker brukes i tilfeller hvor man kan si at løkka skal utføres så lenge noe er sant.

```
1 i=0
2 while i < 3:
3     print("i =", i)
4     i += 1
```

Ved å kjøre programmet får man:

```
1 i = 0
2 i = 1
3 i = 2
```

Andre ganger vil vi gjøre bestemte utregninger så lenge en betingelse vi har satt på forhånd, er oppfylt. Vi skal vise bruken av denne type løkke ved hjelp av et eksempel. Legg merke til at $\text{range}(1, n+1)$ er alle hele tall fra og med 1 til n .

Eksempel 0.10.1: Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når: $a_1 = 1$ og $a_{n+1} = 0.25a_n + 2$. Hva konvergerer a_n når $n \rightarrow \infty$?

Løsning:

```
1 n = 1
2 for i in range(1,10):
3     0.25*n + 2
4     print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 2.666\dots$ når $n \rightarrow \infty$. $\diamond$

Eksempel 0.10.2: Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når: $a_1 = 1$ og $a_{n+1} = \frac{3}{4 - a_n}$. Hva konvergerer a_n når $n \rightarrow \infty$?

Løsning:

```
1 n = 1
2 for i in range(1,10):
3     n = 3/(4-n)
4     print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 1.0$ når $n \rightarrow \infty$. $\diamond$

Eksempel 0.10.3: Betrakt summen $\sum_{n=1}^{\infty} \frac{5}{n}$.

Finn ut hvor mange ledd som trengs for å få summen over 20.

Løsning:

```
1 n = 1 # Definerer n og Sum ved start
2 Sum = 0
3 # Vil regne ut summeringen til summen er mindre eller lik 20
4 while Sum <= 15:
5     Sum += 5/n
6     n += 1
7     print('Summen passerer 15 etter', n, 'ledd. Da er summen', Sum)
```

Ved å kjøre programmet får man:

```
1 Summen passerer 15 etter 32 ledd. Da er summen 20.136225977182594
```

Oppg. 0.10.3: Skriv kort hva følgende programkode utfører:

```
1 innlest = input("Hva er 49+35?")
2 tall = int(innlest)
3
4 if tall != 84:
5     while tall != 84:
6         innlest = input("Regn igjen!")
7         tall = int(innlest)
8     else:
9         print("Du klarte det!")
```

Oppg. 0.10.4: Skriv en programkode som spør hva summen til $52 + 19$ er, og leser inn svaret fra brukeren, og hvis svaret er feil, skriver ut *Feil. Regn igjen!*, og hvis det er rett, skriver ut *Du klarte det!*

Oppg. 0.10.5: Følgende programkode spør leseren om å skrive et partall, og deretter sjekker om dette tallet er partall, og bestemmer summen til alle partall til og med dette tallet.

```
1 tall = int(input("Skriv inn et partall: "))
2
3 if tall%2 == 1:
4     print("Tallet er ikke et partall!")
5 else:
6     sum = 0
7     i=0
8     while(i <= tall):
9         sum = sum + i
10        i = i+2
11
12 print(sum)
```

Skriv en programkode som gjør dette for oddetall.

Oppg. 0.10.6: Vi vil prøve å finne det minste heltallet x (heltall) for at $3^x > 81$.

Følgende program vil telle hvor mange ganger, må gange 3 med seg selv for å få mer enn 81.

```
1 x0= 1
2 b = 81
3 x = 0
4 x1 = x0
5
6 while x1 <= b:
7     x += 1
8     x1*= 3
9
10 print(f"Svaret er {x}.")
```

Skriv programmet og sjekk om du får svaret 4.

Modifiser programmet for å gjøre dette for ligningen $4^x = 1024$

for-løkke

for-løkken kan brukes til å gjenta en kodeblokk et kjent/begrenset antall ganger, eller gå gjennom en sekvens, element for element:

```
1 n=2
2 for i in range(1,n+1):
3     b=i**2+1
4     print(b)
```

Ved å kjøre programmet får man:

```
1 2
2 5
```

Legg merke til at $\text{range}(1, n+1)$ betyr indeksmengde fra og med 1 til og med n (og ikke til og med $n+1$).

Oppg. 0.10.7: Skriv hva resultatet av følgende programkode blir:

a)

```
1 i=0
2 for i in range(2):
3     print('A')
4     for i in range(3):
5         print('B')
```

b)

```
1 for x in [50, 60]:
2     for y in [1, 2, 3]:
3         print(x+y, end=' ')
```

c)

```
1 for x in [50, 60]:
2     print()
3     for y in [1, 2, 3]:
4         print(x+y, end=' ')
```

Oppg. 0.10.8: Hva får man av følgende programkoder? Skriv følgende i et script og kjør programmet.

a)

```
1 for i in range(0,10):
2     print(i, end=" ")
```

b)

```
1 for i in range(1,10):
2     print(i, end=" ")
```

c)

```
1 for i in range(0,10,2):
2     print(i, end=" ")
```

d)

```
1 for i in range(10,1,-1):
2     print(i, end=" ")
```

Oppg. 0.10.9: Skriv en programkode for følgende: Det dukker på skjermen: *Hei, vil du fortsette med programmet?* Hvis svaret er nei, avsluttes programmet.

Oppg. 0.10.10: Skriv en programkode som regner ut summen til de første 5 leddene i en geometisk rekke med førsteledd 3 og kvotient 2, og skriver ut på skjermen summen.

Oppg. 0.10.11: Skriv kort hva følgende program utfører:

```
1 for i in range(1,10):  
2     for k in range (1,5):  
3         print(i,k)
```

Oppg. 0.10.12: Skriv en programkode som regner ut summen til de første 7 leddene i en aritmetisk rekke med førsteledd 5 og kvotient 3, og skriver ut summen på skjermen.

Oppg. 0.10.13: a) Skriv hva resultatet til følgende programkode blir:

```
1 for i in range(1,2):  
2     for k in range (1,3):  
3         print(i,2*k)
```

b) Skriv også hva `range(5)` sender inn i løkken.

Skriv hva resultatet til programkoden blir

```
1 for i in range(5):  
2     if i == 3:  
3         print(i, end="")  
4     else:  
5         print(2*i, end="")
```

c) Skriv også hva `while i < 4` sender inn i løkken.

```
1 i = 1  
2 while i < 4:  
3     print(i)  
4     if i == 3:  
5         break  
6     i += 1
```

Hvordan ville du endre løkken i del c) hvis du ville bruke `range`-syntaksen?

d)

```
1 = 1  
2 while i < 5:  
3     print(i)  
4     i += 1  
5 else:  
6     print("i er ikke lenger mindre enn 5")
```

e) Hva skriver følgende programkode ut?

```
1 r = 4  
2 for i in range(1, r + 1, 1):  
3     for j in range(1, i + 1):  
4         print(j, end=' ')  
5     print()
```

f) Hva skriver følgende programkode ut?

```

1 n=4;
2 for i in range(n):
3     for j in range(i):
4         print ('o ', end=" ")
5     print(',')
6
7 for i in range(n,0,-1):
8     for j in range(i):
9         print('* ', end=" ")
10    print(',')

```

g) Skriv en programkode som får følgende resultat:

```

1 *
2 * *
3 * * *
4 o o o o
5 o o o
6 o o
7 o

```

Oppg. 0.10.14: Følgende programkode

```
1 n = 4
2 k = 4
3 for i in range(0,n+1):
4     for j in range(k-i,0,-1):
5         print(j,end=' ')
6     print()
```

Skriver ut:

```
1 4 3 2 1
2 3 2 1
3 2 1
4 1
```

a) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
```

b) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 4 3 2 1
2 3 2 1
3 2 1
4 1
```

c) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 2 1
3 3 2 1
4 4 3 2 1
```

d) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
5 1 2 3 4 5
```

0.11 Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler. I programmering er funksjoner en måte å samle en beregning utført av en eller flere variabler i en sammensatt bolk, såkalte *funksjoner*, slik at det blir lettere å skrive, modifisere, lese koden, bruke andre steder i programmet eller eventuelt i andre program. Syntaksen for funksjoner er kontrollordet *def*, deretter navnet på funksjonen etterfulgt av to parenteser som inneholder variabel (variabler), som inngår funksjonen og et kolon (:).

```
1 def funksjonsnavn(variabler):
2     kodeblokk
```

Husk å trykke tab (knappen rett over caps lock) for linjene i kodeblokken for å få innrykk. Funksjoner gjør programmeringen lettere.

Litt om funksjoner: Hva og hvorfor?

Funksjoner i Python (**Hva?**):

- ◇ består av en gruppe kodelinjer som utfører en spesifikk oppgave
- ◇ kan minne om matematiske funksjoner, men også Python-funksjoner
 - kan inneholde alle slags handlinger, ikke bare matematikk
 - kan returnere et svar (som `mat.funk.`), men kan også la være
 - kan ta argumenter, men kan også la være

Hvorfor bruke funksjoner?

- Dele et større program opp i mindre deler
 - lettere å lese, skrive og vedlikeholde kode
 - Lettere å forstå, enklere å teste, mer effektivt samarbeid
 - gir ryddigere og enklere kode
- Gjenbruk av kode
 - Vi kan bruke Pythons egne funksjoner, f.eks. `print()`, `round()`
 - Vi kan selv definere nye funksjoner og
 - * bruke dem flere steder i ett program,
 - * eller i flere ulike programmer

Her tar vi et eksempel på hvordan vi kan lage en funksjon som kan brukes flere ganger i programkoden. For eksempel kan vi skrive en funksjon som kan regne ut volumet av en sylinder med radius r og høyde h .

```
1 import numpy as np
2
3 def Volumsyylinder(r, h):
4
5     volum=np.pi*(r**2)*h
6     return volum
7
8 V=Volumsyylinder(3,5)
9 print(V)
```

I scriptet regner vi ut `Volumsyylinder(3,5)` volumet av sylindere med radius = 3 og høyde = 5. Funksjoner begynner med kontrollordet `def`, etterfulgt av navnet på funksjonen, parenteser og et kolon, der input-parameterne er gitt i parentes. Dersom du vil at funksjonen skal kunne ta ett eller flere argumenter, skriver du dem inn i parentesene. I dette eksemplet har vi deklart variablene r (radius) og h (høyde), og det betyr at disse variablene inngår i funksjonen og de må defineres som et tall når funksjon skal regne ut volumet.

Legg merke til at etter funksjonshodet `def Volumsyylinder(r, h):` kommer resten av koden i linjene under med innrykk (bruke TAB-tasten). Så utføres utregningen av volumet, før vi til slutt har med `return volum` som skal returnere svaret funksjonen gir, og vi kan bruke videre i koden. Istedenfor `Volumsyylinder(3,5)`, kunne vi bruke:

```
1 a=3
2 b=5
3 V=Volumsyylinder(a,b)
4 print(V)
```

Ellers variablene kan leses inn i programmet fra skjermen:

```
1 r= float(input('Skriv inn radiusen til en cylinder: '))
2 h= float(input('Skriv inn høyden til en cylinder: '))
3 V=Volumsyylinder(r,h)
```

Eksempel 0.11.1: Her er 3 eksempler med funksjoner uten argument, med argumenter, og med matematiske beregninger og hvordan disse kan brukes (*kalles*).

```
1 def minfunk():
2     print("Her er en funksjon!")
3
4 def minfunkmedarg(Navn, Hilsen):
5     print("Hei %s, %s" %(Navn,Hilsen))
6
7 def minfunktall(x,y):
8     return x**2 + y**2
9
10 minfunk()
11
12 minfunkmedarg("Kari Olsen","Ha en fin dag!")
13
14 a = minfunktall(1,2)
15 print(a)
```

Ved å kjøre programkoden får vi:

```
1 Her er en funksjon!
2 Hei Kari Olsen, Ha en fin dag!
3 5
```

Eksempel 0.11.2: For eksempel setter følgende program opp funksjonen $f(x) = x^3 + 3x$ og lister opp på skjermen x og $f(x)$ under x og $f(x)$:

```
1 def f(x):
2     return x**3 + 3*x
3
4 print(f'x\t f(x)')
5
6 for x in range(10):
7     print(f'{x}\t {f(x)}')
```

Ved å kjøre programmet, ser resultatet slik ut:

```
1 x  f(x)
2 0  0
3 1  4
4 2  14
5 3  36
6 4  76
7 5  140
8 6  234
9 7  364
10 8  536
11 9  756
```

Eksempel 0.11.3: Her tas et eksempel med en funksjon som beregner skalarproduktet til to vektorer:

```
1 def skalarprod(a, b):
2     p= 0
3     for i in range(len(a)):
4         p += a[i]*b[i]
5     return p
6 # Prøv funksjonen:
7 a = [1, 2, 3]
8 b = [0, 3, -1]
9 ab = skalarprod(a,b)
10 print(f'Skalarproduktet av a og b er {ab:.2f}')
```

Ved å kjøre programmet, får man:

```
1 Skalarproduktet av a og b er 3.00
```

Opppg. 0.11.1: Hva får man ved å kjøre følgende program:

```
1 def skalarprod(m, n):
2     d= 0
3     for i in range(len(m)):
4         d += m[i]*n[i]
5     return d
6 # Prøv funksjonen:
7 m = [-1, 3, 0]
8 n=  [2, 1, 5]
9 mn = skalarprod(m,n)
10 print(f'Skalarproduktet av m og n er {mn:.1f}.')
```

- Oppg. 0.11.2:** a) Skriv et program som setter opp en funksjon for arealet til en sirkel med radius r og regn ut arealet til en sirkel med radius 5.
- b) Skriv et program som leser inn radiusen til en sirkel fra brukeren og regner og skriver ut arealet til en sirkel.

Oppg. 0.11.3: Summene nedenfor beregner $\ln x$ når n velges stor nok.

$$s_1(x, n) = \sum_{i=1}^n (-1)^{i-1} \frac{1}{i} (x-1)^i, \quad 0 < x \leq 2$$

$$s_2(x, n) = \sum_{i=1}^n \frac{1}{i} \left(1 - \frac{1}{x}\right)^i, \quad x > \frac{1}{2}$$

Vi ønsker å lage en programkode som hjelper til å beregne disse summene for en gitt x og n . Vi vil studere estimeringsavviket (feilen) man får ved å bruke summen $s(x, n)$ for en gitt x , istedenfor $\ln x$. Estimeringsfeilen for s_1 kan studeres ved to forslag:

- 1) Å bestemme feilen ved å regne ut differansen mellom den eksakte verdien og den estimerte verdien: $\ln(1.5) - s(1.5, n)$ (naiv metode).
- 2) Å Regne ut det først leddet i summen s_1 som ikke er tatt med i beregningen av summen, som kan kalles det numeriske estimeringsavviket. To programkoder er skrevet slik:
For s_2 kan man se nærmere på 1).

```

1 # Programkode 1
2 import numpy as np
3
4 def S1(x, n):
5     S1 = 0
6     for i in range(1, n+1):
7         S1 += (-1)**(i-1) * (x-1)**i / i
8     return S1
9
10 def E1(x, i):
11     E1 = 0
12     E1 = (x-1)**i / i
13     return E1
14
15 x = 1.5
16 n=100
17 e=0.000001
18
19 for i in range(1, n+1):
20     EstimertLog1 = S1(x, i)
21     Estimeringsfeil1 = E1(x, i)
22     EksaktLog = np.log(x)
23     NaivEstimeringsfeil = abs(EstimertLog1 - EksaktLog)
24     if NaivEstimeringsfeil < e:
25         break
26
27 print(f' Ved å bruke {i} første ledd: Estimert= {EstimertLog1},
      Eksaktverdi= {EksaktLog}, Naiv estimeringsfeil = {
      NaivEstimeringsfeil}, Estimeringsfeil = {Estimeringsfeil1}')

```

```

1 # programkode 2
2 import numpy as np
3
4 def S2(x, n):
5     S2 = 0
6     for i in range(1,n+1):
7         S2+= (1-1/x)**i/i
8     return S2
9
10 x = 1.5
11 n=100
12 e=0.000000001
13
14 for i in range(1,n+1):
15     EstimertLog2 = S2(x,i)
16     EksaktLog = np.log(x)
17     NaivEstimeringsfeil = abs(EstimertLog2 -EksaktLog )
18     if NaivEstimeringsfeil< e:
19         break
20 print(f'Ved å bruke {i} første ledd: Estimert= {EstimertLog2},
      Eksaktverdi= {EksaktLog }, Naiv estimeringsfeil = {
      NaivEstimeringsfeil}' )

```

Ved å kjøre programmeringskoder får man disse to visningene på skjermen (som ikke nødvendigvis er skrevet i samme rekkefølge som programkodene):

```

1 Ved å bruke 15 første ledd: Estimert= 0.4054657568451514, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.487369870189497e-07,
    Estimeringsfeil = 2.0345052083333333e-06

```

```

1 Ved å bruke 16 første ledd: Estimert= 0.40546510744296105, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.652033368581556e-10

```

- Forklar hvilken programkode som kan brukes til å beregne $\ln(0.25)$, $\ln(1.75)$ og $\ln(2.25)$?
- Hvordan kan man forbedre presisjonen og få bedre nøyaktighet?

Oppg. 0.11.4: Arealet til en trekant med sidelengdene a , b og c er gitt ved:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

der $s = \frac{1}{2}(a+b+c)$.

Skriv en programkode som beregner arealet til en trekant for en gitt a , b og c .

Oppg. 0.11.5: Kinetisk energi for en partikkel med masse m [kg] som har hastighet v [m/s] er gitt ved:

$$E = \frac{1}{2}mv^2$$

Skriv en programkode som definerer en funksjon for kinetisk energi, regner ut den kinetiske energien for en partikkel som har massen $m = 0.0001$ [kg] og hastigheten $v = 8$ [m/s].

Å kalle inn en funksjon i en annen funksjon

Eksempel 0.11.4: Her tas et eksempel med en funksjon som finner det største tallet blant tre tall ved å kalle en funksjon i en annen funksjon

```
1 def maksavto( x, y ):
2     if x > y:
3         return x
4     return y
5 def maksavtre( x, y, z ):
6     return maksavto( x, maksavto( y, z ) )
7 print(maksavtre(2, -3, 7))
```

Eksempel 0.11.5: Her tas et eksempel med en funksjon som kalles inn i en annen funksjon. Skriv en programkode for en funksjon $f_1(x)$ som multipliserer argumentet x med 4 og en annen funksjon $f_2(x)$ som legger til tallet 5 til argumentet x . Utvid programkoden til å regne ut $f_1(f_2(2))$ og $f_2(f_1(2))$. Ellers formuleres dette slik også:

Gitt $f(x) = 4x$ og $g(x) = x + 5$. Skriv disse som funksjoner i programkode, og bruk programmet til å regne ut $f(g(2))$ og $g(f(2))$.

```
1 def f1(x):
2     y= 4*x
3     return y
4
5 def f2(x):
6     y=x+5
7     return y
8
9 print(f1(f2(2)))
10 print(f2(f1(2)))
```

Svaret ved å kjøre programmet blir:

```
1 28
2 13
```

Sjekker: $f_1(f_2(2))$, der $f_2(2) = 2 + 5 = 7$ og $f_1(f_2(2)) = f_1(7) = 4 \cdot 7 = 28$.

Sjekker $f_2(f_1(2))$, der $f_2(f_1(2)) = f_1(2) = 4 \cdot 2 = 8$ og $f_2(8) = 8 + 5 = 13$. ◇

Oppg. 0.11.6: Hva skrives ut som resultat av kjøring av følgende programkode?

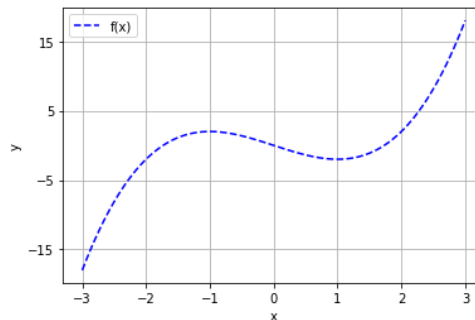
```
1 def printut(a):
2     b = 3*a
3     print('b = %g' % b)
4 x= 5
5 print('x = %g' % x)
6 printut(x)
```

0.12 Plotting i Python

Vi vil ofte ha behov for å visualisere data. Da biblioteket *Matplotlib* hjelpe oss. Her skal vi bruke modulen *pyplot* (som er en del av biblioteket *Matplotlib*) for å tegne en graf:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3,3, num = 101)
5 plt.yticks([-15,-5,5,15])
6 f = x**3-3*x
7
8 plt.plot(x,f, 'b--')
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'], loc='best')
12 plt.grid()
13 plt.show
14 plt.savefig('MinPythonFigMappe/eksplot.png')
```

Kjører vi dette scriptet, vil følgende figur plottes *og lagres*.



Elementene vi har tatt i bruk, her er følgende:

- `plt.plot(x,f, 'b--')` tegner grafen til funksjonen og tar inn x -verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'`, som skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med blåfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
- Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
- `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y -aksen.
- `plt.grid()` legger på et gitter bak grafen.
- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best'`): plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsollvinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

KONTROLL SPØRSMÅL. Hvordan kan du endre graftykkelsen og farger på grafen?
Hvordan kan du endre programkoden ved å skrive f ved hjelp av funksjonssyntaksen? ■

Oppg. 0.12.1: Følgende programkode er gitt:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-2,2, num = 101)
5 plt.yticks([-10,-5,5,10])
6 f = x**5-5*x**2
7
8 plt.plot(x,f, 'g--')
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'], loc='best')
12 plt.grid()
13 plt.show
14 plt.savefig('MinPythonFigMappe/EksGraf.png')
```

- a) Forklar hva programmet utfører linje for linje.
- b) Gjør de nødvendige endringene for at programkoden tegner grafen til: $g(x) = x^3 - 5x$ når $-2 \leq x \leq 2$.

Oppg. 0.12.2: La f være funksjonen gitt ved $f(x) = x^3 + 6x - 3$.

- a) Skriv en programkode som setter opp verditabell for x -verdiene mellom -2 og 2 (med å dele opp intervallet med steglengde = 0.01), og tegner grafen til f .
Utvid programmet til å tegne grafen til funksjonen
- b) Utvid programmet ditt til å la brukeren velge minste og største verdi for x og trinnene, i tabellen.

Oppg. 0.12.3: Skriv en programkode som tegner grafen til følgende funksjoner i samme koordinatsystem:

$$f(x) = \sin(2x - \pi)$$

$$g(x) = \cos(2x + \pi)$$

i intervallet $[-2\pi, 2\pi]$.

Skisser også $h(x) = \cos(2x + \frac{\pi}{2})$. Hva kan vi si om f og h ?

Utvid programmet for at den kan tegne grafen til funksjonene:

$$f(x) = c + a \sin\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

$$g(x) = c + a \cos\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

gitt $c = 60$, $a = 120$, $T = 365$, $k = 2$ og $d = 182.5$.

0.13 Lister

Lister er mengden av elementer atskilt med komma mellom hakeparenteser, for eksempel `[1, 2, 3]` og `['Klara', 'Augusta', 7]`. Lister kan brukes til å lagre objekter underveis i programmet. For å lage en liste, oppretter man en tom liste ved å tilordne verdien `[]` til en variabel, for eksempel `liste=[]` eller `xverdier=[]`. Her er variabelnavnene «liste» og «xverdier».

`liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']:`

Kommando	Resultat	Forklaring
<code>liste[0]</code>	<code>'a'</code>	Første element er nr. 0
<code>liste[0:3]</code>	<code>['a', 'b', 'c']</code>	Tar ikke med nr. 3 (fjerde)
<code>liste[:3]</code>	<code>['a', 'b', 'c']</code>	Tar med element på plass nr. 0, 1 og 2
<code>liste[-1]</code>	<code>'h'</code>	Siste element er nr. -1
<code>liste[3:6]</code>	<code>['d', 'e', 'f']</code>	Fra og med nr. 3 til nr. 6
<code>liste[3:-1]</code>	<code>['d', 'e', 'f', 'g']</code>	Fra nr. 3, tar ikke med siste element
<code>liste[3:]</code>	<code>['d', 'e', 'f', 'g', 'h']</code>	Fra nr. 3, tar ikke med siste element

For å legge til elementer i en liste, brukes kommandoen «*append*», og listevariabelen brukes som forstavelse, atskilt med et punktum. I koden nedenfor opprettes det først en tom liste i linje 1, og i de tre neste linjene legger vi inn tallene 3 og 1 og bokstaven «c». I linje 5 skriver vi lista til skjerm.

```
1 liste = []
2 liste.append(3) # legger tallet 3 inn i lista
3 liste.append(1) # legger tallet 1 inn i lista
4 liste.append('c') # legger bokstaven c inn i lista
5 print(liste)
```

Når vi kjører programmet, får vi dette resultatet:

```
1 [3, 1, 'c']
```

Kontrollordet *len* kan gi oss lengden på lista, *len(liste)*, altså hvor mange elementer den inneholder:

```
1 liste=[1,7,2,4,3,3,5,6] # en liste med sju elementer
2 print(len(liste))
```

Svaret bli da 8.

Eksempel 0.13.1: Skriv en programkode som gir `[0, 1, 'e', 'p']`.

```
1 liste = []
2 liste.append(0)
3 liste.append(1)
4 liste.append('e')
5 liste.append('p')
6 print(liste)
```

Man kan også legge til flere elementer samtidig med programkoden

```
1 liste = []
2 liste.extend([0,1,'e','p'])
```

Oppg. 0.13.1: Hva blir resultatet av følgende?

a) Programkoden:

```
1 A=[1,2,3,4,5]
2 B=[2,4,6,8,10]
3 C=A+B
4 print(C)
```

b) Programkoden:

```
1 A=[1,2,3]
2 A+= [4]
3 A = A + [5]
4 print(A)
```

c) En liste er gitt som: Liste=[1, 2, 3, 4, 5, 6] Bruk en for-løkke til å regne ut produktet av alle elementene i liste.

Oppg. 0.13.2: La f være funksjonen gitt ved $f(x) = -0.2x^2 + 1$. Skriv en programkode som setter opp x -verdiene, mellom -1 og 2 med steglengde 0.25 , og y -verdiene til f inn i to lister.

Oppg. 0.13.3: La f være funksjonen gitt ved $f(x) = -x^3 + x + 2$. Skriv en programkode som setter opp x -verdiene, mellom -2 og 3 med steglengde 0.25 , og y -verdiene til f inn i to lister.

0.14 Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
<code>f = open('filnavn')</code>	Åpner en fil, returnerer filreferanse
<code>f = open('filnavn','tilgang')</code>	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
<code>f.read()</code>	Returnerer hele innholdet av filen
<code>f.read(n)</code>	Returnerer <i>n</i> karakterer av innholdet
<code>f.readline()</code>	Returnerer neste linje (før / <i>n</i>)
<code>f.readlines()</code>	Returnerer hele innholdet av fila som en liste
<code>f.write(s)</code>	Skriver strengen <i>s</i> til fil
<code>f.writelines(liste)</code>	Skriver innholdet av liste av strenger til fil
<code>f.seek(offset fra hvor)</code>	Forflytter filpekeren (index) i filen
<code>f.tell()</code>	Returnerer posisjon til filpekeren i filen
<code>f.close()</code>	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

```
1 f = open('datafil.txt','r') # aapner fil for lesing
2 f = open('datafil.txt','w') # aapner fil for skriving
3 f = open(r'C:\User\files\fil.txt','a' )
4 # r først inni parentesen fo at \ ikke tolkes som spesielle tegn
5 # deretter leser filen fil.txt fra stien.
```

Eksempel 0.14.1:

```
1 f = open('fil1.txt', 'a')
2 f.write('Denne skrives i filen!')
3 print()
4 f.close()
5 print()
6 #åpen og les fra filen
7 f = open('fil1.txt', 'r')
8 print(f.read())
```

0.15 Laste inn, håndtere og plotte datasett

Vi har lært så langt hvordan vi bruke matematiske funksjoner til å regne funksjonsverdier og plotter disse. I naturvitenskapen vil ofte data / tall / målte verdier være hentet fra eksperimenter og observasjoner. Det kan ofte være store datasett med lange lister med tall. Vi skal nå hvordan vi kan laste inn data fra harddisken til vår pc eller eksterne servere og bruke innbygde Python-funksjoner og pakker for å håndtere slike lister.

pandas-biblioteket

pandas-bibliotet brukes til å analysere data.

```
1 import pandas as pd
2 df = pd.read_csv('data.csv')
```

I det neste eksempelet skal vi lese tid–temperatur-målinger, der vi bruker følgende syntakser for å åpne datafilen *tekopp-temp.csv* der i regnearket hver tekstlinje er en rad og verdiene for de ulike søylene er separert med komma (;). For å indikere dette, bruker man *sep=';'* og for å tolke desimaltall bruker man *decimal='.'*.

```
1 import pandas as pd
2 df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv',
3                 sep=';', decimal='.', comment='#', encoding='latin-1')
4 tid = df['Tid'].tolist()
5 temperatur = df['Temperatur'].tolist()
```

0.16 Modellering: tekopp-temperatur

Prosjekt: Modellering av tekopp-temperatur

Vi ønsker å studere endringen i tekopp temperatur i løpet av en time (60 min).

Læringsmål

- Utforske sammenhengen mellom konstant prosentvis endring, vekstfaktor og eksponential-funksjoner
- Bruke funksjoner i modellering og argumentere for framgangsmåter og resultater,
- Modellere situasjoner knyttet til reelle datasett, presentere resultatene og argumentere for at modellene er gyldige
- Utforske matematiske egenskaper og sammenhenger ved å bruke programmering
- Tolke og bruke funksjoner i matematisk modellering og problemløsning.

Læringsutbytte

- Studentene skal forstå hvordan kurven som beskriver at temperaturfallet avtar raskest i starten for så å nærme seg temperaturen i rommet som er en asymptote til kurven.
- Studentene skal se at lineær kurvetilpasning ikke kan dekke behovet, spesielt i de første minuttene.

Programmeringsdidaktikk

Teori (det tas litt teori på tavla før eksperimentet begynner):

Newtons avkjølingslov: dette forsøket prøver å lage en matematisk modell som beskriver hvordan temperaturen endres i et objekt (en kopp med vann) som kjøles ned av omgivelsene. Funksjonsuttrykket:

$$y = f(t) = at + b$$

representerer en lineær funksjon som kan beskrive lineær sammenheng mellom to størrelser. Funksjonsuttrykket:

$$y = f(x) = c + ae^{bt}, \quad \text{der } b < 0$$

representerer en eksponentialfunksjon. Ved start ($t=0$) har funksjonen verdien $y = c + a$, og synker eksponentiell mot $y = c$ når tiden (t) går.

Utstyr:

- 10 eller flere kopper (samme type)
- Vannkoker
- En vekt
- En klokke
- Ett eller flere termometer(e)

Tidsbruk:

Forarbeid: 15-20 min og elevene jobber ca. 60 min, 10-15 min refleksjon over detaljene i prosjektet med åpen samtale og eventuelt kontrollspørsmål.

Gjennomføring:

Kok opp tilstrekkelig vann til å fylle opp alle koppene. Sett koppene på vekta etter tur og fyll opp med like mye av det varme vannet. Mål temperaturen med jevne mellomrom i løpet av en time (3600 sek), for eksempel etter 5 min (300 sek), 10 min (600 sek), ... og noter resultatet i tabellen.

Tid (sek)	Temp (Celsius)

For eksempel (data hentet fra time-temp.pdf (southampton.ac.uk))

Tid (sek)	Temp (Celsius)
0	81.50
310	62.50
620	45.70
910	36.80
1220	29.40
1810	25.30
2550	24.20
3100	23.30
3620	22.50

Eksempel 0.16.1: I dette eksempelet skal vi nå se på filen ‘tekopp-temperatur.csv’. Hver linje i filen er en temperatur-måling i en tekopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 grader celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

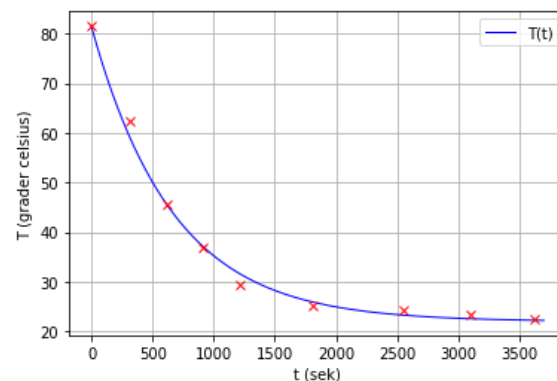
$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i grader celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv',
5                 sep=';', decimal='.', comment='#', encoding='latin-1')
6
7 tid = df['Tid'].tolist()
8 temperatur = df['Temperatur'].tolist()
9 tstart = 0
10 Tstart = temperatur[0]
11 tslutt = 3700
12 deltat = 0.5
13 To = 22 # Rom temperatur
14 k = .0015 # proporsjonalitets koeff.
15 tverdier = []
16 Tverdier = []
17 t = tstart
18 T = Tstart
19 while t <= tslutt:
20     tverdier.append(t)
21     Tverdier.append(T)
22     t = t + deltat
23     deltaT = -deltat * k * (T - To)
24     T = T + deltaT
25
26 plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
27 plt.plot(tid, temperatur, 'x', color='red')
28 plt.grid()
29 plt.xlabel('t (sek)')
30 plt.ylabel('T (Grad celsius)')
31 plt.legend(loc='best')
32 plt.show()

```



0.17 Oppsummering

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
-	Subtraksjon	$5 - 2$	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	$5 / 2$	2.5
//	Heltalldivisjon	$13 // 3$	4
**	Eksponent	$2 ** 3$	8
÷	Modulo	$13 \div 4$	1

Datatyper

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	{1, 3, 5}
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Å lese input fra brukeren og utskrift av tekst og variabel

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra tastaturet, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```
1 print('Her leses inn variablene')
2 a = int(input("Skriv inn a (et heltall) her: "))
3 b = float(input("Skriv inn b (et reelt tall) her: "))
4 alder=input("Skriv inn alderen din:")
5 print(f'Du er {alder} år.')
```

Andre tegn og syntaks

Tegn	betydning
==	lik
!=	ikke lik
>	mer enn
>=	mer eller lik (minst)
<	mindre enn
<=	mindre eller lik (høyst)

Importerings av ulike pakker

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Man kan enten importere hele biblioteket eller det man trenger fra biblioteket.

```
1 from numpy import *
2 from numpy import pi sin cos
```

Noen kommandoer

<code>print ()</code>	Skriv på skjerm
<code>range(1,n+1)</code>	Alle hele tall fra og med 1 til n
<code>range(1,n+1,m)</code>	Alle hele tall fra og med 1 til n med m steg lengde
<code>range(1,n+1,m)</code>	Alle hele tall fra og med 1 til n med m steg lengde
<code>if ... elif ... else</code>	Vilkår, if-test.
<code>while</code>	Løkke som kjører til testen er usann.
<code>for</code>	Løkke som kjører et på forhånd kjent antall ganger.
<code>import matplotlib.pyplot as plt</code>	Importerer selektivt pyplot-modulen til matplotlib som <i>plt</i>
<code>plt.plot()</code>	Tegner grafen til to lister.

Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler.

```
1 def funksjonsnavn(variabler):
2     kodeblokk
```

- Første linje kalles *funksjonshode*, som består av **def**, fulgt av navnet på funksjonen i parenteser og variabler, som inngår i funksjonen et kolon :
- Resten av koden kalles kodeblokk, som hører til funksjonen som må tas med.

Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
<code>f = open('filnavn')</code>	Åpner en fil, returnerer filreferanse
<code>f = open('filnavn','tilgang')</code>	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
<code>f.read()</code>	Returnerer hele innholdet av filen
<code>f.read(n)</code>	Returnerer <i>n</i> karakterer av innholdet
<code>f.readline()</code>	Returnerer neste linje (før / n)
<code>f.readlines()</code>	Returnerer hele innholdet av fila som en liste
<code>f.write(s)</code>	Skriver strengen <i>s</i> til fil
<code>f.writelines(liste)</code>	Skriver innholdet av liste av strenger til fil
<code>f.seek(offset fra hvor)</code>	Forflytter filpekeren (index) i filen
<code>f.tell()</code>	Returnerer posisjon til filpekeren i filen
<code>f.close()</code>	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

0.18 Fasit for programmeringsoppgaver i kapittel 1

Seksjon 1.12

Se fasit - 1.12.1

Oppg. 0.18.1: Oppgave 1.12.1

I teksten er det en programkode som regner ut $x + y$.

- Tast inn dette programmet, og kjør det.
- Gjør om på programmet slik at det i stedet regner ut $x - y$, og kjør det.
- Gjør om på programmet slik at det i stedet regner ut $x \cdot y$, og kjør det. Gangetegn skrives `*` i Python.
- Gjør om på programmet slik at det regner ut $y : x$, altså y del på x , og kjør det. Deletegn skrives med skråstrek i Python, altså skriver du y/x .
- Gjør om på programmet slik at det i stedet regner ut $(x + y)^2$, og kjør det. Å opphøye skrives `**` i Python, så dette uttrykket kodes $(x + y) ** 2$.
- Gjør om på programmet slik at det i stedet regner ut $\sqrt{x + y}$, og kjør det. Kvadratroten skrives `sqrt` i Python, så dette uttrykket kodes `sqrt(x + y)`.

Se fasit - 1.12.2

Oppg. 0.18.2: Oppgave 1.12.2 I teksten er det et program som skriver ut 3-gangen feil ved å bruke uttrykket $n + 3$.

- Tast inn dette programmet, og kjør det. Merk at linjene inne i while-løkken automatisk rykkes inn. Dette markerer at disse linjene er det som skal gjøres inne i løkken.
- Erstatt $n + 3$ med $3 * n$, og kjør programmet igjen. Skrives 3-gangen riktig ut nå?

Se fasit 1.12.3

Oppg. 0.18.3: Oppgave 12.1.3 I denne oppgaven skal vi jobbe videre med programmet i forrige oppgave.

- Gjør om programmet slik at det i stedet skriver ut de 100 første tallene i 4-gangen.
- Gjør om programmet slik at det i stedet skriver ut de 10 første oddetallene.
- Gjør om programmet slik at det i stedet skriver ut de 10 første kvadrattallene, altså 1, 4, 9, 16, 25, 36, 49, 64, 81, 100.

Se fasit 1.12.4

Oppg. 0.18.4: Oppgave 1.12.4 La T_n være kvadratet av oddetall n . Da er $T_1 = 1^2 = 1, T_2 = 3^2 = 9, T_3 = 5^2 = 25, T_4 = 7^2 = 49$, og så videre. Finn en formel for T_n , og test den ved å lage et program som bruker formelen til å skrive ut kvadratet av de 100 første oddetallene.

Se fasit 1.12.5

Oppg. 0.18.5: Oppgave 1.12.5 I hver deloppgave ser du starten på en tallfølge. Finn et mønster i tallfølgen, og lag en formel for T_n som stemmer med mønsteret ditt. Test formelen din ved å lage et program som skriver ut T_n fra $n = 1$ opp til $n = 10$.

- a) 3, 6, 9, 12, 15, ...
- b) 10, 20, 30, 40, 50, ...
- c) 11, 21, 31, 41, 51, ...
- d) 11, 22, 33, 44, 55, ...

Se fasit 1.12.6

Oppg. 0.18.6: Oppgave 1.12.6 Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når $a_1 = 1$ og $a_{n+1} = 0.5a_n + 1$ for $n \geq 1$.

Se fasit 1.12.7

Oppg. 0.18.7: Oppgave 1.12.7 Betrakt summen

$$\sum_{n=1}^{\infty} \frac{2}{n}$$

Skriv et program som kan brukes til å avgjøre hvor mange ledd som trengs for å få summen over 15.

Blandede oppgaver

Fasit - Oppgave 34

Oppg. 0.18.8: Oppgave 34 Ulike materialer har ulike massetetthet. Massetettheten er definert som masse delt på volum, oftest i kg/m^3 .

Materialer	Polystyren (lav tetthet)	Kork	Rhenium	Platinum
Massetetthet (i kg/m^3)	20	220	21020	21450

Tabell 1: Massetettheten til ulike materialer

En kube veier 858 g og har volum 40 cm^3 . Skriv et program som finner massetettheten til kubens, og bruk resultatet til å finne hvilket materiale kubens består av. Kubens består av et av materialene i tabellen over. Det kan hende at svaret ditt ikke vil være den eksakt lik en av de gitte massetetthetene, siden massetetthet i seg selv er et gjennomsnittlig mål. Du kan velge materialet som har massetetthet som er nærmest ditt resultat.

Se fasit - Oppgave 35

Oppg. 0.18.9: Oppgave 35 Newtons gravitasjonslov beskriver hvordan gravitasjonen som en tiltrekningskraft virker mellom to legemer:

$$F = G \frac{m_1 m_2}{r^2}$$

der m_1 og m_2 er massene til de to legemene som tiltrekker hverandre, og r er avstanden mellom dem. Konstanten G er gravitasjonskonstanten, som har følgende verdi:

$$G = 6.674 \times 10^{-11} \text{ m}^3 \text{ kg}^{-2} \text{ s}^{-1}$$

Vi ser på et legeme med masse $M = 3 \text{ kg}$ som blir påvirket av N legemer med masse $m_1, m_2, \dots, m_N$. Det i -te legemet har masse $m_i = \frac{i}{6} + 2 \text{ kg}$, og har avstand $r_i = \sqrt{\left(\frac{i}{4}\right)^2 + 10}$ meter fra legemet med masse M . Antall legemer er $N = 10$.
Skriv et program som beregner den totale kraften $\sum_{i=1}^N F_i = F_1 + F_2 + \dots + F_N$ som virker på legemet med masse M .

Kapittel 1

Funksjoner

Når man bruker programmering, for eksempel i matematikk, er det kanskje noen kodesnutter som går igjen i flere av scriptene vi lager. Vi kan for eksempel tenke oss at vi i løpet av et semester skal bruke abc-formelen mange ganger, og i stedet for å måtte skrive den samme koden hver gang, lar Python oss lett kunne definere denne funksjonen én gang, for så å bruke den igjen senere. I neste delseksjon kommer forklaringen på hvordan vi kan gjøre dette.

Oppg. 1.0.1: En sirkel C kan beskrives ved å oppgi sentrum $S(a, b)$ og radius r .

- a) Beskriv en algoritme som du kan bruke til å avgjøre om et gitt punkt $P(x, y)$ ligger på, inni eller utenfor sirkelen C .
- b) Skriv en kode basert på algoritmen fra oppgave a). Input skal være a, b, r, x og y . Output skal være en av følgende tekster:
 - Punktet ligger innenfor sirkelen.
 - Punktet ligger på sirkelen.
 - Punktet ligger utenfor sirkelen.

Oppg. 1.0.2: Det er mulig å regne ut solens masse ved å bruke

$$M_{sol} = \frac{4\pi^2 \cdot (1 \text{ AU})^3}{G \cdot (1 \text{ yr})^2} \quad (1.1)$$

I denne oppgaven kommer vi til å bruke omtrentlige verdier for AU og G.

Enheten AU er en astronomisk lengdeenhet gitt ved avstanden mellom solen og jorden. Den har en verdi

$$1 \text{ AU} = 1.58 \times 10^{-5} \text{ lysår}$$

der $1 \text{ lysår} = 9.5 \times 10^{12} \text{ km}$.

Konstanten G kalles gravitasjonskonstanten og har verdi

$$G = 6.674 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-1}$$

Lag et program som regner ut solmassen ved å bruke 1.1 og som skriver ut resultatet i kilogram med en pent formatert `print`. Resultatet ditt skal bli omtrent $M_{sol} \approx 2.01 \cdot 10^{30} \text{ kg}$.

Filnavn: `solarmass.py`

Oppg. 1.0.3: Farten til et atom - `velocity_of_atom.py` Atomene i et materiale er strukturert slik at de danner et repeterende gittermønster av deres plasseringer. Vi skal se på et atom

som beveger seg langs overflaten til materialet. På grunn av gitterstrukturen, kan vi lage en modell der farten til atomet er periodisk:

$$v(x) = \sqrt{v_0^2 + \frac{2F_0}{m} \left(\cos\left(\frac{x}{n}\right) - 1 \right)}$$

der m er atomets masse, x er posisjonen til atomet, v_0 er atomets startfart, og n er en skalert avstand mellom atomene i materialet. Vi setter kraften $F_0 = 1$.

Finn farten til atomet når $x = 1$, $v_0 = 2$, $n = 4$ og $m = 3$.

Filnavn: `velocity_of_atom.py`

Oppg. 1.0.4: Følgende fil inneholder informasjon om gjennomsnittlig avstand fra solen, masse, og radius ved ekvator, til en rekke himmellegemer i vårt solsystem.

Object	Distance [km]	Mass [kg]	Radius [km]
Sun	0	1.99e30	695700
Mercury	5.79e7	3.30e23	2440
Venus	1.08e8	4.87e24	6052
Earth	1.50e8	5.97e24	6378
Mars	2.28e8	6.42e23	3396
Jupiter	7.79e8	1.90e27	71492
Saturn	1.43e9	5.68e26	60268
Uranus	2.87e9	8.68e25	25559
Neptune	4.49e9	1.02e26	24764
Pluto	5.90e9	1.46e22	1185

a)

Skriv et program som leser filen `solar_system_data.dat`, som skriver verdiene inn i tre dictionaries `distance`, `mass` og `radius`.

b)

Lag et nytt dictionary, `density`, som inneholder massetettheten til hver av objektene i kg/m^3 . Du kan anta at alle planetene er perfekte sfærer. Volumet til en sfære er $V = \frac{4}{3}\pi r^3$.

c) (Valgfri ekstraoppgave)

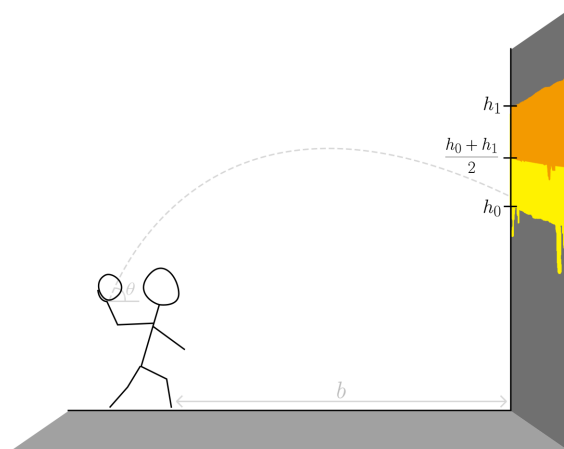
Kombiner informasjonen i et nøstet dictionary, `solar_system` slik at f.eks. `solar_system[Jupiter][radius]` gir radiusen til Jupiter.

Test at det nye dictionary'et du har laget, fungerer som det skal ved å regne ut hvor mange jordkloder det tar å rekke fra solen og til Pluto.

Filnavn: `solar_system_dict.py`

Oppg. 1.0.5: Treffe blink

Vi skal se på hvordan vi kan, ved hjelp av en enkel modell, simulere et spill. Dette spillet går ut på at en person skal treffe en blink malt på en vegg med en ball. Personen får poeng avhengig av hvor på veggen ballen treffer.



Figur 1.1: Illustrasjon av systemet vi skal basere vår spillsimulering på.

Ballens høyde over bakken kan modelleres ved

$$y(t) = -\frac{1}{2}gt^2 + v_0t \sin \theta$$

der v_0 er farten personen kaster ballen med, θ er kastevinkelen, og $g = 9.81 \text{ m/s}^2$.

a)

Skriv en funksjon som returnerer høyden til ballen ved en tid t .

b)

Fra modellen kan vi regne oss fram til at ballen vil treffe veggen ved tiden $T = \frac{b}{v_0 \cos \theta}$, der b er avstanden mellom personen og veggen.

Vi må se på verdien av $y(T)$ for å finne ut hvor mange poeng personen skal få. Antall poeng skal beregnes og returneres fra en funksjon du skal skrive selv.

Blinken er malt slik at den dekker veggen mellom høyden h_0 og høyden h_1 der $h_0 < h_1$. Poengene gis på følgende måte:

- Personen får 0 poeng dersom $y(T) < h_0$ eller $y(T) > h_1$
- Personen får 1 poeng dersom $h_0 \leq y(T) < \frac{1}{2}(h_1 + h_0)$
- Personen får 2 poeng dersom $\frac{1}{2}(h_1 + h_0) \leq y(T) \leq h_1$

Skriv et program som skriver ut i en for-loop hvor mange poeng personen får ved å kalle på din nyskrevne funksjon dersom $h_0 = 3 \text{ m}$, $h_1 = 3.5 \text{ m}$, $\theta = \frac{\pi}{4}$, $b = 3.5 \text{ m}$ for $v_0 = 15, 16, 19, 22 \text{ m/s}$.

1.1 Programmeringsoppgaver i kapittel 2 (er ikke med i læreboken)

Oppg. 1.1.1: Kapittel 2 - seksjon 2.4 - Oppgave 5 (er ikke med i læreboken) Sett opp et Python-program som omgjør fra Fahrenheit (F) til Celsius (C). Omgjør ved hjelp av programmet 100 Fahrenheit til Celsius.

Oppg. 1.1.2: Kapittel 2 - Blandede oppgaver - Oppgave 23 (er ikke med i læreboken) Skriv et program som tegner området i planet avgrenset av ulikhetene:

$$x + y \leq 150, 6x + 10y \leq 1200, y \geq 0.$$

Kapittel 2

Trigonometri og periodiske fenomener

Kapittel 3

Trigonometri og periodiske fenomener

3.1 Fasit for programmeringsoppgaver i kapittel 3

Blandede oppgaver

Se fasit - Oppgave 41

Oppg. 3.1.1: Oppgave 41 Avstanden mellom to punkt på jordens overflate med lengdegrad (spesifiserer øst-vestposisjonen til et punkt på jordoverflaten) og breddegrad (spesifiserer nord-sørposisjonen til et punkt på jordoverflaten): (b_1, l_1) og (b_2, l_2) er gitt ved:

$$d = 6371.01 \cdot \cos^{-1}(\sin(b_1) \sin(b_2) + \cos(b_1) * \cos(b_2) * \cos(l_1 - l_2))$$

Skriv et Python-program som beregner avstanden mellom to punkt gitt ved $(22.5, 68.5)$ og $(24.5, 70.5)$.

Se fasit - Oppgave 42

Oppg. 3.1.2: Oppgave 42 Gitt:

$$y_1 = 1.5 \cos(2n\pi t)$$

$$y_2 = 1.5e^{-2t} \cos(2n\pi t)$$

beskriver en harmonisk og en dempet svingning henholdsvis.

Skriv et program som tegner grafen til disse funksjonene.

Kapittel 4

Grenseverdi og kontinuitet

4.1 Fasit for programmeringsoppgaver i kapittel 4

Blandede oppgaver

Se fasit - oppgave 11

Oppg. 4.1.1: Oppgave 11

Med et annuitetslån betaler du det samme beløpet hver måned, gitt at renten er uforandret. Anta at 100,000 kr. skal betales tilbake med rentefot 5 % i 8 år med 2 terminer pr. år (16 terminer).

- a) Sett opp en geometrisk rekke som bestemmer lånebeløpet L ved hjelp av terminbeløpet T .
- b) Bestem terminbeløpet T .
- c) Her ønsker vi å generalisere problemstillingen. Anta at lånebeløpet L skal betales tilbake med rentefot $p\%$ i t år med n terminer pr. år.
Sett opp et uttrykk for L gitt terminbeløpet T og vekstfaktor a ($a = 1 + p/100$).
- d) Bestem terminbeløpet T uttrykt ved L og p .
- e) Sett opp et program som beregner terminbeløpet T i forrige oppgavedel.

Se fasit - Oppgave 15

Oppg. 4.1.2: Oppgave 15 Skriv et program som leser inn et naturlig tall n samt de to første leddene i en geometrisk følge, og som beregner summen av de n første leddene i følgen.

(ekstra) Bruk programmet til å bestemme summen:

$$2 + 6 + 18 + 54 + 162 + \cdots + 13\,122$$

Kapittel 5

Eksponensialfunksjoner

Oppg. 5.0.1: Prosentvisendring Erfaring viser at prisen til en nybil synker med 18% pr. år eksponentielt. Prisen til en nybil er 350 000.

Vi vil bestemme prisen når den er minst halvert.

Følgende programkode prøver å løse denne problemstillingen:

```
1
2 vekstfaktor = ...
3 verdi = ...
4 startverdi= verdi
5 i=0
6
7 while startverdi >= verdi/2:
8     print(i,startverdi)
9     startverdi = startverdi*vekstfaktor
10    i+=1
11
12 print(i, startverdi)
13 print("Etter", i, "år er verdien mer enn halvert og prisen er",
    startverdi)
```

- Hva skal vekstfaktor og verdi være? Kjør programmet og skriv hva resultatet blir
- En spør deg hvordan du kunne sjekke svarene du fikk i forrige oppgavedel. Hvordan kan dette forklares ved regning?

5.1 Fasit for programmeringsoppgaver i kapittel 5

Blandede oppgaver

Fasit - oppgave 15

Oppg. 5.1.1: Oppgave 15

Med et annuitetslån betaler du det samme beløpet hver måned, gitt at renten er uforandret. Anta at 100 000 kr. skal betales tilbake med rentefot på 5 % i 8 år med 2 termin er pr. år (16 terminer).

- Sett opp en geometrisk rekke som bestemmer lånebeløpet L ved hjelp av terminbeløpet T .
- Bestem terminbeløpet T .

- c) Her ønsker vi å generalisere problemstillingen. Anta at lånebeløpet L skal betales tilbake med rentefot $p\%$ i t år med n terminer pr. år.
Sett opp et uttrykk for L gitt terminbeløpet T og vekstfaktor a ($a = 1 + p/100$).
- d) Bestem terminbeløpet T uttrykt ved L og p .
- e) Sett opp et program som beregner terminbeløpet T i forrige oppgavedel.

Kapittel 6

Derivasjon

6.1 Numerisk derivasjon

På en eksamen i 1T får man spørsmål om hva følgende program utfører

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def f(x):
5     y=x/(1+x**2)
6     return y
7
8 x=0
9 h=0.001
10 t=0
11
12 while f(x) <= f(x+h):
13     t+=1
14     x=x+h
15
16 print(x)
17 print(t)
18
19 x = np.linspace(-3,3, num = 101)
20
21 plt.plot(x,f(x), '--')
22 plt.grid()
23 plt.xlabel('x')
24 plt.ylabel('y')
25 plt.legend(['f(x)'])
```

I programmet er definert funksjonen $f(x) = \frac{x}{1+x^2}$ og programmet prøver å finne nærmeste minimumspunkt til $x = 0$ med å ta tusen steg fra 0 med steglengde $h = 0.001$.

Oppg. 6.1.1: Skriv kort hva følgende program utfører.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     y=x**3-3*x**2
6     return y
7
8
9 def derivert(a, deltax):
10     return(f(a + deltax) - f(a))/deltax
11
12 xstart = -2
13 xslutt = 5
14 deltax = .001
15 antallpunkter = 501
16
17 x = np.linspace(xstart, xslutt, antallpunkter)
18 y=f(x)
19 dy = derivert(x, deltax)
20 plt.plot(x,y)
21 plt.plot(x, dy, '--')
22
23 plt.plot(x,y, '--')
24 plt.plot(x,dy, '--')
25 plt.legend(['Funksjonen y=f(x)', 'Den deriverte dy/dx'] )
26 plt.xlabel('x')
27 plt.ylabel('y')
28 plt.grid()
29 plt.show()
```

Numerisk derivasjon og endelige differanser

Stigningstallet mellom to punkt $x = x_0$ og $x = x_0 + h$ kan uttrykkes ved:

$$\frac{f(x_0 + h) - f(x_0)}{h} \quad (6.1)$$

Hvis h blir mindre og mindre, så nærmer sekanten seg mer og mer tangenten, fordi punktet $(x + h, f(x + h))$ kommer nærmere og nærmere $(x, f(x))$:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0)}{h}$$

La f være en analytisk funksjon. Den taylorutviklingen til f om punktet $x = x_0$ er:

$$f(x_0 + h) = f(x_0) + \frac{f'(x_0)}{1!}h + \frac{f''(x_0)}{2!}h^2 + \frac{f'''(x_0)}{3!}h^3 + \dots$$

$$\frac{f(x_0 + h) - f(x_0)}{h} = \frac{f'(x_0)}{1!} + \frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

som beskriver feilen i estimeringen av sekanten mellom punktene $x = x_0$ og $x = x_0 + h$:

$$\frac{f(x_0 + h) - f(x_0)}{h} - f'(x_0) = \frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

- Tilnærmingen:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0)}{h}$$

kalles *forlengsdifferanse* og dens største estimeringsfeilen er av orden h . Tilsvarende:

$$\frac{f(x_0) - f(x_0 - h)}{h} = \frac{f'(x_0)}{1!} - \frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

som beskriver feilen i estimeringen av sekanten mellom punktene $x = x_0 - h$ og $x = x_0$:

- Tilnærmingen:

$$f'(x_0) \simeq \frac{f(x_0) - f(x_0 - h)}{h} \quad (6.2)$$

kalles *baklengsdifferanse*, og dens største estimeringsfeilen er av orden h .

$$\frac{f(x_0) - f(x_0 - h)}{h} - f'(x_0) = -\frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

Nå kan vi sette opp:

$$\frac{f(x_0 + h) - f(x_0 - h)}{2h} - f'(x_0) = \frac{f'''(x_0)}{3!}h^2 + \frac{f^{(5)}(x_0)}{3!}h^2 + \dots \quad (6.3)$$

- Tilnærmingen:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0 - h)}{2h} \quad (6.4)$$

kalles *sentraldifferanse* (endelig differanse), og estimeringsfeilen er av orden h^2 .

Lignende endelige differanser kan brukes til å bestemme deriverte av høyere orden:

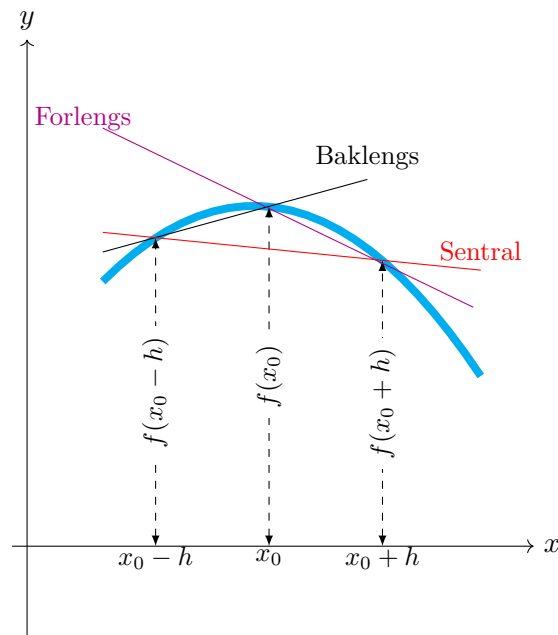
$$\frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h))}{2h} - f''(x_0) = \frac{f^{(4)}(x_0)}{12}h^2 + \dots$$

Da blir:

$$f''(x_0) \approx \frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h))}{h^2}$$

De tre endelige differanse approksimasjonene for å bestemme $f'(x_0)$ er da:

$$\begin{aligned} f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0)}{h}, && \text{forlengsdifferanse,} \\ f'(x_0) &\approx \frac{f(x_0) - f(x_0 - h)}{h}, && \text{baklengsdifferanse,} \\ f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}, && \text{sentraldifferanse.} \end{aligned}$$



Figur 6.1: Endelige differanse-metoder

I figuren 6.1 viser den lilla linjen en forlengsdifferanse, den svarte linjen en baklengsdifferanse og den røde linjen en sentraldifferanse.

I sentraldifferansen er avstanden til hvert av punktene $2h$, mens det for forlengs- og baklengsdifferansen er h . Dermed er i sentraldifferansen delt på $2h$, i stedet for h for å approksimere den deriverte i punktet x_0 .

MERK. Feilleddet i sentraldifferansen er av størrelsesorden h^2 (konstant $\cdot h^2$), mens det for forlengs- og baklengsdifferansen er av størrelsesorden h (konstant $\cdot h$). Dermed er sentraldifferansen en bedre approksimasjon av den deriverte enn forlengs- og baklengsdifferansen. Sentraldifferansen er middelverdien til forlengs- og baklengsdifferansen.

Oppg. 6.1.2: Bestem en tilnæringsverdi $f'(0.5)$ ved hjelpe de tre differanse-metodene gitt:

-0.25	0.273
0	0.385
0.25	0.448

Oppg. 6.1.3: De tre differansene for å bestemme den deriverte til f i punktet $x = x_0$ kan skrives som

$$\begin{aligned} f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0)}{h}, && \text{forlengsdifferanse,} \\ f'(x_0) &\approx \frac{f(x_0) - f(x_0 - h)}{h}, && \text{baklengsdifferanse,} \\ f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}, && \text{sentraldifferanse.} \end{aligned}$$

Skriv et program som beregner den deriverte til funksjonen til $f(x) = e^x - 4x$ i $x = 2$ ved hjelp av de tre metodene for $h = 0.1$, $h = 0.01$, $h = 0.001$ og $h = 0.0001$.

Tegn grafen til funksjonen sammen med den numeriske derivasjonen med $h = 0.0001$ i intervallet $[-3, 3]$.

Oppg. 6.1.4: Skriv et program som bruker numerisk derivasjon til å finne tilnærminger til de deriverte av $f(x) = x^2$ i $x = 3$ for $h = 0.1$, $h = 0.01$, $h = 0.001$ og $h = 0.0001$.

Tegn grafen til funksjonen sammen med den numeriske derivasjonen for $h = 0.0001$ i intervallet $[-2, 2]$.

Oppg. 6.1.5: Gitt et python program:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n = 6
5
6 x = np.linspace(1,4,num = n+1)
7 delta_x = x[1] - x[0]
8 f = 4*x**3 - 2*x
9
10 Sum = 0
11
12 for i in range(len(x)-1):
13     Sum = Sum + ((f[i]+f[i+1])/2)*HVA
14
15 print(Sum)
16
17 N=10000
18 u = np.linspace(1,4,num = N+1)
19 f1 = 4*u**3 - 2*u
20
21 plt.plot(x,f)
22 plt.plot(u,f1, label='f(x)')
23 plt.grid(ls='--')
24 plt.legend()
```

- Forklar hva programmet utfører og skriv kort om detaljene i programmet.
Hva kan det skrives for HVA i programmet?
- Hvordan kan man øke presisjonen for å bestemme Sum i programmet?

6.2 Newtons metode

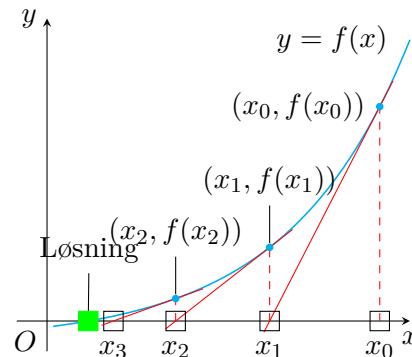
Newtons metode er en numerisk metode til å finne en løsning til ligningen $f(x) = 0$, som ligger i intervallet $[a, b]$.

La f være kontinuerlig på $[a, b]$ og ifølge skjæringssetningen har ligningen minst en løsning dersom $f(a) \cdot f(b) < 0$. Algoritmen begynner med en startverdi $x = x_0$, der $a < x_0 < b$ og man bestemmer hvor tangenten gjennom punktet $(x_0, f(x_0))$ treffer x -aksen og neste punkt blir utgangspunkt for neste iterasjon og da man bestemmer neste x -verdi, x_{n+1} er på samme måte skjæringspunktet mellom tangenten gjennom $(x_n, f(x_n))$:

$$y = f(x_n) + f'(x_n)(x - x_n)$$

og x -aksen ($y = 0$), som blir x_{n+1} ($x = x_{n+1}$):

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Oppg. 6.2.1: Forklar kort hva følgende programkode utfører:

```
1  xn = x0
2  for n in range(0,N):
3      fxn = f(xn)
4      if abs(fxn) < epsilon:
5          print('Svar er funnet etter',n,'iterations.')
6          return xn
7      Dfxn = Df(xn)
8      if Dfxn == 0:
9          print('Ingen svar')
10         return None
11         xn = xn - fxn/Dfxn
12     print('Overstiger maks antall interasjoner. Ingen løsning funnet')
13     return None
14
15 f = lambda x: x**3 - 3*x**2 - 1
16 Df = lambda x: 3*x**2 - 6*x
17
18 Losn= newton(f,Df,0.1,1e-2,100)
19 print(Losn)
```

Oppg. 6.2.2: Forklar kort hva følgende programkode utfører:

```
1  import numpy as np
2
3  def newton(f,df,x0,tol,N):
4      '''Approximate solution of f(x)=0 by Newton's method.
5
6      xn = x0
7      for n in range(0,N):
8          fxn = f(xn)
9          if abs(fxn) < tol:
```

```

10         print('Losn er funnet etter',n,'iterasjoner.')
11         return xn
12     dfxn = df(xn)
13     if dfxn == 0:
14         print('Den deriverte er null. Ingen losn er funnet.')
15         return None
16     xn = xn - fxn/dfxn
17     print('Maksimalt antall iterasjoner er overskredet. Ingen losn
funnet.')
18     return None
19
20 def f(x):
21     return x**3- np.cos(x)
22
23 def df(x):
24     return 3*x**2 +np.sin(x)
25
26 Losn = newton(f,df,1,1e-10,10)
27 print(Losn)

```

Oppg. 6.2.3: Vis at ligningen $x + \sin x + 1 = 0$ har *presis* ett nullpunkt x_0 i intervallet $[-\pi/2, 0]$. Bruk Newtons metode tre ganger, med startverdi $x_0 = -1$, til å finne en tilnærmet verdi til dette nullpunktet.

Oppg. 6.2.4: Skriv et program som finner en tilnærmet løsning for ligningen $x^3 - x^2 = 3$ som ligger i intervallet $[0, 2]$ (bruk startverdien $x = 1.5$).
(Hint: programmet fortsetter til $|f(x_n)| < \epsilon$, for en gitt toleranseverdi (ϵ))

Halveringsmetoden

Halveringsmetoden er en enkel iterativ metode for å finne et nullpunkt til en funksjon, dvs. finne en løsning på likningen $f(x) = 0$. For å bruke metoden må vi kjenne to punkt a og b som er slik at $f(a)$ og $f(b)$ har ulikt fortegn ($f(a) \cdot f(b) < 0$), og funksjonen må være kontinuerlig mellom på intervallet $[a, b]$. Vi starter med å finne midtpunktet m mellom a og b . Hvis $f(m)$ og $f(a)$ har ulikt fortegn ($f(a) \cdot f(m) < 0$), så letter vi videre i intervallet $[a, m]$. Hvis ikke, fortsetter vi med intervallet $[m, b]$. Slik kan vi fortsetter til svaret tilfredsstillende toleransekravet at når funksjonsverdien til midtpunktet er tilstrekkelig nær null (hva som er tilstrekkelig velger du!), er algoritmen ferdigkjørt, og du har estimert nullpunktet for funksjonen.

Oppg. 6.2.5: Skriv et program som finner en tilnærmet løsning for ligningen $2x \cos x - 1 = 0$ som ligger på intervallet $(0, 1)$ ved hjelp av halveringsmetoden der estimeringsavviket er mindre enn 0,001.

Oppg. 6.2.6: Vi ønsker å bestemme en tilnærmet verdi for $\sqrt[3]{x}$ og vil bruke halveringsmetoden for å løse ligningen $x^3 = 5$.

- Skriv et program som finner en tilnærmet løsning for ligningen ved hjelp av halveringsmetoden. I programmet gis endepunktene til et intervallet $[1, 2]$ og en presisjon(ϵ). Sett inn en kommando som viser hvor mange ganger løken kjører.
- En student foreslår en algoritme for å finne nullpunktet slik at å nærme seg mot løsningen fra en av endepunktene med en gitt margin (en infinitesimal endring) Δx . Programmet sjekker hver gang om grafen har passert x -aksen, for eksempel $f(\text{endepunkt}) \cdot f(\text{endepunkt} + \text{Delta}x)$

ikke har skiftet fortegn fra fortegnet til funksjonsverdi i det endepunktet. Programmet skal angi antall iterasjoner også.

c) En annen student foreslår en algoritme som bruker tilfeldig søk:

```

1 import random
2 def f(x):
3     return x**3 - 5
4 a = 1
5 b = 2
6 N = 1000
7 n = 0
8 Beste_y = f(a)
9 Beste_x = a
10 for i in range(N):
11     x = random.uniform(a, b)
12     y = f(x)
13     if abs(y) < abs(Beste_y):
14         Beste_y = y
15         Beste_x = x
16     n = n+1
17 print(Beste_x)
18 print("Antall iterasjoner:", n)

```

Ved å kjøre programmet får vi forskjellige verdier som forventet da vi bruker tilfeldig søk blant 1000 tall i intervallet.

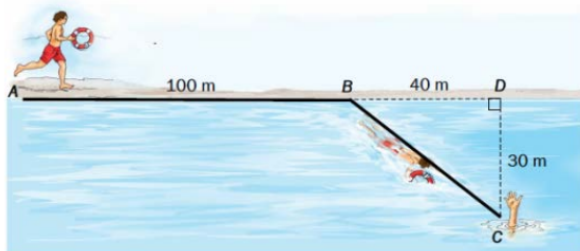
```

1 1.710007277735471
2 Antall iterasjoner: 7
3 1.7099466831544587
4 Antall iterasjoner: 13

```

Oppg. 6.2.7: Vis at ligningen $x + \sin x + 1 = 0$ har *presis* ett nullpunkt x_0 i intervallet $[-\pi/2, 0]$. Bruk Newtons metode tre ganger, med startverdi $x_0 = -1$, til å finne en tilnærmet verdi til dette nullpunktet.

Oppg. 6.2.8: En badevakt løper fra A til B og svømmer videre fra B til C for å hjelpe en person i nød. Se skissen nedenfor:



- Bestem avstanden fra B til C.
- Badevakten løper i 20 sekunder og svømmer i 1 min .
Regn ut forholdet mellom farten han har når han løper, og farten han har når han svømmer.
- La nå avstanden mellom A og D være 140 meter, $CD = 30\text{m}$, og $BD = x$.
Badevakten kan løpe på land med farten 5 m/s og kan svømme i vann med en fart på 1 m/s.
Badevakten ønsker å finne hvor på land han skal begynne å svømme for å kunne komme til

punktet C så raskt som mulig.

Hva blir den minste tiden da? Ser tiden du får fornuftig ut?

Hvis man skulle bruke programmering, og kunne la lengden av AD og CD leses av skjermen, hvordan du hadde laget programkoden. Du trenger ikke å skrive hele programmet detaljert men forklar kort fremgangen.

Oppg. 6.2.9: a) Sett opp Taylor-rekken til $f(x) = e^x$ om $x = 2$ og angi det generelle (almenne) leddet i rekken.

b) Skriv et program som beregner $f(2)$ ved å bruke 7 og 10 de første leddene i Taylor-rekken. Utvid programmet slik at den også beregner estimeringsfeilen og skriver den ut sammen med den estimerte verdien og den eksakte verdien.

c) Gjør eventuelle endringer i programmet i forrige del for å kunne beregne Taylor-rekken til $g(x) = e^{x^2}$ i $x = 2$ og skriver ut den eksakte verdien og estimeringsfeilen.

6.3 Fasit for programmeringsoppgaver i kapittel 6

Blandede oppgaver

Se fasit - Oppgave 75

Oppg. 6.3.1: Oppgave 75 Finn grenseverdien om den eksisterer: $\lim_{x \rightarrow \infty} \frac{\ln x}{x}$.
Skriv et Python-program som undersøker svaret.

Oppgave 77

Oppg. 6.3.2: Oppgave 77 Det finnes flere metoder for å beregne den deriverte til en funksjon i et punkt numerisk. Her er tre metoder:

Forlengsmetoden: $\frac{f(x+h) - f(x)}{h}$

Baklengsmetoden: $\frac{f(x-h) - f(x)}{-h}$

Sentraldifferanse: $\frac{f(x+h) - f(x-h)}{2h}$

Her vil vi sammenligne disse tre metodene med den eksakte deriverte for funksjonen: $f(x) = xe^{2x}$.
Skriv et Python-program som bruker disse metodene og beregner også den eksakte verdien for den deriverte til $f(x) = xe^{2x}$ i punktet $x = 1$ med steglengde $h = 0,1$ først.

Modifiser deretter programmet slik at den beregner relativt estimeringsavvik for disse tre metodene, og tegner grafen til relativt estimeringsavvik versus steglengden h for intervallet $0,001 \leq h \leq 0,1$. Hvilken av disse vil du anbefale?

Se fasit - Oppgave 78

Oppg. 6.3.3: Oppgave 78 Vi ønsker å bestemme en tilnærmet løsning for ligningen $x + 1 = 2e^{-x}$ som er nær 1.

- Skriv et program som finner en tilnærmet løsning for ligningen ved hjelp av halveringsmetoden. Programmet leser inn endepunktene til intervallet vi skal søke løsningen i, og gitt en presisjon(ϵ). Sett inn en kommando som viser hvor mange ganger løkken kjører.
- En student foreslår en algoritme for å finne nullpunktet slik at den nærmer seg mot løsningen fra et av endepunktene med en gitt margin (en infinitesimal endring) Δx og programmet sjekker hver gang om grafen har passert x -aksen, for eksempel at $f(\text{endepunkt}) \cdot f(\text{endepunkt} + \text{Deltax})$ ikke har skiftet fortegn fra fortegnet til funksjonsverdien i det endepunktet. Programmet skal angi antall iterasjoner også.
- En annen student foreslår en algoritme som bruker tilfeldig søk for å finne nullpunktet, slik at programmet plukker tilfeldig en kandidat fra løsningsintervallet og programmet fortsetter mens absoluttverdien til funksjonsverdien er mindre enn absoluttverdien til funksjonsverdien til endepunktet, eller stopper.

Se fasit - Oppgave 82

Oppg. 6.3.4: Oppgave 82 Skriv et program som bruker definisjonen av den deriverte til å bestemme en tilnærmet verdi for den deriverte ($h = 10^{-5}$) og bruker Newtons metode til å finne en tilnærmet løsning for likningen $x^3 - x = 1$ som ligger i intervallet $t \in [0, 2]$. Bruk startverdien $x = 1$. Programmet skal fortsette til $|x_n - x_{n-1}| < \text{epsilon}$, der $\epsilon > 0$ er en presisjon angitt av brukeren. Test programmet med $\text{epsilon} = 10^{-4}$

Kapittel 7

Integrasjon og numerisk integrasjon

Numerisk integrasjon

Det bestemte integralet $\int_a^b f(x)dx$ lar seg ikke alltid integrere ved å integrere analytisk for å bestemme den eksakte verdien til integralet. Det kan være fordi integranden $f(x)$ er noe som vi ikke kan integrere ved hjelp av analytiske metoder eller $f(x)$ er et resultat av målinger og man ikke har noen analytisk funksjon å integrere. Dermed er det viktig å ha numeriske metoder som kan beregne integralet. Det finnes flere numeriske metoder for å regne ut et bestemt integral. Vi fokuserer her på to kjente metoder.

Trapesmetoden

Vi skal regne ut $\int_a^b f(x)dx$ numerisk. Vi begynner med å dele inn intervallet $[a, b]$ i n like store deler,

$$a = x_0 < x_1 < x_2 < \dots < x_n = b,$$

slik at alle delintervallene $[x_{i-1}, x_i]$ har samme bredde $\Delta x = x_i - x_{i-1}$. Nå kan vi tilnærme grafen til $f(x)$ ved å trekke rette linjestykker mellom punktene $(x_i, f(x_i))$. Idéen er nå å tilnærme integralet $\int_a^b f(x)dx$ ved hjelp av summen av alle arealene (med fortegn) mellom x -aksen og disse linjestykkene. Arealet under hvert linjestykke er et trapes. Vi har da følgende formel:

$$\int_a^b f(x)dx \approx \frac{\Delta x}{2} (f(x_0) + 2f(x_1) + 2f(x_3) + \dots + 2f(x_{n-1}) + f(x_n))$$

der $\Delta x = \frac{b-a}{n}$ og $x_i = a + i\Delta x$.

Legg merke til at i denne metoden er første funksjonsverdi, $f(x_0)$, og siste funksjonsverdi $f(x_n)$ benyttet kun en gang i henholdsvis utregning av første og siste areal. De andre funksjonsverdiene er benyttet to ganger av forrige og neste areal i grafen.

Simpsons metode

I trapesmetoden tilnærmet vi grafen til funksjonen $f(x)$ ved hjelp av linjestykker. Nå skal vi i stedet tilnærme grafen til $f(x)$ ved hjelp av biter av andregradskurver. En rett linje er entydig bestemt av *to punkt* på linjen. Et andregradspolynom er, som vi skal se, entydig bestemt av *tre punkt* på grafen. Derfor skal vi dele inn integrasjonsintervallet $[a, b]$ i et partall antall $2n$ delintervaller,

$$a = x_0 < x_1 < x_2 \dots < x_{2n} = b$$

og så, på hvert “doble” delintervall $[x_{2i-2}, x_{2i}]$, tilnærme $f(x)$ ved hjelp andregradspolynomet som har de samme verdier som f i punktene x_{2i-2} , x_{2i-1} og x_{2i} . På denne måten tilnærmer vi

grafen til f ved hjelp av n andregradskurver. Arealet under hver andregradskurve finner vi ved integrasjon, og så summerer vi alle disse arealene.

Simpsons metode med $2n$ delintervaller gir tilnærmingen

$$\int_a^b f(x) dx \approx \frac{\Delta x}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + 4f(x_5) + \cdots + f(x_{2n}))$$

der $\Delta x = \frac{b-a}{2n}$ og $x_i = a + i\Delta x$.

Litt om feilen i trapesmetoden og Simpson metode

Det kan være krevende å vise feilestimat for trapesmetode. Her ser vi kun på resultatet. For bevis, se: Adams, *Calculus*.

Teorem 7.0.1:

Anta at f er en to ganger deriverbar funksjon, definert på et åpent intervall som inkluderer $[a, b]$. Anta også at $|f''(x)| < K$ for alle $x \in [a, b]$. Da er estimeringsfeilen ved å bruke trapesmetoden med n delintervaller er mindre enn $K(b-a)^3/12n^2$:

$$\left| \int_a^b f(x) dx - \frac{1}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(a + i\Delta x) + f(b) \right) \cdot \Delta x \right| < \frac{K(b-a)^3}{12n^2}$$

Teorem 7.0.2:

Anta at f er en fire ganger deriverbar funksjon, definert på et åpent intervall som inkluderer $[a, b]$. Anta også at $|f^{(4)}(x)| < M$ for alle $x \in [a, b]$. Da er estimeringsfeilen ved å bruke Simpsons metode med n delintervaller er mindre enn $M(b-a)^4/180n^4$:

$$\left| \int_a^b f(x) dx - \frac{1}{3} \left(f(a) + \sum_{i=1}^{n/2} f(x_{2i}) + \sum_{i=1}^{n/2-1} f(x_{2i+1}) + f(b) \right) \cdot \Delta x \right| < \frac{M(b-a)^5}{180n^4}$$

Se oppgave 7.0.7.

Oppgaver- Numerisk integrasjon

Oppg. 7.0.1: Gitt integralet

$$\int_0^5 \frac{1}{x^2 + 1} dx$$

- a) Regn ut integralet.
- b) Sett opp et Python-program som bestemmer nedre (venstre) Riemann sum (nedre sum) med antall partisjoner $N = 10$ og $N = 100$.

$$\int_a^b f(x) dx \approx \Delta x (f(x_0) + f(x_1) + f(x_2) + \cdots + f(x_{n-1}))$$

Angi også estimeringsfeilen for hver av delsummene ved å bestemme differensen mellom den høyre og venstre Riemannsummen.

Oppg. 7.0.2: Gitt integralet

$$\int_0^5 \frac{1}{x^2 + 1} dx$$

- a) Regn ut integralet.
- b) Sett opp et Python-program som bestemmer nedre (venstre), midtpunkt (rektangelmetoden) og øvre (høyre) Riemannsum med antall partisjoner $N = 10$.

$$\int_a^b f(x) dx \approx \Delta x (f(x_0) + f(x_1) + f(x_2) + \cdots + f(x_{n-1}))$$

$$\int_a^b f(x) dx \approx \frac{\Delta x}{2} (f(x_0) + 2f(x_1) + 2f(x_3) + \cdots + 2f(x_{n-1}) + f(x_n))$$

$$\int_a^b f(x) dx \approx \Delta x (f(x_1) + f(x_2) + f(x_3) + \cdots + f(x_n))$$

Angi også estimeringsfeilen for hver av delsummene ved å bestemme differensen mellom den høyre og venstre Riemannsummen.

Oppg. 7.0.3: Gitt integralet

$$\int_0^5 \frac{1}{x^2 + 1} dx$$

- a) Sett opp et program som bruker venstre, høyre Riemannsum, midtpunkt rektangelmetoden, trapesmetoden og symponsmetoden til å regne integralet med $N = 10$ og $N = 100$.

Oppg. 7.0.4: Gitt integralet

$$\int_0^5 \frac{1}{x^2 + 1} dx$$

- a) Sett opp et program som bruker venstre, høyre Riemannsum, midtpunkt rektangelmetoden, trapesmetoden og symponsmetoden til å regne integralet med $N = 10$ og $N = 100$.

Oppg. 7.0.5: Gitt integralet

$$\int_0^2 e^{-x^2} dx$$

Sett opp et program som bruker venstre, høyre Riemannsum, midtpunkt rektangelmetoden, trapesmetoden og symponsmetoden til å regne integralet med $N = 10$ og $N = 100$.

Oppg. 7.0.6: Skriv et program som tegner grafen til $f(x) = x^3$ sammen med figuren til nedre Riemannsum til

$$\int_0^{10} x^3 dx$$

Oppg. 7.0.7: Estimeringsavvikene for trapesmetoden og Simpsons regel er gitt ved:

$$|e_T| < \frac{\max_{[a,b]} f''(x)}{12n^2} (b-a)^3$$

$$|e_S| < \frac{\max_{[a,b]} f'''(x)}{180n^4} (b-a)^5$$

Bruk trapesmetoden og Simpsons regel til å beregne en tilnæringsverdi for integralet og bestem estimeringsavvikene for disse metodene for $n = 4$ når: $\int_0^1 x^4 dx$

Oppg. 7.0.8: Gitt $\int_0^1 \frac{x}{\cos x} dx$.

- Regn ut integralet ved hjelp av trapesmetoden ($n = 4$). Regn også ut estimeringsfeilen.
- Regn ut integralet ved hjelp av Simpsons formel ($n = 4$). Regn også ut estimeringsfeilen.
- Forklar kort hvilken av disse numeriske metodene som er best og hvorfor.

Oppg. 7.0.9: Gitt $\int_0^2 e^{x^2} dx$. Ved å bruke MATLAB får man: $\int_0^2 e^{x^2} dx \simeq 16,45262776$.

- Regn ut integralet ved hjelp av trapesmetoden ($n = 4$). Regn også ut estimeringsavviket.
- Regn ut integralet ved hjelp av Simpsons regel ($n = 4$). Regn også ut estimeringsfeilen.
- Kommentar svarene.

7.1 Fasit for programmeringsoppgaver i kapittel 7

Blandede oppgaver

Se fasit - oppgave 20

Oppg. 7.1.1: Oppgave 20

I oppgave del b) skal du ikke bruke programmering.

- Forklar kort om trapesmetoden, der man bruker formelen:

$$\int_a^b f(x) dx \approx \frac{f(a) + 2 \sum_{i=1}^{n-1} f(a + i\Delta x) + f(b)}{2} \cdot \Delta x.$$

- Bruk trapesmetoden (uten programmering) til å beregne for hånd

$$\int_0^\pi \sin(x) dx$$

ved å dele intervallet i 6 like store deler.

Hva er den eksakte verdien?

c) Bruk trapesmetoden ved hjelp av programmering til å beregne

i) $\int_0^{\pi} \sin(x) dx$

ii) $\int_0^{\pi} \sin(x^2) dx$

Bruk programmet til å sammenligne med svaret i del b).

Kapittel 8

Komplekse tall

Kapittel 9

Differensialligninger og anvendelser

Differensialligninger er en dynamisk beskrivelse av et kontinuerlig system eller en prosess, basert på de "likevektsligningene" (balanseligningene) vi har satt opp for prosessen.

En differensialligning er en ligning som uttrykker en sammenheng mellom en uavhengig variabel x (eller t), en funksjon (avhengig variabel) $y(x)$, dens deriverte $y'(x)$ og eventuelt høyere ordens deriverte.

$$L(x, y, y', y'', \dots) = 0$$

For eksempel, hvis vekstraten $\frac{dy}{dx}$ er proporsjonal med y , kan man få differensialligningen:

$$\frac{dy}{dx} = ay, \text{ der } a \text{ er proporsjonalitetskonstanten}$$

Hensikten med å løse en differensialligning er å bestemme funksjonen, $y = y(x)$.

Klassifisering (type)

En differensialligning er *lineær* dersom ligningen er lineær med hensyn til den ukjente funksjonen og dens deriverte.

Lineær	Ulineær
$y'' + y \cos x = e^x$	$y'' + x \cos y = e^x$
$y' + x^2 y = \ln x$	$y'' + y^2 x = \ln x$

Har differensialligningen minst ett ledd uten den avhengige variabelen, er differensialligningen *homogen* ellers den er *inhomogen*.

Homogen	Inhomogen
$y'' + y \cos x = 0$	$y'' + x \cos y = e^x$
$y' + x^2 y = \ln y$	$y'' + y^2 x = \ln x$

Separable differensialligninger på formen: $\frac{dy}{dx} = f(x)g(y)$

ligningen kan løses ved separasjon:

$$\int \frac{1}{g(y)} dy = \int f(x) dx$$

Løs integralet og bestem $y = y(x)$.

Separable differensialligninger på formen $y' = f(x)g(y)$ kan ha *konstant(e) løsning(er)*, $y = k$ når $g(y) = 0$.

Førsteordens lineær differensialligning med konstante koeffisienter

$$ay' + by = f(x)$$

Den generelle løsningen er lik summen til løsningen til den homogene differensialligningen og løsningen til den inhomogene differensialligningen (den partikulære (spesielle) løsningen):

$$y(x) = y_h(x) + y_p(x)$$

Homogen $ay' + by = 0$

Ved å anta en løsning er $y = e^{rx}$, kan man sette opp den karakteristiske ligningen: $ar + b = 0$.

Den generelle løsningen av den homogene differensialligningen er $y_h = Ce^{-\frac{b}{a}x}$. For y_p se under.

Andreordens lineær differensialligning med konstante koeffisienter

$$ay'' + by' + cy = f(x)$$

Den generelle løsningen er $y(x) = y_h(x) + y_p(x)$.

Homogen $ay'' + by' + cy = 0$

Ved å anta en løsning er $y = e^{rx}$, kan man sette opp den karakteristiske ligningen:

$$ar^2 + br + c = 0$$

Den generelle løsningen av den homogene differensialligningen er:

$$y_h = \begin{cases} C_1 e^{r_1 x} + C_2 e^{r_2 x} & r_1 \neq r_2 \text{ (reelle)} \\ e^{rx}(C_1 + C_2 x) & r = r_1 = r_2 \text{ (reelle)} \\ e^{\alpha x}(C_1 \cos \beta x + C_2 \sin \beta x) & r = \alpha \pm i\beta \end{cases}$$

Andreordens inhomogen lineær differensialligning $ay'' + by' + cy = f(x)$

For å finne den partikulære løsningen, kan du gjette en spesiell løsning y_p og finne konstantene ved innsetting (ubestemte koeffisienters metode). Metoden kan også brukes for $ay' + by = f(x)$.

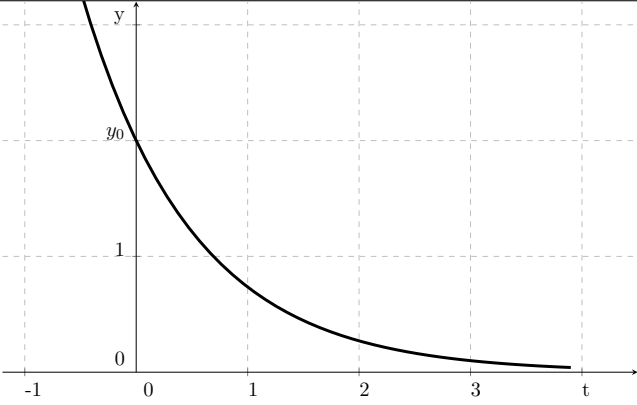
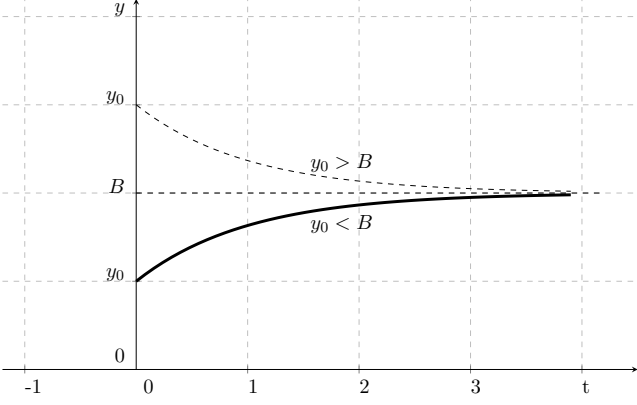
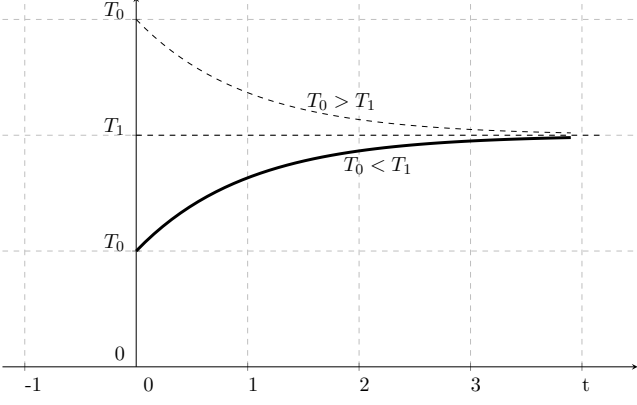
$f(x)$	$y_p(x)$
k	A
kx	$Ax + B$
$kx + l$	$Ax + B$
$ke^{\alpha x}$	$Ae^{\alpha x}$
$k \sin \beta x$	$A \cos \beta x + B \sin \beta x$
$k \cos \beta x$	$A \cos \beta x + B \sin \beta x$

Dobbel og trippelløsning

Dersom y_p er faktor av deler av y_h , ganger vi den med x^r ($r = 1$ for dobbelløsning, $r = 2$ for trippelløsning).

$f(n)$	y_h	y_p
$y'' - 4y = 5e^{2x}$	$y_h = C_1 e^{-2x} + C_2 e^{2x}$	$y_p = A x e^{2x}$
$y'' - 4y' + 4y = 7e^{2x}$	$y_h = (C_1 x + C_2) e^{2x}$	$y_p = A x^2 e^{2x}$

Noen anvendelser i naturvitenskap og teknologi

Anvendelser differensialligning	Grafen til løsningskurve (integralkurve)
Radioaktiv stråling $\frac{dy}{dt} = -ky$ gitt $y(0) = y_0$ Løsning $y(t) = y_0 e^{-kt}$	
Sykdomsspredning (vekstmodell) $\frac{dy}{dt} = k(B - y), k > 0$ gitt $y(0) = y_0$ Løsning $y(t) = B + (y_0 - B)e^{-kt}$	
Temperaturendring $\frac{dT}{dt} = -k(T - T_1), k > 0$ gitt $T(0) = T_0$ Løsning $T(t) = T_1 + (T_0 - T_1)e^{-kt}$	

Eulers metode

Det er sjeldent man kan finne en eksplisitt løsning for en differensialligning. Det finnes flere numeriske metoder for å finne en tilnærmet løsning for en del ordinære differensialligninger. Eulers metode, oppkalt etter den sveitsiske matematikeren Leonhard Euler, er en numerisk metode for å regne en tilnærmet løsning til:

$$\frac{dy}{dx} = f(x, y), \text{ gitt } y(x_0) = y_0, \text{ bestem } y_i = y(x_i)$$

Metoden bygger på tilvekstformelen: $\frac{y(x_n + h) - y(x_n)}{h} \simeq y'(x_n) = f(x_n, y_n)$, som skrives som: $y(x_n + h) \simeq y(x_n) + hf(x_n, y_n)$.

Med andre ord kan man bevege seg langs tangenten for å bestemme neste y -verdi i hvert steg: $y_1 = y_0 + hy'_0$: y_1 beregnes, og da kan man bestemme $x_1 = x_0 + h$ og $y'_1 = f(x_1, y_1)$ for neste steg:

$y_2 = y_1 + hy'_1$: y_2 beregnes, og da kan man bestemme $x_2 = x_1 + h$ og $y'_2 = f(x_2, y_2)$ for neste steg

Denne formelen kan også settes opp ved Taylors formel: $y(x_n + h) \simeq y(x_n) + hf(x_n, y_n) + O(h^2)$. Eulers metode er en 1. ordens metode, da uttrykket for $y_n(x)$ er av orden h (1. orden av h). Metoden har en global feil av orden h og en lokal av orden h^2 .

Retningsfelt og vektorfelt

Retningsfelt er en grafisk beskrivelse av en differensialligning. Eulers problem $y' = f(x, y)$, $y(x_0) = y_0$ angir stigningstallet til tangenten. Ved å tegne disse stigningstallene, korte linjestykker i planet gjennom mange punkter, kan vi få et inntrykk hvordan løsningskurvene ser ut. Ved å tegne korte vektorpiler kalt *retningsvektor* ($\vec{r} = \frac{1}{\sqrt{1 + f'^2(x)}}[1, f'(x)]$) gjennom mange punkter, kan vi få *vektorfelt*.

9.1 Fasit for programmeringsoppgaver i kapittel 9

Seksjon 9.7

Se fasit - Oppgave 9.7.3 del c)

Oppg. 9.1.1: oppgave 9.7.3 del c) Skriv et program som løser differensialligningen $\frac{dc}{dt} = -kc^2$, og tegner løsningskurven for $k = 0.1, 0.2, 0.4$ gitt $c(0) = 5$.

Se fasit Oppgave 9.7.4 del c)

Oppg. 9.1.2: Oppgave 9.7.4 del c) Skriv et program som løser differensialligningen $\frac{dV}{dt} = kV^{(2/3)}$, og tegner løsningskurven for $V_0 = 0.1, 0.3, 0.5$ gitt $V(0) = 2$.

Seksjon 9.9

Fasit - Oppgave 9.9.4

Oppg. 9.1.3: Oppgave 9.9.4 Betrakt startverdiproblemet

$$\frac{dy}{dx} = 3xe^{-y}, \quad y(0) = 2,$$

- Bruk Eulers metode med $h = 0.2$ til å bestemme $y(0.6)$ ved å bruke $h = 0.2$.
- Løs startverdiproblemet ved å løse differensialligningen, og sammenlign svaret med a).
- Skriv et program som bruker Eulers metode med $h = 0.2$ til å løse startverdiproblemet, og som skriver ut løsningen i en tabell der man ser at $x_{n-1}, y_{n-1}, dy/dx$ og (x_n, y_n) for hvert steg.

Blandede oppgaver

Se fasit - oppgave 11

Oppg. 9.1.4: Oppgave 11 Løs startverdiproblemet, og sett opp et program som løser startverdiproblemet, og skisserer løsningskurven i samme koordinatsystem: $\frac{dy}{dt} = -ay^2 + by$, $y(0) = 5$ når:

- $a = 0.1$ og $b = 0.2$
- $a = 0.1$ og $b = 0.01$
- $a = 0.01$ og $b = 0.2$.

Se fasit - oppgave 24 og 25

Oppg. 9.1.5: Oppgave 24 og 25 Skriv et program som løser differensialligningen og tegner løsningskurvene til følgende startverdiproblem:

$$\frac{dy}{dt} = -2ty, y(0) = 2$$

$$\frac{dy}{dt} = -2ty^2, y(0) = 2$$

Se fasit- oppgave 26

Oppg. 9.1.6: Oppgave 26 Skriv et program som løser differensialligningen og tegner løsningskurven til startverdiproblemet:

$$x'' + 0.4x' + 0.05x = \cos(2t), x(0) = 1, x'(0) = 0$$

Hint: Skriv differensialligningen som et lineært differensialligningssystem og bruk Eulers metode.

Se fasit - oppgave 28

Oppg. 9.1.7: Oppgave 28 Torricellis lov sier at når væske strømmer ut av et lite hull i bunnen av et kar, så er hastigheten v i utløpet gitt ved $v = \sqrt{2gh}$, der h er høydeforskjellen mellom væskeoverflaten og åpningen, og g er tyngdeakselerasjonen:

$$\frac{dh}{dt} = \frac{1}{A} \cdot (v - k\sqrt{2gh}), h(0) = 2$$

Gitt $v = 0,125 \text{ m}^3/\text{min}$, $k = 0,0225$, $A = 1 \text{ m}^2$, $g = 9,8 \text{ m/s}^2$. Skriv et program som løser differensialligningen og tegner løsningskurven til vannhøyden.

Kapittel 10

Lineær algebra

I dette kapittelet skal vi kort gå gjennom noen funksjoner vi kan bruke i Python når vi jobber med vektorer og matriser. Mer utfyllende informasjon kan finne her: <https://numpy.org/doc/stable/user/quickstart.html>.

10.1 Vektorer

Å definere vektorer kan vi gjøre ved hjelp av pakken *numpy*. I kodesnutten under definerer vi en vektor $x = [1, 2, 3, 4]$:

```
1 import numpy as np
2
3 x = np.array([1,2,3,4])
```

En viktig ting å notere seg her, er at Python starter å telle på 0. Det vil si at det første elementet i en vektor får vi ved å be Python gi oss element nr. 0. Bygger vi videre på kodesnutten over, kan vi printe det første elementet i vektoren, og det vil se slik ut:

```
1 import numpy as np
2
3 x = np.array([1,2,3,4])
4 print(x[0])
```

Som sagt, vil Python her gi oss det første elementet i x , altså 1.

10.2 Matriser

Matriser kan defineres på en lignende måte som vektorene:

```
1 import numpy as np
2
3 M = np.array([[1,2],[3,4]])
4 print(M)
```

Resultatet av kodesnutten over er:

```
[[1,2]
 [3,4]].
```

Ønsker vi å hente ut et element av en matrise, må vi presisere plasseringen til elementet ved hjelp av radnummeret og kolonnennummeret det sitter på. Igjen er det viktig å huske på at første rad og kolonne vil være nr. 0. Det vil si at dersom vi ønsker å hente ut elementet 3 av matrisen M over, gjør vi dette slik:

```

1 import numpy as np
2
3 M = np.array([[1,2],[3,4]])
4 print(M[1,0])

```

Dersom vi ønsker å hente ut hele rader eller kolonner fra matrisen, gjøres dette ved å angi enten rad- eller kolonnennummeret og kolon i den andre indeksen. For eksempel vil kodesnutten under gi oss hele rad nr. 2 i matrisen M .

```

1 print(M[1,:])

```

Ønsker vi å hente ut for eksempel kolonne nr. 1 kan vi bruke

```

1 print(M[:,0])

```

Noen ofte brukte matriser

Noen matriser som vi bruker mye, er det egne funksjoner for å lage i numpy. Dette gjelder matriser der alle elementene er null, matriser der alle elementene er 1 og identitetsmatrisen. Ønsker vi å lage en 5×5 -matrise der alle elementene er 0, kan dette gjøres på følgende måte:

```

1 np.zeros([5,5])

```

Tilsvarende kan en 5×5 -matrise der alle elementene er 1 defineres slik:

```

1 np.ones([5,5])

```

og til slutt kan vi definere en 5×5 -identitetsmatrise med følgende kommando:

```

1 np.eye(5)

```

Hvis vi ønsker å endre en verdi på et element i en vektor eller matrise, spesifiserer vi det elementet vi vil endre, og angir den nye verdien med likhetstegn. Hvis vi for eksempel vil endre elementet i rad 2, kolonne 2 i matrisen vår, M , over, kan vi skrive:

```

1 M[1,1] = 7

```

hvis vi nå printer M , ser vi at elementet har fått verdien 7.

10.3 Vektor-matriseoperasjoner

Numpy-pakken inneholder også innebygde funksjoner som kan være nyttige for oss når vi jobber med vektorer og matriser. Mer om disse funksjonene kan dere lese om her: <https://numpy.org/doc/stable/reference/routines.linalg.html>. Noen funksjoner som vi kan få bruk for er:

1. **np.transpose(M)**: Denne funksjonen gir oss den transponerte matrisen av M .
2. **np.dot(a,b)**: Denne funksjonen regner ut prikkproduktet (skalarproduktet) av vektorene a og b .
3. **np.matmul(M,N)**: Denne funksjonen regner ut matriseproduktet av to matriser. Den kan også regne ut matrise-vektorprodukter. Vi vet fra teorien at $a^T M$ ikke nødvendigvis gir en vektor med de samme elementene som Ma , og avhengig av hvilket produkt vi ønsker, må vi spesifisere argumentene i funksjonen riktig. **np.matmul(a,M)** vil gi oss produktet $a^T M$, mens **np.matmul(M,a)** vil gi oss produktet Ma .

4. `np.linalg.det(M)`: Denne funksjonen finner determinanten til M .
5. `np.linalg.inv(M)`: Denne funksjonen finner den inverse matrisen til M .
6. `np.linalg.solve(M,a)` løser likningssystemet $Mx = a$ for x .

Oppg. 10.3.1: Gitt matrisen: $\mathbf{A} = \begin{bmatrix} 0 & 3 \\ -3 & 0 \end{bmatrix}$ og $\mathbf{B} = \begin{bmatrix} 1 & 3 \\ 3 & 2 \end{bmatrix}$

Skriv et Python-program som regner ut $\mathbf{A} + \mathbf{B}$, $\mathbf{A} - \mathbf{B}$, $\mathbf{AB}$, $\mathbf{A}^T$ (transposen til $\mathbf{A}$), $\det(\mathbf{A})$, $\det \mathbf{B}$, $\mathbf{B}^{-1}$, egenverdi og egenvektorene til $\mathbf{A}$.

Oppg. 10.3.2: Gitt matrisen: $M = \begin{bmatrix} 3 & -2 & 4 & -2 \\ 5 & 3 & -3 & -2 \\ 5 & -2 & 2 & -2 \\ 5 & -2 & -3 & 3 \end{bmatrix}$

- a) Diagonaliser M ved $M = PDP^{-1}$, der P er matrisen til egenvektorene, og D er en diagonalmatrise som har egenvektorene som hoved diagonal.
Hint: Skriv et Python-program og sjekk om M er diagonaliserbar med `print(M.is_diagonalizable())`.
- b) Regn ut A^{20} og sjekk dette mot diagonalisering.
Hint: $A^{20} = PD^{20}P^{-1}$

Diskrete dynamiske systemer

Oppg. 10.3.3: Et diskret dynamisk system som beskriver bestanden til rådyr ($x(t)$) og bestanden til kaniner ($y(t)$) gitt ved:

$$\begin{aligned}x_{n+1} &= 0,86x_n + 0,08y_n \\ y_{n+1} &= -0,15x_n - 1,15y_n\end{aligned}$$

- a) Skriv systemet på matriseform: $\vec{x}_{n+1} = \mathbf{M}\vec{x}_n$, der $\vec{x}_n = \begin{pmatrix} x_n \\ y_n \end{pmatrix}$.

Regn ut $\vec{x}_1$ og $\vec{x}_2$, gitt $\vec{x}_0 = \begin{pmatrix} 100 \\ 300 \end{pmatrix}$.

- b) Bestem egenverdiene og egenvektorene til $\mathbf{M}$.

Sett opp et Python-program som beregner x_4 ved to metoder: $\mathbf{x}_4 = \mathbf{A}^4\mathbf{x}_0$ og $x_4 = \mathbf{PD}^4\mathbf{P}^{-1}\mathbf{x}_0$, der $\mathbf{P}$ har egenverdiene som søylene og D er en diagonalmatrise med egenverdiene på hoved diagonalen.

Løsningen til et slikt system med to reelle distinkte egenverdier kan skrives på formen $\vec{x}_n = C_1\vec{v}_1(\lambda_1)^n + C_2\vec{v}_2(\lambda_2)^n$ der $\vec{v}_1$ og $\vec{v}_2$ er egenvektorene til egenverdiene λ_1 og λ_2 .

Sett opp løsningen.

Eksempel 10.3.1 (En fordel med koding): Nå skal vi se på et eksempel, som, selv om matrisen og vektorene fremdeles er små, viser en fordel med å bruke koding. Se gjennom eksempelet på side 353 i læreboken. Dette er et eksempel på studium av populasjonsvekst. Anta nå at vi har to aldersgrupper:

1. De under 25 år
2. De fra 25 år opptil 50 år

Hvis vi nå lar $F_1 = 1.8$, $F_2 = 0.1$ og $p_1 = 0.9$, får vi likningssystemet:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1.8 & 0.1 \\ 0.9 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Vi antar nå at etter en tidsperiode på $T = 25$ år, så er tilstanden $y_1 = 216$ og $y_2 = 99$. Ved hjelp av likningssystemet kan vi finne ut hva tilstanden var ved start. Til dette kan vi bruke innsettingsmetoden eller Gausseliminasjon, og løsningene kan vi finne relativt enkelt, siden likningssystemet vårt er så lite.

Men hva om vi ønsker å undersøke en finere inndeling av aldersgruppene? I stedet for å la det være 25 års forskjell, lar vi det nå være 10 år. Da får vi denne inndelingen:

1. De under 10 år
2. De fra 10 år opptil 20 år
3. De fra 20 år opptil 30 år
4. De fra 30 år opptil 40 år
5. De fra 40 år opptil 50 år.

Vi tenker oss at $F_1 = 0$, $F_2 = 0.2$, $F_3 = 1.2$, $F_4 = 0.7$, $F_5 = 0.1$ og $p_1 = 0.8$, $p_2 = 0.8$, $p_3 = 0.9$, $p_4 = 0.8$. Etter en periode på $T = 10$ år er antallet i hver aldersgruppe gitt ved

$$y = \begin{bmatrix} 295 \\ 112 \\ 144 \\ 84 \\ 108 \end{bmatrix}.$$

Hvis vi nå spør oss om hva antallet i de ulike aldersgruppene var ved start, kan vi raskt se at vi kommer til å bruke en del lengre tid enn i sted dersom vi skal løse dette for hånd. Fremdeles er likningssystemet vårt veldig lite, men ved hjelp av funksjonen `np.linalg.solve(M,y)` i numpy-pakken får vi svaret umiddelbart, og vi har spart oss for mye tid og regning.

Når vi allerede på et så lite eksempel kan se nytten av å bruke koding, tenk deg hvordan situasjonen er dersom vi har *store* systemer vi vil løse. For eksempel kan matrisene være av størrelse én million $\times$ én million, og det blir praktisk talt umulig for oss å løse det for hånd. $\diamond$

Oppg. 10.3.4: Gitt matrisen: $\mathbf{A} = \begin{bmatrix} 0 & 3 \\ -3 & 0 \end{bmatrix}$ og $\mathbf{B} = \begin{bmatrix} 1 & 3 \\ 3 & 2 \end{bmatrix}$

Skriv et Python-program som regner ut $\mathbf{A} + \mathbf{B}$, $\mathbf{A} - \mathbf{B}$, $\mathbf{AB}$, $\mathbf{A}^T$ (transposen til $\mathbf{A}$), $\det(\mathbf{A})$, $\det \mathbf{B}$, B^{-1} , egenverdi og egenvektorene til $\mathbf{A}$.

Oppg. 10.3.5: Gitt matrisen: $M = \begin{bmatrix} 3 & -2 & 4 & -2 \\ 5 & 3 & -3 & -2 \\ 5 & -2 & 2 & -2 \\ 5 & -2 & -3 & 3 \end{bmatrix}$

- a) Diagonaliser M ved $M = PDP^{-1}$, der P er matrisen til egenvektorene, og D er en diagonalmatrise som har egenvektorene som hoved-diagonal. Hint: Skriv et Python-program og sjekk om M er diagonaliserbar med `print(M.isdiagonalizable())`.
- b) Regn ut A^{20} og sjekk dette mot diagonalisering.
Hint: $A^{20} = PD^{20}P^{-1}$

Diskrete dynamiske systemer

Oppg. 10.3.6: Et diskret dynamisk system som beskriver bestanden til rådyr ($x(t)$) og bestanden til kaniner ($y(t)$) er gitt ved:

$$\begin{aligned}x_{n+1} &= 0,86x_n + 0,08y_n \\ y_{n+1} &= -0,15x_n - 1,15y_n\end{aligned}$$

- a) Skriv systemet på matriseform: $\vec{x}_{n+1} = \mathbf{M}\vec{x}_n$, der $\vec{x}_n = \begin{pmatrix} x_n \\ y_n \end{pmatrix}$.

Regn ut $\vec{x}_1$ og $\vec{x}_2$, gitt $\vec{x}_0 = \begin{pmatrix} 100 \\ 300 \end{pmatrix}$.

- b) Bestem egenverdiene og egenvektorene til $\mathbf{M}$.

Sett opp et Python-program som beregner x_4 ved to metoder: $\mathbf{x}_4 = \mathbf{A}^4\mathbf{x}_0$ og $x_4 = \mathbf{P}\mathbf{D}^4\mathbf{P}^{-1}x_0$, der $\mathbf{P}$ har egenverdiene som søylene, og $\mathbf{D}$ er en diagonalmatrise med egenverdiene på hoveddiagonalen.

Løsningen til et slikt system med to reelle distinkte egenverdier kan skrives på formen $\vec{x}_n = C_1\vec{v}_1(\lambda_1)^n + C_2\vec{v}_2(\lambda_2)^n$ der $\vec{v}_1$ og $\vec{v}_2$ er egenvektorene til egenverdiene λ_1 og λ_2 .

Sett opp løsningen.

Eksempel 10.3.2 (Modellering med programmering): Nå skal vi se på et eksempel fra MAT102-kompendium [?] som tar utgangspunkt fra eksempel på side 353 i læreboken [1] (se gjennom eksempelet på side 353 i læreboken [1]). Dette er et eksempel på en studie av populasjonsvekst. Anta nå at vi har to aldersgrupper:

1. De under 25 år
2. De fra 25 år opptil 50 år

Hvis vi nå lar $F_1 = 1.8$, $F_2 = 0.1$ og $p_1 = 0.9$, får vi likningssystemet:

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1.8 & 0.1 \\ 0.9 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Vi antar nå at etter en tidsperiode på $T = 25$ år så er tilstanden $y_1 = 216$ og $y_2 = 99$. Ved hjelp av likningssystemet kan vi finne ut hva tilstanden var ved start. Til dette kan vi bruke innsetningsmetoden eller Gauss-eliminasjon, og løsningene kan vi finne relativt enkelt, siden likningssystemet vårt er så lite.

Men hva om vi ønsker å undersøke en finere inndeling av aldersgruppene? I stedet for å la det være 25 års forskjell, lar vi det nå være 10 år. Da får vi denne inndelingen:

1. De under 10 år
2. De fra 10 år opptil 20 år
3. De fra 20 år opptil 30 år
4. De fra 30 år opptil 40 år
5. De fra 40 år opptil 50 år.

Vi tenker oss at $F_1 = 0$, $F_2 = 0.2$, $F_3 = 1.2$, $F_4 = 0.7$, $F_5 = 0.1$ og $p_1 = 0.8$, $p_2 = 0.8$, $p_3 = 0.9$, $p_4 = 0.8$. Etter en periode på $T = 10$ år er antallet i hver aldersgruppe gitt ved

$$y = \begin{bmatrix} 295 \\ 112 \\ 144 \\ 84 \\ 108 \end{bmatrix}.$$

Hvis vi nå spør oss om hva antallet i de ulike aldersgruppene var ved start, kan vi raskt se at vi kommer til å bruke en del lengre tid enn i stedet dersom vi skal løse dette for hånd. Ved å bruke programmering kan vi enkelt løse problemet. $\diamond$

10.4 Fasit for programmeringsoppgaver i kapittel 11

Blandede oppgaver

Se fasit - Oppgave 3

Oppg. 10.4.1: Oppgave 3 Populasjonsdynamikken i predator/byttedyr-system kan beskrives ved hjelp av Lotka-Volterra-modellen. Modellen består av to likninger, en for vekst i byttedyrpopulasjonen (kaniner) og en for vekst i predatorpopulasjonen (rovdyr). Ut fra Lotka-Volterras predator/byttedyr-modell kan vi forstå hvorfor svingninger i predator- og byttedyrpopulasjoner kan oppstå. Følgende differensialligningssystem kan beskrive bestandsendringen til kaniner ($x = x(t)$) og rovdyr ($y = y(t)$) ved tiden (antall måneder):

$$\begin{aligned} \frac{dx}{dt} &= x(a - by) \\ \frac{dy}{dt} &= -y(c - dx) \end{aligned}$$

- a) Skriv et Python-program som kan skissere fasediagram, gitt $a = b = c = d = 1$, og $x(0) = 1.5$, $y(0) = 1$:
- for bestandsendringen til kaniner ($x = x(t)$) ved tiden t
 - bestandsendringen til rovdyr ($y = y(t)$) ved tiden t
 - bestandsendringen til rovdyr versus kaniner

Hint:

La, $x = N(0)$ og $y = N(1)$ og programmet kan løse differensialligningen ved å bruke *Odeint*. Man kan dele tidsaksen i 100-deler: `ts = np.linspace(0, 12, 100)`.

- b) Sett i programmet bestandsendringen til rovdyr versus kaniner ved å bruke $1 \leq x(0) \leq 3$.

Hint:

Se fasit - Oppgave 4

Oppg. 10.4.2: Oppgave 4 Her tar vi et eksempel fra MAT102-kompendium [?] som bruker eksempel 9.9.3 fra læreboken [1] med et lodd med masse M , som henger i en fjær med stivhet k . Vi antar at loddet blir trukket nedover og så blir sluppet. Formelen som beskriver hvordan loddets posisjon, x , varierer med tiden, t , kan beskrives av funksjonen

$$Mx'' + \alpha x' + kx = 0,$$

hvor α representerer luftmotstanden. Ved teorem 9.9.11 i læreboken kan vi finne en analytisk løsning til denne differensiallikningen. Skal vi derimot løse likningen numerisk ved hjelp av Eulers metode, kan vi innføre $y = x'$, og skrive differensiallikningen som et system:

$$\begin{aligned}x' &= y \\ y' &= -\frac{\alpha}{M}y - \frac{k}{M}x\end{aligned}$$

Vi tar nå utgangspunkt i at $M = 1$, $\alpha = 2$ og $k = 1$. I tillegg har vi initialbetingelsene $x(0) = 1$ og $x'(0) = 0$. Skriv et program som løser dette problemet, og tegner løsningskurven. Kjør koden med verdier $\alpha \in [-3, 3]$. Vi kan si følgende om ulike verdier for α . Etter testene du nettopp har kjørt, la du kanskje merke til:

- $\alpha = 0$: I dette tilfellet vil den analytiske løsningen være en svigning med konstant utslag. Loddet vil svinge med samme utslag for all framtid. Koden over viser imidlertid kanskje ikke det samme, og dette skyldes feilen i Eulers metode. Prøv å redusere steglengden og øke antall punkter for å få dette tydeligere fram.
- $0 < \alpha < 2$: gir en dempet svigning, der utslaget blir mindre og mindre.
- $\alpha > 2$: representerer en fjær med så mye friksjon at det egentlig ikke gir noen svigning (tenk på en rusten fjær).
- $-2 < \alpha < 0$: gir en funksjon som svinger med større og større utslag.
- $\alpha < -2$: gir en funksjon som går mot minus uendelig. Både dette tilfellet og det forrige representerer ikke egentlige svigninger, fordi det må *tilføres* ekstra energi for å få denne oppførselen.

Kapittel 11

Funksjoner av flere variabler

11.1 Optimering

I matematikk handler *optimering* om å finne den best mulige løsningen på et problem. For funksjoner av flere variabler er maksimums- og minimumsproblemer der variablene kan ha hvilke som helst verdier i definisjonsområdet til funksjonen, og maksimums- og minimumspunkter når det er begrensninger (såkalte bibetingelser) på hvilke verdier variablene kan tilfredsstille optimeringsproblemet under bibetingelsen.

Gradient descent-metoden (*gradientnedstigningsmetoden*) er en optimaliseringsalgoritme som ofte brukes til å trene maskinlæringsmodeller og nevralt nettverk. Treningsdata hjelper disse modellene å lære over tid, og *kostnadsfunksjonen* (*objektivfunksjonen* eller *målfunksjonen*) innenfor gradientnedstigning fungerer spesifikt som et barometer, som måler nøyaktigheten med hver iterasjon av parameteroppdateringer. Inntil funksjonen er nær eller lik null, vil modellen fortsette å justere sine parametere for å gi minst mulig feil. Når maskinlæringsmodeller er optimalisert for nøyaktighet, kan de være kraftige verktøy for kunstig intelligens og informatikkapplikasjoner.

11.2 Gradientnedstigningsmetoden (gradient descent)

Gradientnedstigningsmetoden (den bratteste skråningen) er en førsteordens iterativ optimaliseringsalgoritme for å finne et lokalt minimum av en deriverbar funksjon. Idéen er å ta gjentatte skritt i motsatt retning av gradienten til funksjonen på gjeldende punkt, fordi dette er retningen for den bratteste nedstigningen. Omvendt vil det å gå i retning av gradienten føre til et lokalt maksimum av den funksjonen; prosedyren er da kjent som gradientstigning. Her mener vi med optimering å bestemme punkt som minimumsverdi til funksjonen, forutsatt at funksjonen har et lokalt minimumspunkt. Optimering brukes blant annet i økonomi for kostnadsfunksjon, i fysikk for arbeid/energi, i biologi for nevralt nettverk, i kjemi for kinetiske mekanismer, i kunstig intelligens, i mønstergjenkjenning og i genetiske algoritmer.

For funksjoner av en variabel kan man finne den minste verdien til en funksjon $y = f(x)$ ved å finne nullpunktet til den deriverte $f'(x) = 0$, som lokaliserer stasjonære punkt (et minimums-, et maksimums-, eller et vendepunkt) og kan identifiseres ved hjelp av andrederivert-test. Her skal vi bestemme minimumspunktet til $F(\mathbf{x})$. Det vil si, kost-funksjonen F skal minimeres:

$$\min_{\mathbf{x}} F(\mathbf{x}), \quad \text{der } \mathbf{x} = (x_1, x_2, \dots, x_n) \quad (11.1)$$

altså å bestemme $\mathbf{x}$ som gir den minste verdien til F . Notasjonen $f(x)$ brukes for funksjoner av en variabel, og $F(\mathbf{x})$ for funksjoner av to eller flere variabler.

Gradient descent-metoden

Gitt et vilkårlig valgt startpunkt $\mathbf{x}_0$, er metoden definert som følger:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \gamma \nabla F(\mathbf{x}_n). \quad (11.2)$$

der $F(\mathbf{x}_n)$ er kostfunksjonen, $\mathbf{x}_n$ med den forrige verdien og et steg i retningen til $-\nabla F$ og parameteren γ er steglengden som kalles *læringsraten*. For en veldig liten γ koster flere steg til å komme frem til minimumspunktet. Parameterne i algoritmen oppdateres i motsatt retning av gradientene med læringssteglengden γ som bestemmer hvor stor oppdatering som utføres. $\mathbf{x}$ blir beregnet med den forrige verdien og et steg i retningen til $-\nabla F$. Størrelsen til γ vil variere fra problem til problem. En av utfordringene med gradientnedstigning er å velge den optimale verdien for læringsraten γ . Hvis du velger en læringsrate som er for liten, kan gradientnedstigningsalgoritmen ta veldig lang tid å finne minimumsverdien til kostfunksjonen. Tilsvarende hvis man velger en læringsrate som er for stor, vil algoritmen overskride minimumsverdien for kostfunksjonen, og kanskje aldri oppnå den optimale løsningen. Man kan begynne med en læringsrate mindre enn 1, for eksempel 0.8, og senke verdien til algoritmen konvergerer mot et minimumspunkt.

Estimeringsnøyaktighet

Det er flere faktorer som kan bidra til bedre estimeringsnøyaktighet. En vilkår/betingelse, som definerer hva som er en *god nok* løsning, kan sørge for at programmet fortsetter å kjøre til det oppnås et ønsket nøyaktighet. Gitt et vilkårlig valgt startpunkt $\mathbf{x}_0$, er metoden definert som følger:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \gamma \nabla F(\mathbf{x}_n). \quad (11.3)$$

1. Lengden til gradienten (eller normen til gradienten) er liten, $\|\nabla F\| < \text{toleranse}$, der *toleranse* er en liten verdi. I minimumspunktet er gradienten 0. Det kan ta flere steg å komme dit fordi stegene er av samme størrelsesorden som gradienten. Når gradienten begynner å bli liten, og kostfunksjonen endrer seg veldig lite, blir derfor stegene veldig små.
2. Kostfunksjonen begynner å øke: $F(\mathbf{x}_{i+1}) > F(\mathbf{x}_i)$
3. Kostfunksjonen holder seg nesten konstant og endrer seg veldig lite: $F(\mathbf{x}_i) - F(\mathbf{x}_{i+1}) < \text{toleranse}$, der *toleranse* er et lite tall.

$\mathbf{x}$ oppdateres altså med den forrige verdien og et steg i retningen til $-\nabla F$. Vi justerer lengden på steget med parameteren γ . Hvor stor γ skal være vil variere fra problem til problem. I praksis må vi prøve oss frem. Om vi velger γ for stor, vil ikke metoden fungere, og om vi velger γ veldig liten, vil trenge veldig mange steg for å nå frem til et minimum. Den beste 'idéelle' læringsraten i hvert steg kan identifiseres og konfigureres med fornuftige standardinnstillinger, diagnostisere atferd, og ved å utvikle en sensitivitetsanalyse. Det er mange faktorer som vil påvirke hvor robust og effektiv en gradientnedstigningsalgoritmen kan løse et problem. Legg merke til at ingen av kravene ovenfor kan sikre at algoritmen vil konvergere, og ender i et minimum.

Robusthet og begrensninger

Det er mange faktorer som vil påvirke hvor robust og effektiv en gradientnedstigningsalgoritmen kan løse et problem. Algoritmen har ingen garanti for konvergens alltid og overalt, og bestemmer et minimum. For eksempel for en funksjon som ikke er strengt synkende, blir den konstant eller nesten konstant på et intervall/område før den synker videre.

Antall steg

Det er mulig å legge inn en parameter t som teller antall iterasjoner i stopp-kriterium i programkoden, som er avhengig av:

- valg av steglengden
- valg av startverdiene
- estimeringsnøyaktighet (toleranse)

Eksempel 11.2.1: Å finne minimum i 1 dimensjon ($f(x)$, er en funksjon av en variabel).

Algoritmen fungerer for funksjoner med både én og flere variable. Vi skal nå se på et eksempel i 1 dimensjon, altså med en funksjon $f(x)$ av en enkel variabel. I denne sammenheng vil gradienten vil være den deriverte $\nabla f(x) = f'(x)$.

Koden under løser optimeringsproblemet $\min_x f(x)$, der $f(x) = x^2 - 6x + 1$ og $f'(x) = 2x - 6$. Kostfunksjonen $f(x)$ er plottet i figur 11.1b.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 ' Definerer uttrykket til kostfunksjonen og gradienten'
5
6 def F(x):
7     F= x**2 - 6*x +1
8     return F
9
10 def gradF(x):
11     gradF= 2*x-6
12     return gradF
13
14 x0 = 1 #startpunkt
15 n = 100 # maks antall iterasjoner
16 tol = 10**-3 # estimeringsnøyaktighet
17 gamma = .35; # læringsrate
18 x = x0 # initialiserer x
19 t=0 # initialiserer antall iterasjoner
20 cost = [F(x)]
21
22 # liste over x , F(x) og gradF(x)
23 print(f'x\t t \t F(x) \t gradF(x) \t F(n) - F(n-1)')
24 for i in range(n):
25     x = x - gamma * gradF(x)
26     cost.append(F(x))
27     # Skriver ut x , F(x), gradF(x) og endringen til kostnadsfunksjonen
28     print(f'{x:.5f} {F(x):.5f} {gradF(x):.5f} {(cost[-2]-cost[-1]):.5f}
29     ')
30     t+= 1
31     if abs(gradF(x))<tol:
32 # if np.abs(cost[-1] - cost[-2]) < tol:
33         break
34 print(f'Det tok {t} steg å nå minimumspunktet {x}')
35
36 xstart=0
37 xslutt=6
38 steglengde=0.25
39 x = np.arange(xstart, xslutt, steglengde)
40 y = F(x)
41 dy= gradF(x)
42 plt.plot(x,y, '--')
43 plt.plot(x,dy, '--')
44 plt.legend(['Kostfunksjonen F(x)', 'Den deriverte dF(x)'] )
45 plt.xlabel('x')
46 plt.ylabel('y')
```

```

47 plt.grid()
48 plt.show()

```

Ved å kjøre programkoden får vi:

```

1 x      F(x)      gradF(x)  F(n) - F(n-1)
2 2.40000 -7.64000 -1.20000 -3.64000
3 2.82000 -7.96760 -0.36000 -0.32760
4 2.94600 -7.99708 -0.10800 -0.02948
5 2.98380 -7.99974 -0.03240 -0.00265
6 2.99514 -7.99998 -0.00972 -0.00024
7 2.99854 -8.00000 -0.00292 -0.00002
8 2.99956 -8.00000 -0.00087 -0.00000
9 Det tok 7 steg å nå minimumspunktet 2.9995626

```

Løsningen har 3 korrekte desimaler. Om vi hadde brukt flere steg (mindre γ), hadde svaret blitt enda mere nøyaktig. Valg av x_0 forteller om man leter etter minimumspunktet (x_{min}) på venstre ($x_0 > x_{min}$) eller høyre side ($x_0 < x_{min}$) av x_0 . For eksempel for $\gamma = 0.3$ blir svaret:

```

1 x      F(x)      gradF(x)  F(n) - F(n-1)
2 2.20000 -7.36000 -1.60000 -3.36000
3 2.68000 -7.89760 -0.64000 -0.53760
4 2.87200 -7.98362 -0.25600 -0.08602
5 2.94880 -7.99738 -0.10240 -0.01376
6 2.97952 -7.99958 -0.04096 -0.00220
7 2.99181 -7.99993 -0.01638 -0.00035
8 2.99672 -7.99999 -0.00655 -0.00006
9 2.99869 -8.00000 -0.00262 -0.00001
10 2.99948 -8.00000 -0.00105 -0.00000
11 2.99979 -8.00000 -0.00042 -0.00000
12 Det tok 10 steg å nå minimumspunktet 2.9997902848

```

Følgende programkode kunne også skrives på denne måten, der gjentakelen med **for**-løkken og stopp-kriterium med **if** settes opp ved hjelp av **while**-løkke:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def F(x):
5     F= x**2 - 6*x +1
6     return F
7 def gradF(x):
8     gradF= 2*x-6
9     return gradF
10
11 x0 = 1
12 N = 100 # maks antall iterasjoner
13 tol = 10**-3
14 gamma =.35;
15 x = x0
16 t=0
17 n=0
18 cost = [F(x)]
19
20 print(f'x\t t \t F(x) \t gradF(x) \t F(n) - F(n-1)')
21
22 while (abs(gradF(x)) > tol) and (n < N):

```

```

23     x = x - gamma * gradF(x)
24     n +=1
25     cost.append(F(x))
26     print(f'{x:.5f} {F(x):.5f} {gradF(x):.5f} {(cost[-2]-cost[-1]):.5f}
    ')
27     t+= 1
28
29 print(f'Det tok {t} steg å nå minimumspunktet {x}')

```

Eksempel 11.2.2: Vi ønsker å bruke gradientnedstigningsmetoden til å bestemme et minimumspunkt for $z = f(x, y) = (x - 3)^2 + (2y - 1)^2$.

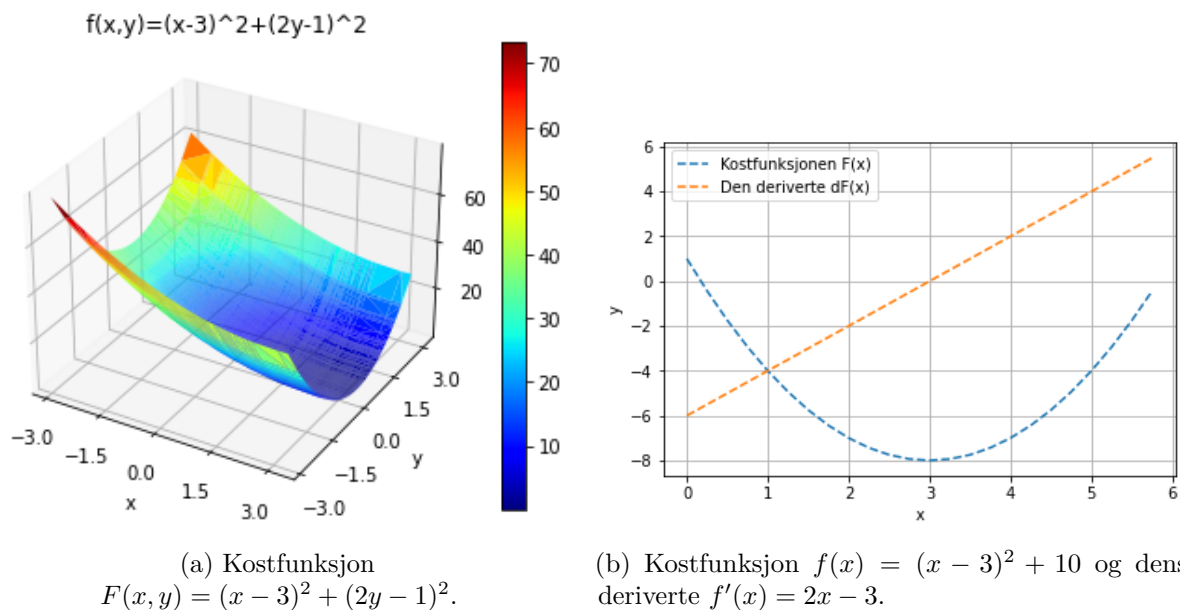
```

1 import numpy as np
2 def F(x,y):
3     return (x-3)**2 + (2*y-1)**2
4 def gradF(x,y):
5     return 2*(x-3), 4*(2*y-1)
6
7 x0 = 1
8 y0 = 1
9 n = 100
10 tol = 10**-4
11 gamma =.2;
12 x = x0
13 y = y0
14 t=0
15 cost = [F(x,y)]
16
17 print(f'x\t y \t      F(n)-F(n-1)')
18 for i in range(n):
19     gradFx, gradFy = gradF(x, y)
20     x = x - gamma * gradFx
21     y = y - gamma * gradFy
22     cost.append(F(x,y))
23     print(f'{x:.5f} {y:.5f} {(cost[-1]-cost[-2]):.5f}')
24     t+= 1
25     if np.abs(cost[-1] - cost[-2]) < tol:
26         break
27
28 print(f'Minimumspunktet er ({x},{y})')
29 print(f'Det tok {t} steg å nå minimumspunktet')

```

Ved å kjøre programmet, får vi:

	x	y	F(n)-F(n-1)
1	1.80000	0.20000	-3.20000
2	2.28000	0.68000	-1.15200
3	2.56800	0.39200	-0.41472
4	2.74080	0.56480	-0.14930
5	2.84448	0.46112	-0.05375
6	2.90669	0.52333	-0.01935
7	2.94401	0.48600	-0.00697
8	2.96641	0.50840	-0.00251
9	2.97984	0.49496	-0.00090
10	2.98791	0.50302	-0.00032
11	2.99274	0.49819	-0.00012
12			



Figur 11.1: (a) Gradienten $\nabla F = [2(x - 3), 4(2y - 1)]$, en vektor med to komponenter, som kan peke oppover eller nedover (b) Retningen til gradienten vil være det samme som fortegnet til den deriverte $f'(x) = 2(x - 3)$, altså pluss eller minus.

```

13 2.99565 0.50109 -0.00004
14 Minimumspunktet er (2.995646435328, 0.501088391168)
15 Det tok 12 steg å nå minimumspunktet

```

Finne minimum av funksjon av to eller flere variabler

Vi skal løse optimeringsproblemet

$$\min_{x,y} (x - 3)^2 + (2y - 1)^2 \quad (11.4)$$

Da gir gradienten en mer naturlig retning (ikke bare høyre og venstre). I eksempelet under skal vi løse dette problemet, kan vi også løse analytisk og vil da få løsningen bli:

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -\frac{2}{3} \\ \frac{4}{3} \end{bmatrix}.$$

La kostfunksjonen hete $F(x, y)$. I figur 11.1a ser vi et plot av funksjonen $F(x, y)$. Først partiellderiverer vi og finner gradienten

$$\nabla F(x, y) = \begin{bmatrix} 2(x - 3) \\ 4(2y - 1) \end{bmatrix}$$

Den analytiske løsningen er da $(x, y) = (3, 0.5)$. Fra grafen 11.1a kan vi også se at den analytiske løsningen av minimumspunktet ser rett ut. Programkoden for funksjon av en variabel modifiseres til å løse optimeringsproblemet av funksjoner av to variable.

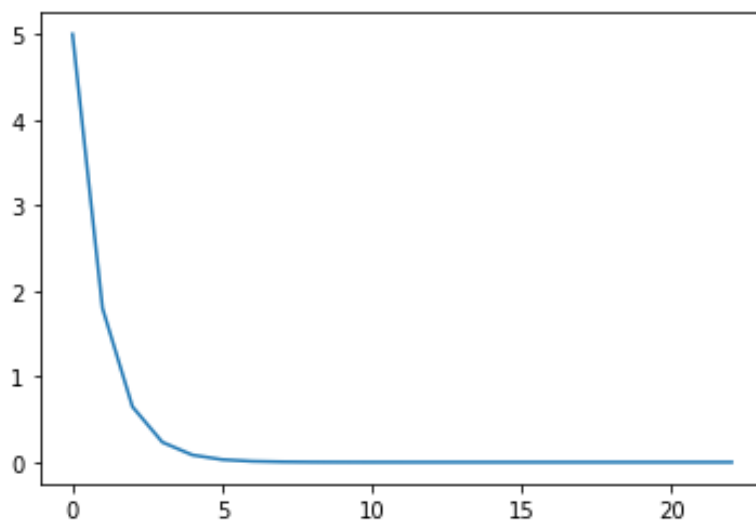
```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Definerer kostfunksjonen, gradientfunksjonen og variable
5
6 # Setter maks antall iterasjoner
7 n = 100
8
9 # Setter startverdier for søket
10 x0 = 1
11 y0 = 1
12
13 # Definerer toleranserverdien for stoppkriterium
14 tol = 10**-4
15 # Definerer steglengden
16 gamma = .2;
17
18 # Definerer kostfunksjonen
19 def F(x,y):
20     z = np.array([(x-3)**2 + (2*y-1)**2 ])
21     return z
22
23 # Definerer gradientfunksjonen
24 def gradF(x,y):
25     gradient = np.array([2*(x-3),4*(2*y-1)])
26     return gradient
27
28 # Definerer startverdiene
29 x0 = np.array([1])
30 y0 = np.array([1])
31
32
33 # Initialiserer x og y med startpunktene
34 x = x0
35 y = y0
36
37 # Regner ut kostfunksjonen
38 cost = F(x,y)
39
40 # Initialiserer antall steg (antall iterasjoner)
41 t=0
42
43 # Optimeringsalgoritmen
44
45 for i in range(n):
46     x, y = [x,y] - gamma * gradF(x,y) #oppdaterer x og y
47     cost = np.append(cost, F(x,y))
48     t+= 1
49     if np.linalg.norm(gradF(x,y)) < tol:
50         break
51
52 print(f'Minimumspunktet er ({x},{y})')
53 print(f'det tok {t} steg å nå minimumspunktet')
54
55 plt.plot(cost)

```

```
56 # Tegner grafen til kostfunksjonen
```

Selv om vi søker bare etter minimumspunktet, $x = 2.99997368$, $y = 0.50000658$, så er det praktisk å plote utviklingen av kostfunksjonen i tillegg. Det kan hjelpe oss til å se om optimeringen har konvergert til én løsning, eller om vi må endre på noen av parameterene våre. Plottet som returneres av koden over, er vist i figur 11.2.



Figur 11.2: Endringen til kostfunksjonen til $\min_{x,y} (x-3)^2 + (2y-1)^2$.

For å studere utviklingen av optimeringen er det mulig å se på normen til gradienten eller endringen til kostfunksjonen. Normen, lengden til gradientvektoren er en skalar. Test koden over med $\gamma = 0.3$ og $\gamma = 0.2$ med nøyaktighetskravene:

1. $\|\nabla F\| < \text{toleranse}$
2. $|F(\mathbf{x}_{n+1}) - F(\mathbf{x}_n)| < \text{toleranse}$

Kapittel 12

Ekstra - typiske eksamensoppgaver

Oppg. 12.0.1: En funksjon er definert på $\mathbb{R}^2$ ved at $f(x, y) = x^2 + y^2 - 6x + 4y$.

- a) Bestem gradientvektoren i punktet $P = (1, 1)$.

Regn ut hvor raskt f endrer seg i punktet P i retningen $\mathbf{v} = -3\mathbf{i} + 4\mathbf{j}$.

Et program er gitt som:

```
1 import numpy as np
2 def F(x,y):
3     return x**2 + y**2 - 6*x + 4*y
4 def gradF(x,y):
5     return HVA1, HVA2
6
7 x0 = 1
8 y0 = 1
9 n = 100
10 tol = 10**-3
11 gamma = .5;
12 x = x0
13 y = y0
14 t=0
15 cost = [F(x,y)]
16 for i in range(n):
17     gradFx, gradFy = gradF(x, y)
18     x = x - gamma * gradFx
19     y = y - gamma * gradFy
20     cost.append(F(x,y))
21     t+= 1
22     if np.abs(cost[-1] - cost[-2]) < tol:
23         break
24
25 print(x,y)
26 print(t)
```

- b) Hvilket problem prøver programmet å løse?

Hvilken metode er brukt til å løse problemet?

Hva skal stå for *HVA1* og *HVA2* i programmet? Du trenger ikke å beskrive detaljene i programmet.

- c) Hvor i koden er en betingelse som definerer hva som er en "god nok" løsning? Kan du endre denne betingelsen slik at programmet stopper når gradienten (eller normen til gradienten) er liten, $\|\nabla F\| < tol$, der tol er en liten verdi? Hint: bruk *np.linalg.norm* kommandoen.

- d) Hva forventer vi å få med *print(x, y)* når man kjører programmet?

Kan du si hva variabelen t prøver å telle for uten å si hva *print(t)* blir?

Oppg. 12.0.2: Et program er gitt som:

```
1 # DEL 1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def f(x,y):
6     return 0.5*y
7
8 xmin = 0
9 xmax = 10
10 N = 4
11 x = np.linspace(xmin, xmax, num = N+1)
12 deltax = (xmax - xmin)/N
13 y = np.zeros(N+1)
14 y[0] = 1
15
16 print('\tx[n]\ty[n]\t dy/\dx \ty[n+1]')
17
18 for i in range(N):
19     y[i+1] = y[i] + f(x[i],y[i])*deltax
20
21 # DEL 2
22
23     print('n=',i, '%.4f\t%.4f\t%.4f\t%.4f'%(x[i],y[i],f(x[i],y[i]),y[i]
24         +1]))
25
26 plt.plot(x,y,'--',label='Numerisk løsning')
27 plt.xlabel('x')
28 plt.ylabel('y')
29
30 def yeksakt(x):
31     return np.exp(0.5*x)
32
33 xpen = np.linspace(xmin,xmax,100)
34 plt.plot(xpen,yeksakt(xpen),label = 'ekte løsning')
35 plt.legend()
36
37 print('Den numeriske løsningen gir at y(,xmax,) er',y[N])
38 print('Den eksakte løsningen er at y(,xmax,) er',yeksa(x[N]))
```

a) Hvilket problem prøver programmet å løse?

b) Hvilken metode er brukt til å løse problemet?

Skriv kort om detaljene i programmet. Her kan du forklare den delen av programmet merket DEL 1 linje for linje (du trenger ikke å beskrive/forklare DEL 2).

c) Ved å kjøre programmet får man:

	x[n]	y[n]	dy/\dx	y[n+1]
n= 0	0.0000	1.0000	0.5000	2.2500
n= 1	2.5000	2.2500	1.1250	5.0625
n= 2	5.0000	5.0625	2.5312	11.3906
n= 3	7.5000	11.3906	5.6953	25.6289

Den numeriske løsningen gir at y(xx) er xxxxxxxx

Den eksakte løsningen er at y(xx) er 148.4131591025766

- i) Hva skal stå for xx og xxxxxxxx? (Her kan du bruke programmet og utskriften fra programmet)
- ii) Regn ut numerisk ved hjelp av Eulers metode $y(2.5)$ og $y(5)$, og sjekk svarene du får med utskriften fra programmet (utregning skal vises).
- d) Hva kan endres i programmet for å øke presisjonen i programmet?

Oppg. 12.0.3: Gitt ligningen: $1 - x + \cos(x) = 0$

- a) Forklar hvorfor ligningen har kun én løsning i intervallet $[1, 2]$.
- b) Bruk Newtons metode til å bestemme nullpunktet til funksjonen $f(x) = 1 - x + \cos(x)$, som ligger i intervallet $[1, 2]$. La $x_0 = 1.5$, og du trenger ikke å ta med mer enn 3 desimaler i utregningene.
Det er nok å ta med bare to steg, altså å beregne x_1 og x_2 .

$$\text{Hint: } x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Oppg. 12.0.4: Et program er gitt som:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return np.sin(x)
6
7 a = 0
8 b = np.pi
9 N = 10
10 x = np.linspace(a, b, num=N+1)
11 deltax = (b-a)/N
12
13 sum = f(a) + f(b)
14 for i in range(1,N):
15     sum = sum + 2*f(x[i])
16 sum = sum*deltax/2

```

- a) Hvilket problem prøver programmet å løse?
Hvilken metode bruker programmet?
- b) Skriv kort om detaljene i programmet.
Hvordan kan man øke presisjonen for å bestemme sum i programmet?
- c) Regn ut for hånd

$$\int_1^5 \frac{1}{x^3} dx$$

ved hjelp av trapesmetoden ved å dele integrasjonsintervallet i 4 like store deler ($N = 4$).

$$\text{Hint: } \int_a^b f(x) dx \approx \frac{f(a) + 2 \sum_{i=1}^{n-1} f(a + i\Delta x) + f(b)}{2} \cdot \Delta x$$

Oppg. 12.0.5: En funksjon er definert på $\mathbb{R}^2$ ved at $f(x, y) = x^2 + y^2 - 6x + 4y$.

- a) Bestem gradientvektoren i punktet $P = (1, 1)$.
Regn ut hvor raskt f endrer seg i punktet P i retningen $\mathbf{v} = -3\mathbf{i} + 4\mathbf{j}$.

Et program er gitt som:

```
1 import numpy as np
2 def F(x,y):
3     return x**2 + y**2 - 6*x + 4*y
4 def gradF(x,y):
5     return HVA1, HVA2
6
7 x0 = 1
8 y0 = 1
9 n = 100
10 tol = 10**-3
11 gamma = .5;
12 x = x0
13 y = y0
14 t=0
15 cost = [F(x,y)]
16 for i in range(n):
17     gradFx, gradFy = gradF(x, y)
18     x = x - gamma * gradFx
19     y = y - gamma * gradFy
20     cost.append(F(x,y))
21     t+= 1
22     if np.abs(cost[-1] - cost[-2]) < tol:
23         break
24
25 print(x,y)
26 print(t)
```

b) Hvilket problem prøver programmet å løse?

Hvilken metode er brukt til å løse problemet?

Hva skal stå for *HVA1* og *HVA2* i programmet? Du trenger ikke å beskrive detaljene i programmet.

c) Hvor i koden er en betingelse som definerer hva som er en "god nok" løsning? Kan du endre denne betingelsen slik at programmet stopper når gradienten (eller normen til gradienten) er liten, $\|\nabla F\| < tol$, der *tol* er en liten verdi? Hint: bruk *np.linalg.norm* kommandoen.

d) Hva forventer vi å få med *print(x,y)* når man kjører programmet?

Kan du si hva variabelen *t* prøver å telle for uten å si hva *print(t)* blir?

Oppg. 12.0.6: Et program er gitt som:

```
1 # DEL 1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 def f(x,y):
6     return -0.5*y
7
8 xmin = 0
9 xmax = 10
10 N = 4
11 x = np.linspace(xmin, xmax, num = N+1)
12 deltax = (xmax - xmin)/N
13 y = np.zeros(N+1)
14 y[0] = 2
15
```

```

16 print('\tx[n]\ty[n]\t dy/dx \ty[n+1]')
17
18 for i in range(N):
19     y[i+1] = y[i] + f(x[i],y[i])*deltax
20
21 # DEL 2
22
23     print('n=',i, '%.4f\t%.4f\t%.4f\t%.4f'%(x[i],y[i],f(x[i],y[i]),y[i+1]))
24
25 plt.plot(x,y,'--',label='Numerisk løsning')
26 plt.xlabel('x')
27 plt.ylabel('y')
28
29 def yeksakt(x):
30     return np.exp(-0.5*x)
31
32 xpen = np.linspace(xmin,xmaks,100)
33 plt.plot(xpen,yeksakt(xpen),label='ekte løsning')
34 plt.legend()
35
36 print('Den numeriske løsningen gir at y(,xmaks,) er',y[N])
37 print('Den eksakte løsningen er at y(,xmaks,) er',yeksakt(x[N]))

```

- Hvilket problem prøver programmet å løse?
- Hvilken metode er brukt til å løse problemet?
Skriv kort om detaljene i programmet. Her kan du forklare den delen av programmet merket DEL 1 linje for linje (du trenger ikke å beskrive/forklare DEL 2).
- Ved å kjøre programmet får man:

```

1  x[n]  y[n]  dy/dx  y[n+1]
2  n= 0 0.0000 2.0000 -1.0000 0.7500
3  n= 1 1.2500 0.7500 -0.3750 0.2812
4  n= 2 2.5000 0.2812 -0.1406 0.1055
5  n= 3 3.7500 0.1055 -0.0527 0.0396
6  n= 4 5.0000 0.0396 -0.0198 0.0148
7  n= 5 6.2500 0.0148 -0.0074 0.0056
8  n= 6 7.5000 0.0056 -0.0028 0.0021
9  n= 7 8.7500 0.0021 -0.0010 0.0008
10 Den numeriske løsningen gir at y( xx ) er 0.0007821321487426758
11 Den eksakte løsningen er at y( xx ) er xxxxxxxx

```

- Hva skal stå for xx og xxxxxxxx? (Her kan du bruke programmet og utskriften fra programmet)
 - Regn ut numerisk ved hjelp av Eulers metode $y(2.5)$, og sjekk svaret du får med utskriften fra programmet (utregning skal vises).
- Hva kan endres i programmet for å øke presisjon i programmet?

Oppg. 12.0.7: Gitt ligningen: $1 - x + \cos(x) = 0$

- Forklar hvorfor ligningen har kun én løsning i intervallet $[1, 2]$.
- Bruk Newtons metode til å bestemme nullpunktet til funksjonen $f(x) = 1 - x + \cos(x)$, som ligger i intervallet $[1, 2]$. La $x_0 = 1.5$, og du trenger ikke å ta med mer enn 3 desimaler i

utregningene.

Det er nok å bare ta med to steg, altså å beregne x_1 og x_2 .

$$\text{Hint: } x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Oppg. 12.0.8: Et program er gitt som:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return np.sin(x)
6
7 a = 0
8 b = np.pi
9 N = 10
10 x = np.linspace(a, b, num=N+1)
11 deltax = (b-a)/N
12
13 sum = f(a) + f(b)
14 for i in range(1,N):
15     sum = sum + 2*f(x[i])
16 sum = sum*deltax/2
```

- a) Hvilket problem prøver programmet å løse?
Hvilken metode bruker programmet?
- b) Skriv kort om detaljene i programmet.
Hvordan kan man øke presisjonen for å bestemme *sum* i programmet?
- c) Regn ut for hånd

$$\int_1^5 \frac{1}{x^3} dx$$

ved hjelp av trapesmetoden ved å dele integrasjonsintervallet i 4 like store deler ($N = 4$).

$$\text{Hint: } \int_a^b f(x) dx \approx \frac{f(a) + 2 \sum_{i=1}^{n-1} f(a + i\Delta x) + f(b)}{2} \cdot \Delta x$$

Fasit

Fasit 0.4.1:

a)

```
1 print('Denne setningen skrives ut!')
```

b) P i print skal være liten bokstav. Mangler apostrof i enden til setningen.

Fasit 0.4.2:

a) 5+7

b) 12

c) SyntaxError. Legg merke til at det mangler en apostrof etter 7.

d) Print skal være print.

e)

```
1 5+7
```

Fasit 0.4.3:

a)

```
1 > 3*5+4/5
2 15.8
```

b)

```
1 > 2**3+3**2+1.25/4
2 17.25
```

c) Resultatet til $18\%4$ blir 2 (restleddet i divisjonen 18 på 4).

Fasit 0.4.4:

a) Skriver og kjører i console:

```
1 > 5*8/2
2 20.0
```

b) Skriver og kjører i konsollvinduet:

```
1 > 7*10
2 70.0
```

c) Skriver og kjører i konsollvinduet:

```
1 >6*(8+10)/2
2 54.0
```

Fasit 0.6.1:

a)

```
1 m=7
2 n=1
3 m=m+3 # eller m+=3
4 n=n*5 # eller n*=5
5 p= m*n
6 print('Produktet blir da',p)
```

Svaret blir da 30.

b)

```
1 a=5
2 b='TALL'
3 a=a*2
4 b=b* 3
5
6 print(a)
7 print(b)
```

Utskriften er da:

```
1 10
2 TALLTALLTALL
```

Fasit 0.6.2:

```
1 l = 5
2 b = 4
3 h = 3
4 volum=l*b*h
5 overflate = 2*l*b + 2*b*h + 2*l*h
6 print(f"Volume er {volum}.")
7 print(f"Overflaten er {overflate}.")
```

Ved å kjøre programmet blir svaret:

```
1 Volume er 60.
2 Overflaten er 94.
```

Fasit 0.6.3:

```
1 h= 3.5
2 g = 9.81
3 v0 = 20
4 t = 5
5
6 y= h - 0.5*g*t**2+v0*t
7
8 print('y er lik',y)
```

Fasit 0.7.1:

```
1 a = float(input("Skriv inn a (et reelt tall) her: "))
2 b= 3
3 c= 4
4 d = (1/3)*a*b+c**2
5
6 print('d er lik',d)
```

Fasit 0.7.2:

```
1 r = float(input("Skriv inn radiusen til en sylinder (et reelt positivt  
tall) her: "))  
2 h= float(input("Skriv inn radiusen til en sylinder (et reelt positivt  
tall) her: "))  
3 pi=3.14  
4 V = pi*r**2*h  
5  
6 print('Volumet til sylindere er lik',V)
```

Fasit 0.8.1:

```
1 import numpy as np  
2  
3 x=0.5  
4 y = np.sin(np.pi*x)  
5  
6 print('y er lik',y)
```

Fasit 0.8.2:

a) Programkode:

```
1 import numpy as np  
2 svar1=np.cos(2)+np.tan(1.5)  
3 print(svar1)
```

b) Programkode:

```
1 import numpy as np  
2 svar2=np.exp(4)+np.log10(2)+np.log(0.1)  
3 print(svar2)
```

Fasit 0.8.3:

```
1 import pylab  
2  
3 x=0.5  
4 y = sin(pi*x)  
5  
6 print('y er lik',y)
```

Fasit 0.10.1:

Hvis man kjører programmet, blir man spurt Skriv et tall, og når man skriver hvis tallet er negativt, blir den erstattet med det tallet med positiv fortegn. Til slutt skriver programmet ut Absluttverdien til a er x, der x er et positivt tall uansett hva input a var. Resultatet blir blir Absluttverdien til -7 er 7.

Fasit 0.10.2:

a) Programkode:

```
1 tall = int(input("Skriv inn et tall: "))  
2 if tall > 0:  
3     print(f"Tallet {tall} er positivt.")  
4 elif tall < 0:
```

```

5     print(f"Tallet {tall} er negativt.")
6 else:
7     print(f"Tallet {tall} er null.")

```

b) Programkode:

```

1 tall = int(input("Skriv inn et positivt tall: "))
2
3 if tall == 0 or tall < 0:
4     print(f"{tall} er ikke positivt, skriv et positivt tall.")
5 elif tall % 2 == 0:
6     print(f"Tallet {tall} er jevntall.")
7 else:
8     print(f"Tallet {tall} er oddetall.")

```

c) Programkode:

```

1 Tall1 = float(input("Skriv inn et tall: "))
2 Tall2 = float(input("Skriv inn et tall: "))
3
4 if Tall1*Tall2 < 0:
5     print(f"Tallene {Tall1} og {Tall2} har motsatt fortegn.")
6 elif Tall1*Tall2 > 0:
7     print(f"Tallene {Tall1} og {Tall2} har samme fortegn.")
8 else:
9     print(f"Et av tallene {Tall1} eller {Tall2} er null.")

```

Fasit 0.10.3:

Programkoden spør hva summen $49 + 35$, og når brukeren svarer, programkoden svarer om svaret ikke er rett (*Regn igjen!*) eller om det er rett (*Du klarte det!*).

Fasit 0.10.4:

```

1 innlest = input("Hva er 52+19?")
2 tall = int(innlest)
3
4 if tall != 71:
5     while tall != 71:
6         innlest = input("Feil. Regn igjen!")
7         tall = int(innlest)
8     else:
9         print("Du klarte det!")

```

Fasit 0.10.5:

```

1 tall = int(input("Skriv inn et oddetall: "))
2
3 if tall % 2 != 0:
4     print("Tallet er ikke et oddetall!")
5 else:
6     sum = 0
7     i = 1
8     while(i <= tall):
9         sum = sum + i
10        i = i + 2
11
12 print(sum)

```

```
1 Skriv inn et oddetall: 5
2 9
```

Fasit 0.10.6:

```
1 x0= 1
2 b = 1024
3 x = 0
4 x1 = x0
5
6 while x1 != b:
7     x += 1
8     x1*= 4
9
10 print(f"Svaret er {x}.")
```

```
1 Svaret er 5
```

Fasit 0.10.7:

a) Resultatet blir:

```
1 A
2 B
3 B
4 B
5 A
6 B
7 B
8 B
```

b) Resultatet blir:

```
1 51 52 53 61 62 63
```

c) Resultatet blir:

```
1 51 52 53
2 61 62 63
```

Fasit 0.10.8:

a) Resultatet blir:

```
1 0 1 2 3 4 5 6 7 8 9
```

b)

```
1 1 2 3 4 5 6 7 8 9
```

c)

```
1 0 2 4 6 8
```

d)

```
1 10 9 8 7 6 5 4 3 2
```

Fasit 0.10.9:

```

1 svar = ""
2
3 while svar != "nei":
4     svar = input("Hei! Vil du fortsette programmet? ")
5
6 print("Du tastet nei, programmet avsluttes")

```

Fasit 0.10.10:

```

1 a1 = 3
2 k = 2
3 n=5
4 Sum=0
5 for i in range(n):
6     an=a1
7     an = a1 * k
8     Sum+=an
9
10 print(f'Summen til {n} ledd i en geometrisk rekke med a1 {a1} og
    kvotient {k} er {Sum} ')

```

Fasit 0.10.11:

Programkoden lister opp for hvert tall fra og med til og med 9, tallet fra (fra og med) 1 til (til og med) 4.

Fasit 0.10.12:

```

1 a1 = 5
2 k = 3
3 n=7
4 Sum=0
5 for i in range(n):
6     an=a1
7     an = a1+k
8     Sum+=an
9
10 print(f'Summen til {n} ledd i en geometrisk rekke med a1 {a1} og
    kvotient {k} er {Sum} ')

```

Fasit 0.10.13:

- a) Programkoden lister opp for hvert tall fra og med til 1 og med 2, tallet fra (fra og med) 2 til (til og med) 6.

```

1 1 2
2 1 4
3 1 6
4 2 2
5 2 4
6 2 6

```

- b) `range(5)` sender inn 0, 1, 2, 3 og 4 og resultatet blir for `i in range(5)`:

```

1 02438

```

c) `while i < 4` sender inn 1, 2, 3 og 4 og resultatet blir

```
1 1
2 2
3 3
```

`while`-delen erstattes med `for i in range(1,4):`

d)

```
1 1
2 2
3 3
4 4
5 i er ikke lenger mindre enn 5
```

e)

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
```

f)

```
1 o
2 o o
3 o o o
4 * * * *
5 * * *
6 * *
7 *
```

g)

```
1 n=4;
2 for i in range(n):
3     for j in range(i):
4         print ('* ', end="")
5         print(',')
6
7 for i in range(n,0,-1):
8     for j in range(i):
9         print('o ', end="")
10        print(',')
```

Fasit 0.10.14:

a) Programkode:

```
1 n = 4
2 k = 4
3 for i in range(n+1,1,-1):
4     for j in range(1,k-i+3,1):
5         print(j,end=' ')
6     print()
```

b) Programkode:

```
1 n = 4
2 k = 4
3 for i in range(1,n+1,1):
4     for j in range(k-i+1,0,-1):
```

```

5     print(j, end= ' ')
6     print()

```

c) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+1,1,-1):
4     for j in range(k-i+2,0,-1):
5         print(j, end= ' ')
6     print()

```

d) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+2,0,-1):
4     for j in range(1,k-i+3,1):
5         print(j, end= ' ')
6     print()

```

Fasit 0.11.1:

1 man får Skalarproduktet av a og b er 1.0 . Legg merke til $-1 \cdot 0 + 3 \cdot 1 + 0 \cdot 5 = 1$

Fasit 0.11.2:

a)

```

1 import numpy as np
2
3 def A(r):
4     return np.pi*r**2
5
6 print(A(5))

```

b)

```

1 import numpy as np
2
3 r= float(input('Skriv inn radiusen til en sirkel: '))
4
5 def A(r):
6     return np.pi*r**2
7
8 print(A(r))

```

Fasit 0.11.3:

- Programkode 1 (0.25 ligger i konvergensintervall til s_1 . Programkode 1 og 2 (1.75 ligger i konvergensintervall for begge) Programkode 2 (2.25 ligger i konvergensintervall til s_2)
- Ved å synke estimeringsnøyaktighet e i programkodene.

Fasit 0.11.4:

Programkode:

```

1 import numpy as np
2
3 def A(a,b,c):
4     s=1/2*(a+b+c)
5     A=np.sqrt(s*(s-a)*(s-b)*(s-c))
6     return A
7
8 a = 3
9 b = 4
10 c = 5
11
12 Areal=A(a,b,c)
13 print(f'Arealet til en trekant med sidelengdene {a}, {b} og {c} er {
14     Areal}')
15 print(Areal)

```

Fasit 0.11.5:

Programkode:

```

1
2 def KE(m,v):
3     """
4     Denne funksjonen bestemmer kinetisk enegi for en partikkel gitt:
5     Input:
6         masse (kg)
7         hastighet(m/s)
8
9     Output:
10        Partikkelens kinetisk energi (J)
11        """
12        KE = 0.5 * m * v** 2
13        return KE
14
15 m= 0.0001
16 v=8
17 E=KE(m, v)
18 print(f'Kinetisk energi for en partikkel med massen {m} og hastighet {v
19     } m/s er {E} J')

```

Fasit 0.11.6:

```

1 5
2 15

```

Fasit 0.12.1:

- a)
- `plt.plot(x,f,'g--')` tegner grafen til funksjonen og tar inn x -verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'` skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med grønnfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
 - Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
 - `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y -aksen.

- `plt.grid()` legger på et gitter bak grafen.
- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best'`): plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsoll-vinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

b)

Fasit 0.12.2:

a)

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 xstart = -2
5 xslutt = 2
6 steglengde = 0.01
7
8 def f(x):
9     return x**3 + 6*x - 3
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     x += steglengde
16     y = f(x)
17     print(f'{x:.3f} {y:.3f}')
18
19 x = np.arange(xstart, xslutt, steglengde)
20 y = f(x)
21
22 plt.plot(x,y)

```

b) Programkode:

```

1 xstart = float(input('Skriv inn nedre grense for x: '))
2 xslutt = float(input('Skriv inn øvre grense for x: '))
3 steglengde = float(input('Skriv inn steglengden: '))
4
5 def f(x):
6     return x**3 + 6*x - 3
7
8 x = xstart
9 y = f(x)
10
11 while x <= xslutt:
12     x += steglengde
13     y = f(x)
14     print(f'{x:.3f} {y:.3f}')

```

Fasit 0.12.3:

Programkode:

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Her skal vi definere en funksjon, og deretter plotte den
6
7 # Deler inn x-aksen i 100 deler (101 punkt)
8 x = np.linspace(-2*np.pi,2*np.pi, num=101)
9
10 def f(x):
11     f= np.sin(2*x-np.pi)
12     return f
13
14 plt.plot(x, f(x), color='green')
15
16 def g(x):
17     g= np.cos(2*x+np.pi)
18     return g
19
20 plt.plot(x, g(x), color='blue')
21
22 def h(x):
23     h= np.cos(2*x+np.pi/2)
24     return h
25
26 plt.plot(x, h(x), color='red')
27
28 plt.show()

```

f og h representerer samme funksjon.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 c= 60
5 a=120
6 T= 365
7 k =2
8 d= 182.5
9
10
11 # Deler inn x-aksen i 1000 deler (1001 punkt)
12 x = np.linspace(0,T, num=1001)
13
14 def f(x):
15     f= c+a*np.sin((2*np.pi*k/T)*x+ d)
16     return f
17
18 plt.plot(x, f(x), color='blue')
19 plt.grid()
20 plt.xlabel('x ')
21 plt.ylabel('y')
22
23 def g(x):
24     g= c+a*np.cos((2*np.pi*k/T)*x+d)
25     return g

```

```

26 plt.plot(x, g(x), color='red')
27
28
29 plt.legend(loc='best')
30 plt.legend(['f(x)', 'g(x)'])
31
32 plt.show()

```

Fasit 0.13.1:

- a) [1, 2, 3, 4, 5, 2, 4, 6, 8, 10]
- b) A=[1,2,3,4,5]
- c) Programkode:

```

1 liste = [1, 2, 3, 4, 5, 6]
2
3 produkt = 1
4
5 for t in liste:
6     produkt *= t
7
8 print(produkt)

```

Fasit 0.13.2:

```

1 def f(x):
2     return -.2*x**2+1
3
4 xstart = -1
5 xslutt = 2
6 xsteg = .25
7
8 xverdier = []
9 yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)

```

Fasit 0.13.3:

```

1 def f(x):
2     return -x**3+x+2
3
4 xstart = -2
5 xslutt = 3
6 xsteg = .25

```

```

7
8 xverdier = []
9 yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)

```

Fasit 0.18.1:

a) Programkode:

```

1 x = float(input("Skriv inn x her: "))
2 y = float(input("Skriv inn y her: "))
3 print('Summen til x og y er', x+y)

```

b) Programkode:

```

1 x = float(input("Skriv inn x her: "))
2 y = float(input("Skriv inn y her: "))
3 print('Differeansen til x og y er', x-y)

```

c) Programkode:

```

1 x = float(input("Skriv inn x her: "))
2 y = float(input("Skriv inn y her: "))
3 print('Multiplikasjonen til x og y er', x*y)

```

d) Programkode:

```

1 x = float(input("Skriv inn x her: "))
2 y = float(input("Skriv inn y her: "))
3 print('Divisjonen til y og x er', y/x)

```

e) Programkode:

```

1 x = float(input("Skriv inn x her: "))
2 y = float(input("Skriv inn y her: "))
3 print('Kvadratet til summen til x og y er', (x+y)**2)

```

f) Programkode:

```

1 import numpy as np
2 x = float(input("Skriv inn x her: "))
3 y = float(input("Skriv inn y her: "))
4 print('Kvadratroten til summen til x og y er', np.sqrt(x+y))

```

Fasit 0.18.2:

- a) Legg merke til at *range*(1,11) er hele tall fra og med 1 til og med 10, og *range*(0,10) er hele tall fra og med 0 til og med 9.

```
1 for n in range(1,11):
2     tregangen =3*n
3     print(tregangen)
```

```
1 n=0
2 while n<11:
3     n =n+3
4     print(n)
5     n+= 1
```

Svaret blir:

```
1 3
2 7
3 11
```

$n + 3$ gir feil svar, men man kan bestemme 3-gangen på flere måter:

```
1 n=1
2 while n<11:
3     tregangen =3*n
4     print(tregangen)
5     n+= 1
```

Eller

```
1 n = 0
2 for i in range(0,10):
3     n =n+3
4     print(n)
```

Eller

```
1 for n in range(1,11):
2     tregangen =3*n
3     print(tregangen)
```

```
1 3
2 6
3 9
4 12
5 15
6 18
7 21
8 24
9 27
10 30
```

- b) Se svar fra forrige oppgavedel

Fasit 0.18.3:

- a) Programkode:

```

1 for n in range(1,101):
2     firegangen =4*n
3     print(firegangen)

```

b) Programkode:

```

1 for n in range(0,10):
2     oddetall=(2*n+1)
3     print(oddetall)

```

```

1 1
2 3
3 5
4 7
5 9
6 11
7 13
8 15
9 17
10 19

```

```

1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
10 100

```

c) Programkode:

```

1 for n in range(1,11):
2     kvadrat =n**2
3     print(kvadrat)

```

```

1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
10 100

```

Fasit 0.18.4:

```

1 for n in range(0,100):
2     kvadratoddetall =(2*n+1)**2
3     print(kvadratoddetall)

```

Fasit 0.18.5:

a) $a_n = 3n, n = 1, 2, 3, \dots$

```
1 n = 0
2 for i in range(0,10):
3     n =n+3
4     print(n)
```

Eller

```
1 for n in range(1,11):
2     tregangen =3*n
3     print(tregangen)
```

```
1 3
2 6
3 9
4 12
5 15
6 18
7 21
8 24
9 27
10 30
```

b) Programkode:

```
1 n = 0
2 for i in range(0,10):
3     n =n+10
4     print(n)
```

Eller

```
1 for n in range(1,11):
2     tigangen =10*n
3     print(tigangen)
```

```
1 10
2 20
3 30
4 40
5 50
6 60
7 70
8 80
9 90
10 100
```

c) Programkode:

```
1 n = 1
2 for i in range(0,10):
3     n =n+10
4     print(n)
```

```
1 11
2 21
3 31
4 41
5 51
6 61
7 71
8 81
9 91
10 101
```

d) Programkode:

```
1 n = 0
2 for i in range(0,10):
3     n = n+11
4     print(n)
```

Eller

```
1 for n in range(1,11):
2     elvegangen = 11*n
3     print(elvegangen)
```

```
1 11
2 22
3 33
4 44
5 55
6 66
7 77
8 88
9 99
10 110
```

Fasit 0.18.6:

```
1 n = 1
2 for i in range(1,10):
3     n = 0.5*n + 1
4     print(n)
```

```
1 1.5
2 1.75
3 1.875
4 1.9375
5 1.96875
6 1.984375
7 1.9921875
8 1.99609375
9 1.998046875
```

Fasit 0.18.7:

```

1 # Definerer n og Sum ved start
2 n = 1
3 Sum = 0
4
5 # Vil utføre summeringen saa lenge summen er mindre eller lik 15
6 while Sum <= 15:
7     Sum += 2/n
8     n += 1
9
10 print('Summen passerer 15 etter', n, 'ledd. Da er summen', Sum)

```

```

1 Summen passerer 15 etter 1016 ledd. Da er summen 15.000704172652771

```

Fasit 0.18.8:

```

1 M = 858          # g
2 V = 40           # cm**3
3
4 M_kg = 858*1E-3   # kg
5 V_m = 40*1E-6     # m**3
6
7 mass_density = M_kg/V_m
8 print("The mass density of the material is: %.2f"%mass_density)

```

Med å kjøre programmet, får vi:

```

1 The mass density of the material is: 21450.00

```

Fasit 0.18.9:

Viktig at summeringen blir utført og at de passer på å ikke gjøre heltalsdivisjon.

Her skal det ikke være nødvendig å lagre massene og avstandene i lister. Det hadde vært flott om at det ble tydelig sagt i fra om at lister **ikke er nødvendige** dersom du ser noen gjøre dette.

```

1 from math import sqrt
2 G = 6.674*1E-11          #m^3*kg^(-1)*s^(-2)
3 M = 3.
4 N = 10
5
6 F = 0
7 for i in xrange(N):
8     m_i = i/6. + 2
9     r_i = sqrt((i/4.）**2 + 10)
10    F += G*(m_i*M)/(r_i)**2
11
12 print "The total attractive force on the object with mass M = %g is %g
    N"%(M,F)

```

Den totale kraften F skal bli:

```
>> The total attractive force on the object with mass M = 3 is 4.65492e-10 N
```

Fasit 1.0.1:

a) Forslag til algoritme:

1) Regn ut avstanden fra sentrum til P .

2) Sammenlikne med r . Hvis avstanden er lik r , ligger punktet på sirkelen. Hvis avstanden er større enn r , ligger punktet utenfor sirkelen. Ellers ligger den inni

b)

```
1 import numpy as np
2
3 # Sirkel med senter i (a, b) og radius r
4 # Punkt P(x, y)
5 a = 3
6 b = -4
7 r = 2
8 x = 6
9 y = 8
10
11 # Finner avstanden mellom S og P:
12 d = np.sqrt((x - a)**2 + (y - b)**2)
13
14 if d > r:
15     print('Punktet ligger utenfor sirkelen.')
16 elif d == r:
17     print('Punktet ligger på sirkelen.')
18 else:
19     print('Punktet ligger innenfor sirkelen.')
```

Med å kjøre programmet får vi:

```
1 MPIpython')
2 Punktet ligger utenfor sirkelen.
```

Fasit 1.0.2:

Igjen er det viktig de passer på å få riktige enheter. En riktig så skjult 'felle', er at et år må konverteres til sekunder.

```
1 from math import pi
2
3 AU = 1.58*1E-5          # 10(-5) lysår
4 ly = 9.5*1E15           # 10(15) m
5 G = 6.674*1E-11         # 10(-11) m3*kg(-1)*s(-2)
6
7 AU_km = AU*ly           # 10(10) km
8
9 yr = 365*24*60*60       # 10(7) s
10 M_sun = (4*pi**2*(AU_km)**3)/(G*(yr)**2) # 10(27)
11
12 print("The calculated solar mass is: %g kg"%M_sun)
```

Med å kjøre programmet får vi:

```
1 The calculated solar mass is: 2.01142e+30 kg
```

Fasit 1.0.3:

```
1 from math import sqrt, pi, cos
2
3 F0 = 1.
4 x = 1.
5 n = 4.
```

```

6 m = 3.
7 v0 = 2.
8
9 v = sqrt(v0**2 + 2./m*(cos(x/n) - 1.) )
10
11 print("The velocity of the atom at position x = %g is v = %g"%(x,v))

```

Med å kjære programmet, får vi:

```

1 The velocity of the atom at position x = 1 is v = 1.99481

```

Fasit 1.0.4:

```

1 # __Exercise a__
2 with open("solar_system_data.dat", "r") as infile:
3     infile.readline(); infile.readline() # Skipping two first lines.
4     distance = {}
5     mass = {}
6     radius = {}
7     for line in infile:
8         words = line.split()
9         distance[words[0]] = float(words[1])
10        mass[words[0]] = float(words[2])
11        radius[words[0]] = float(words[3])
12
13 # __Exercise b__
14 from math import pi
15 density = {}
16 for planet in mass: # Looping over a dictionary gives its keys.
17     planet_volume = 4.0/3*pi*(radius[planet]*1000)**3
18     density[planet] = mass[planet]/planet_volume
19
20 # __Exercise c__
21 with open("solar_system_data2.dat", "w") as outfile:
22     outfile.write("%8s %16s %16s %16s %16s\n" % ("Object", "Distances[
23 km]", "Mass[kg]", "Radius[km]", "Densities[kg/m^3]"))
24     for planet in mass:
25         outfile.write("%8s %16.2e %16.2e %16d %16d\n" % (planet,
26 distance[planet], mass[planet], radius[planet], density[planet]))
27
28 # __Exercise d__
29 solar_system = {}
30 for planet in mass:
31     solar_system[planet] = {}
32     solar_system[planet]["mass"] = mass[planet]
33     solar_system[planet]["distance"] = distance[planet]
34     solar_system[planet]["radius"] = radius[planet]
35     solar_system[planet]["density"] = density[planet]
36
37 number_of_earth = solar_system["Pluto"]["distance"]/solar_system["
38 Earth"]["radius"]
39 print("Number of earths from Sun to Pluto = %.2f" % number_of_earth)

```

Fasit 1.0.5:

Python-program:

```

1 from math import cos,pi,sin
2
3 def height(t,v0,th):
4     g = 9.81
5     return -.5*g*t**2 + v0*t*sin(th)
6
7 def find_points(h0,h1,y):
8     points = 0
9     mid_point = (h0 + h1)/2.
10    if (h0 <= y <= mid_point):
11        points = 1
12    elif (mid_point < y <= h1 ):
13        points = 2
14    return points
15
16 h0 = 3
17 h1 = 3.5
18 b = 3.5
19 g = 9.81
20 th = pi/4
21
22 for v0 in [15.,16.,19.,22.]:
23     T = b/(v0*cos(th))
24     y = height(T,v0,th)
25
26     print("Number of points using v0 = %g: %g"%(v0,find_points(h0,h1,y)
    ))

```

Fasit 1.1.1:

Python-program:

```

1 print("Angi temperaturen i fahrenheit: ")
2 Fahrenheit = float(input())
3 Celsius = (Fahrenheit - 32) * 5.0/9.0
4 print('%0.1f Fahrenheit er %0.1f Celsius' %(Fahrenheit,Celsius))

```

Ved å kjøre programmet får man:

```

1 Angi temperaturen i fahrenheit:
2 100
3 100.0 Fahrenheit er 37.8 Celsius

```

Fasit 1.1.2:

```

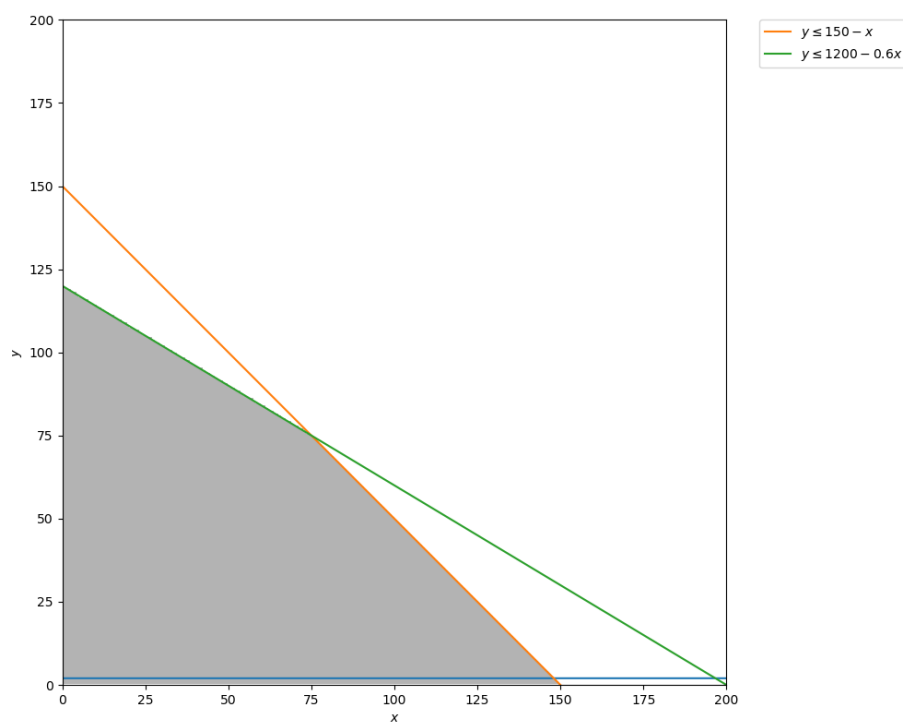
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 d = np.linspace(-2,150,200)
5 x,y = np.meshgrid(d,d)
6 plt.imshow( ((y>=0) & (y<=150-x) & (10*y<=1200-6*x)).astype(int) ,
7             extent=(x.min(),x.max(),y.min(),y.max()),origin="lower"
8             , cmap="Greys", alpha = 0.3);
9
10 x = np.linspace(0, 200, 2000)
11 # y >= 2

```

```

11 y1 = (x*0) + 0
12 y2 = (150-x)
13 y3= (1200-6*x)/10.0
14
15 plt.plot(x, 2*np.ones_like(y1))
16 plt.plot(x, y2, label=r'$y \leq 150-x$')
17 plt.plot(x, y3, label=r'$y \leq 1200-0.6x$')
18 plt.xlim(0,200)
19 plt.ylim(0,200)
20 plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
21 plt.xlabel(r'$x$')
22 plt.ylabel(r'$y$')

```



Fasit 3.1.1:

Python-program:

```

1 from math import radians, sin, cos, acos
2
3 print("Sett opp koordinatene til to punkt:")
4 b1 = radians(float(input("Breddegrad punkt 1: ")))
5 l1 = radians(float(input("lengdegrad punkt 1: ")))
6 b2 = radians(float(input("Breddegrad punkt 2: ")))
7 l2 = radians(float(input("lengdegrad punkt 2: ")))
8
9 Avstand = 6371.01 * acos(sin(b1)*sin(b2) + cos(b1)*cos(b2)*cos(l1 - l2)
10 )
11 print("Avstanden er %.2fkm." % Avstand)

```

Ved å kjøre programmet får man:

```

1 Breddegrad punkt 1: 22.5
2 lengdegrad punkt 1: 68.5

```

```

3 Breddegrad punkt 2: 24.5
4 lengdegrad punkt 2: 70.5
5 Avstanden er 301.73km.

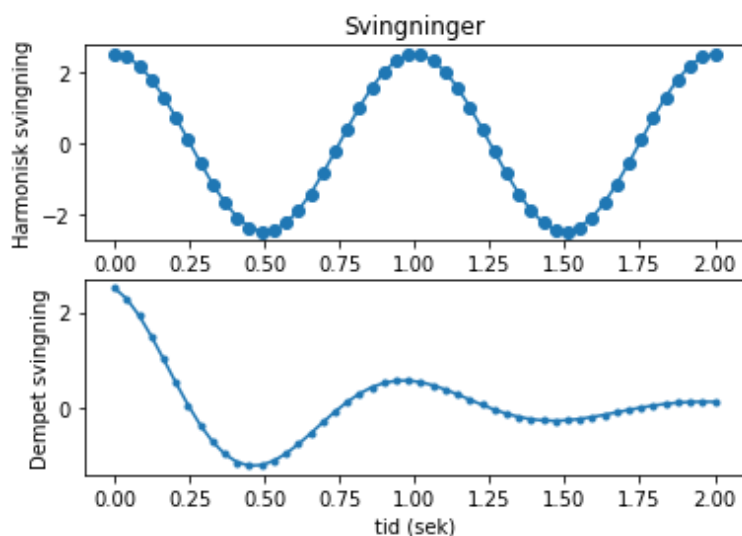
```

Fasit 3.1.2:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 t = np.linspace(0.0, 2.0)
5 alpha = 1.5
6 A=2.5
7
8 y1 = A*np.cos(2 * np.pi * t)
9 y2 = A*np.exp(-alpha*t)*np.cos(2 * np.pi * t)
10
11 plt.subplot(2, 1, 1)
12 plt.plot(t, y1, 'o-')
13 plt.title('Svingninger')
14 plt.ylabel('Harmonisk svingning')
15
16 plt.subplot(2, 1, 2)
17 plt.plot(t, y2, '.-')
18 plt.xlabel('tid (sek)')
19 plt.ylabel('Dempet svingning')
20
21 plt.show()

```



Fasit 4.1.1:

a) $\frac{T}{1,05} + \frac{T}{(1,05)^2} + \frac{T}{(1,05)^3} + \dots + \frac{T}{(1,05)^{16}} = 100,000.$

b) $T = \frac{100,000 \cdot 0,05}{1 - (1,05)^{-16}} \simeq 7192$

c) $\frac{T}{a} + \frac{T}{a^2} + \frac{T}{a^3} + \dots + \frac{T}{a^{n \cdot t}} = L.$

$$d) T = \frac{p \cdot 0,01 \cdot L}{(1 - (1 + p/100)^{-n \cdot t})}$$

e) Program:

```

1 # Termin.py beregner terminbeløpet gitt L (lånebeløp), P (rentefot)
  og antall termin.
2 L= float(input('Hva er låneverdien? '))
3 P = float(input('Hva er rentefot? '))
4 AntallTermin = float(input('Hvor mange termin skal ha betales pr. år?
  '))
5 Laanetid = float(input('Hva er nedbetalingstid?'))
6 def LaaneKalk(L, P, AntallTermin, Laanetid):
7     Teller = (P / 100) * L
8     Nevner = (1 + P / 100) ** (-1 * (Laanetid* AntallTermin))
9     return str(round((Teller / (1 - Nevner)),2))
10
11 print('Terminbeløpet er: ' + LaaneKalk(L, P, AntallTermin, Laanetid))

```

Ved å kjøre programmet, får vi:

```

1 Hva er låneverdien? 1000000
2 Hva er rentefot? 2
3 Hvor mange termin skal ha betales pr. år?4
4 Hva er nedbetalingstid?15
5
6 Terminbeløpet er: 7191.99

```

Fasit 4.1.2:

```

1 import math
2
3 a = int(input("Skriv første ledd i den geometriske rekken: "))
4 n = int(input("Skriv antall ledd i rekken: "))
5 r = int(input("Skriv kvotienten: "))
6
7 total = (a * (1 - math.pow(r, n))) / (1- r)
8 an = a * (math.pow(r, n - 1))
9
10 print("an (n-te ledd i den geometriske rekken) = " , an)
11 print("\nSummen til den geometriske rekken = " , total)

```

```

1 Skriv første ledd i den geometriske rekken: 2
2 Skriv kvotienten: 3
3 Skriv antall ledd i rekken: 9
4
5 an (n-te ledd i den geometriske rekken) = 13122.0
6 Summen til den geometriske rekken = 19682.0

```

Fasit 5.0.1:

a) Vekstfaktor er 0.82 ($1 - 0.18 = 0.82$) og verdi = 350000.

```

1 0 350000
2 1 287000.0
3 2 235340.0

```

```

4 3 192978.8
5 4 158242.61599999998
6 Etter 4 år er verdien mer enn halvert og prisen er
  158242.61599999998

```

- b) For 10. klasse elever kan dette forklares ved: $350000(0.82)^3 \simeq 192979$, $350000(0.82)^4 \simeq 158243$ og $350000(0.82)^{3.5} \simeq 174749,7$

For videregående skole elever kan dette forklares ved: $350000(0.82)^x = 175000$ som gir $0.82^x = 0.5$ og dermed $x = \log 0.5 / \log 0.82 \simeq 3.49$ år.

Fasit 5.1.1:

a) $\frac{T}{1,05} + \frac{T}{(1,05)^2} + \frac{T}{(1,05)^3} + \dots + \frac{T}{(1,05)^{16}} = 100\,000.$

b) $T = \frac{100\,000 \cdot 0,05}{1 - (1,05)^{-16}} \simeq 7192$

c) $\frac{T}{a} + \frac{T}{a^2} + \frac{T}{a^3} + \dots + \frac{T}{a^{n-t}} = L.$

d) $T = \frac{p \cdot 0,01 \cdot L}{(1 - (1 + p/100)^{-n \cdot t})}$

- e) Program:

```

1 # Termin.py beregner terminbeløpet gitt L (lånebeløp), P (rentefot)
  og antall termin.
2 L= float(input('Hva er låneverdien? '))
3 P = float(input('Hva er rentefot? '))
4 AntallTermin = float(input('Hvor mange termin skal ha betales pr. år?
  '))
5 Laanetid = float(input('Hva er nedbetalingstid?'))
6 def LaaneKalk(L, P, AntallTermin, Laanetid):
7     Teller = (P / 100) * L
8     Nevner = (1 + P / 100) ** (-1 * (Laanetid * AntallTermin))
9     return str(round((Teller / (1 - Nevner)),2))
10
11 print('Terminbeløpet er: ' + LaaneKalk(L, P, AntallTermin, Laanetid))

```

Ved å kjøre programmet, får vi:

```

1 Hva er låneverdien? 100000
2 Hva er rentefot? 2
3 Hvor mange termin skal ha betales pr. år?4
4 Hva er nedbetalingstid?15
5
6 Terminbeløpet er: 7191.99

```

Fasit 6.1.1:

Programmet beregner den deriverte til $f(x) = x^3 - 3x^2$ numerisk i intervallet $[-2, 5]$ og plotter grafen til funksjonen og dens deriverte med å dele intervallet i 500 deler (501 punkter).

Fasit 6.1.2:

Ved hjelp av de 3 differanse-metodene får vi:

$$f'(0) \approx \frac{f(x_0 + h) - f(x_0)}{h} = \frac{f(0.25) - f(0)}{0.25} = \frac{0.448 - 0.385}{0.25} \simeq 0.252, \quad \text{forlengsdifferanse,}$$

$$f'(x_0) \approx \frac{f(x_0) - f(x_0 - h)}{h} = \frac{f(0) - f(-0.25)}{0.25} = \frac{0.385 - 0.273}{0.25} \simeq 0.448, \quad \text{baklengsdifferanse,}$$

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0 - h)}{2h} = \frac{f(0.25) - f(-0.25)}{0.50} = \frac{0.448 - 0.273}{0.50} \simeq 0.350, \quad \text{sentraldifferanse.}$$

Sentraldifferanse-metoden gir middelveiden av de to andre metodene.

Fasit 6.1.3:

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 N = 100
6 x = np.linspace(-3,3, num = N)
7 f = np.exp(x) - 4*x #Svart
8 plt.plot(x,f,'k')
9
10 def Funksjon_f(v):
11     return np.exp(v) - 4*v
12
13 x0=2
14 for i in range(1,5):
15     h = 10**(-i)
16     print('Forlengsdifferanse gir:', (Funksjon_f(x0+h)-Funksjon_f(x0))/
17     h)
18     print('Baklengsdifferanse gir:', (Funksjon_f(x0)-Funksjon_f(x0-h))/
19     h)
20     print('Sentraldifferanse gir:', (Funksjon_f(x0+h)-Funksjon_f(x0-h))
21     /(2*h))
22
23 y1 = np.zeros(N)
24 y2 = np.zeros(N)
25 y3 = np.zeros(N)
26 h = 10**(-6)
27
28 for i in range(N):
29     y1[i] = (Funksjon_f(x[i]+h)-Funksjon_f(x[i]))/h
30     y2[i] = (Funksjon_f(x[i])-Funksjon_f(x[i]-h))/h
31     y3[i] = (Funksjon_f(x[i]+h)-Funksjon_f(x[i]-h))/(2*h)
32
33 plt.plot(x,y1,'x','b')
34 plt.plot(x,y2,'.','r')
35 plt.plot(x,y3,'o')

```

Fasit 6.1.4:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 1000

```

```

5 x = np.linspace(-np.pi,np.pi, num = N)
6 f = np.sin(x) #Svart
7 plt.plot(x,f,'k')
8
9 def Funksjon_f(v):
10     return v**2
11
12 x0=3
13
14 for i in range(1,5):
15     a = 10**(-i)
16     print((Funksjon_f(x0+a)-Funksjon_f(x0))/a)
17
18 y = np.zeros(N)
19 a = 10**(-4)
20
21 for i in range(N):
22     y[i] = (Funksjon_f(x[i]+a)-Funksjon_f(x[i]))/a
23
24 plt.plot(x,y)

```

Fasit 6.1.5:

```

a)
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n = 6 # Antall delintervaller
5
6 x = np.linspace(1,4,num = n+1)
7 deltax = x[1] - x[0] # storrelsen paa delintervallene
8 f = 4*x**3 - 2*x # funksjonen vi vil integrere
9
10 Sum = 0
11
12 # Trapesmetoden
13 for i in range(len(x)-1):
14     Sum = Sum + ((f[i]+f[i+1])/2)*deltax
15
16 # print resultatet
17 print(Sum)
18
19 # Definer funksjonen paa ny for glattere graf
20 u = np.linspace(1,4,num = 10000)
21 f1 = 4*u**3 - 2*u
22
23 # Plot funksjonen og tilnaermingen
24 plt.plot(x,f)
25 plt.plot(u,f1, label='f(x)')
26 plt.grid(ls='--')
27 plt.legend()

```

HVA skal være *deltax*.

b) Med å endre N til en større verdi.

Fasit 6.2.1:

Programkoden bruker Newtonsmetoden for å bestemme løsningen til $x^3 - 3x^2 - 1 = 0$ med startverdi 0.1.

Fasit 6.2.2:

Programkoden bruker Newtonsmetoden for å bestemme løsningen til $x^3 - \cos x = 0$ med startverdi 1.

Fasit 6.2.3:

Skjæringssetningen: $f(x) = x + \sin(x) + 1$, som er kontinuerlig. Vi velger endepunktene gitt av intervallet:

$$f\left(\frac{-\pi}{2}\right) = \frac{-\pi}{2} + \sin\left(\frac{-\pi}{2}\right) + 1 = \frac{-\pi}{2} - 1 + 1 = \frac{-\pi}{2} < 0$$

$$f(0) = 0 + \sin(0) + 1 = 1 > 0$$

Da sier skjæringssetningen at f har minst et nullpunkt i $[-\pi/2, 0]$.

Den deriverte: $f'(x) = 1 + \cos(x) \geq 0$ i $[-\pi/2, 0]$. Siden cosinus alltid er mellom -1 og 1, er alltid $f'(x) \geq 0$. I tillegg er cosinus mellom 0 og 1 i intervallet, så $f' > 0$. Det betyr at f er en-entydig, og den kan ikke ha mer enn ett nullpunkt i intervallet.

Newton's metode: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = -1 - \frac{-1 + \sin(-1) + 1}{1 + \cos(-1)} = -0,4536975$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = -0,4536975 - \frac{-0,4536975 + \sin(-0,4536975) + 1}{1 + \cos(-0,4536975)} = -0,510580$$

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} = -0,510580 - \frac{-0,510580 + \sin(-0,510580) + 1}{1 + \cos(-0,510580)} = -0,51097$$

Løsningen er da $x = -0,51$ (her var det ikke presisert hvor mange desimaler man måtte ta med. Det er da nok med 4-5 desimaler).

Fasit 6.2.4:

```

1
2 def newton(f,Df,x0,epsilon,maks_iter):
3
4     '''Løsningsmetode for f(x)=0 ved Newtons metode.
5
6     Parameters
7     -----
8     f : Vi ønsker å finne nullpunktet til f(x)=0.
9     Df : Deriverte til f(x).
10    x0 : startverdi
11    -----
12    xn : number
13    Newtons metode:
14    x = xn - f(xn)/Df(xn)
15        Fortsett til abs(f(xn)) < epsilon og sett opp xn.
16        Hvis Df(xn) == 0, Ingen svar.
17        Hvis antall iterasjoner er mer enn maks_iter, ingen svar.
18    '''
19
20    xn = x0
21    for n in range(0,maks_iter):
22        fxn = f(xn)

```

```

23     print(n,xn,abs(fxn))
24     if abs(fxn) < epsilon:
25         print('Løsningen etter',n,'iterasjoner.')
26         return xn
27     Dfxn = Df(xn)
28     if Dfxn == 0:
29         print('Den deriverte blir 0. Ingen løsning funnet.')
30         return None
31     xn = xn - fxn/Dfxn
32
33     print('Overstiger maks antall iterasjoner. Ingen løsning funnet.')
34     return None
35
36 p = lambda x: x**3 - x**2 -3
37 Dp = lambda x: 3*x**2 - 2*x
38 approx = newton(p,Dp,1.5,1e-5,10)
39 print(approx)

```

Med å kjøre programmet, får vi:

```

1 0 1.5 1.875
2 1 2.0 1.0
3 2 1.875 0.076171875
4 3 1.8637931034482758 0.0005794671778258653
5 4 1.8637065329604223 3.4409209703767374e-08
6 Løsningen etter 4 iterasjoner.
7 1.8637065329604223

```

Fasit 6.2.5:

```

1 import math
2
3 def f(x):
4     return 2*x*math.cos(x)-1
5
6 a = 0
7 b = 1
8 e = .001
9
10 m = (a + b)/2
11
12 n= 0
13
14 while b - a > e:
15     if f(a) * f(m) < 0:
16         b = m
17     elif f(m) == 0:
18         break
19     else:
20         a = m
21     m = (a + b)/2
22     n = n+1
23
24 print(m)
25 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet får vi:

```
1 0.60986328125
2 Antall iterasjoner: 10
```

Fasit 6.2.6:

a)

```
1 def f(x):
2     return x**3-5
3
4 a = 1
5 b = 2
6 d = b - a #Finn avstanden mellom a og b
7 m = (a+b)/2
8 e=0.001
9
10 while abs(d) > e:
11
12     if f(m)*f(a) < 0: #Hvis fm og fa har motsatt fortegn:
13         b = m         #Flytter b til midten
14     elif f(m)*f(b)<0:
15         a = m         #Flytter a til midten
16     elif f(m)==0:
17         break
18     else:
19         print("Feil: f(a) og f(b) skal ligge på hver si side av x-
20               aksen!")
21         break
22 # Sett opp et nytt midtpunkt og reknar ut f-verdi:
23     m = (a+b)/2
24     d=b-a
25 print(f'Nullpunktet med presisjonen {e} er: ', m)
```

Her kan vi legge inn antall iterasjoner:

```
1 def f(x):
2     return x**3-5
3
4 a = 1
5 b = 2
6 d = b - a #Finn avstanden mellom a og b
7 m = (a+b)/2
8 n=0
9 e=0.001
10
11 while abs(d) > e:
12
13     if f(m)*f(a) < 0: #Hvis fm og fa har motsatt fortegn:
14         b = m         #Flytter b til midten
15     elif f(m)*f(b)<0:
16         a = m         #Flytter a til midten
17     elif f(m)==0:
18         break
19     else:
20         print("Feil: f(a) og f(b) skal ligge på hver si side av x-
21               aksen!")
22         break
```

```

22 # Sett opp et nytt midtpunkt og reknar ut f-verdi:
23     m = (a+b)/2
24     d=b-a
25     n=n+1
26 print(f'Nullpunktet med presisjonen {e} er: ', m)
27 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet får vi:

```

1 Nullpunktet med presisjonen 0.001 er: 1.71044921875
2 Antall iterasjoner: 10

```

```

1 def f(x):
2     return x**3 - 5
3 a = 1
4 b = 2
5 n = 0 # Antall iterasjoner
6
7 Nøyaktighet = .001
8 deltax = .01
9 while deltax > Nøyaktighet:
10     if f(a) * f(a + deltax) < 0:
11         deltax = deltax / 10
12     else:
13         a = a + deltax
14         n = n+1
15
16 print(a)
17 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet får vi:

```

1 1.7000000000000006
2 Antall iterasjoner: 70

```

b) Det er tydelig at den første metoden er numerisk kostbar.

Fasit 6.2.7:

Skjæringssetningen: $f(x) = x + \sin(x) + 1$, som er kontinuerlig. Vi velger endepunktene gitt av intervallet:

$$f\left(\frac{-\pi}{2}\right) = \frac{-\pi}{2} + \sin\left(\frac{-\pi}{2}\right) + 1 = \frac{-\pi}{2} - 1 + 1 = \frac{-\pi}{2} < 0$$

$$f(0) = 0 + \sin(0) + 1 = 1 > 0$$

Da sier skjæringssetningen at f har minst et nullpunkt i $[-\pi/2, 0]$.

Den derivate: $f'(x) = 1 + \cos(x) \geq 0$ i $[-\pi/2, 0]$. Siden cosinus alltid er mellom -1 og 1, er alltid $f'(x) \geq 0$. I tillegg er cosinus mellom 0 og 1 i intervallet, så $f' > 0$. Det betyr at f er en-entydig, og den kan ikke ha mer enn ett nullpunkt i intervallet.

Newtons metode: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = -1 - \frac{-1 + \sin(-1) + 1}{1 + \cos(-1)} = -0,4536975$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = -0,4536975 - \frac{-0,4536975 + \sin(-0,4536975) + 1}{1 + \cos(-0,4536975)} = -0,510580$$

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} = -0,510580 - \frac{-0,510580 + \sin(-0,510580) + 1}{1 + \cos(-0,510580)} = -0,51097$$

Løsningen er da $x = -0,51$ (her var det ikke presisert hvor mange desimaler man måtte ta med. Det er da nok med 4-5 desimaler).

Fasit 6.2.8:

Løsning:

a) Ifølge Pytagoras læresetning er: $BC = \sqrt{BD^2 + DC^2} = \sqrt{40^2 + 30^2} = 50\text{m}$.

b) Deler opp problemet i to deler, regner farten på land og i vann:

$$v_1 = \frac{100}{20} = 5 \text{ m/s, og } v_2 = \frac{50}{60} = \frac{5}{6} \text{ m/s.}$$

Forholdet blir da: $\frac{v_1}{v_2} = 5/(5/6) = 6$, det vil si badevakten løper 6 ganger raskere på land enn badevakten svømmer i vann.

c) Problemløsning:

1) Dele opp problemet:

i) Bestem tiden fra A til C: $t(x)$ = Tiden for å løpe AB + tiden for å svømme BC :

$$t = \frac{140 - x}{5} + \frac{\sqrt{x^2 + 30^2}}{1}.$$

2) Analyserer problemet, og gjenkjenner problemstillingen (minste tid): ii) For å bestemme minste tid, kan vi derivere funksjonen:

$$t'(x) = -\frac{1}{5} + \frac{2x}{2\sqrt{x^2 + 900}}$$

3: Løs problemet

$$t'(x) = -\frac{1}{5} + \frac{2x}{2\sqrt{x^2 + 9}} = -\frac{1}{5} + \frac{x}{\sqrt{x^2 + 9}} = 0.$$

$$5x = \sqrt{x^2 + 900} \text{ som gir } 24x^2 = 900 \text{ og dermed } x = \frac{30}{2\sqrt{6}} = \frac{15}{\sqrt{6}} \simeq 6,1 \simeq 6 \text{ m.}$$

Dette er den minste verdien fordi:

$$t''(x) = \frac{1 - \frac{2x}{2\sqrt{x^2 + 900}}}{(\sqrt{x^2 + 900})^2} = \frac{\sqrt{x^2 + 900} - x}{x^2 + 900}$$

og $t''(\frac{15}{\sqrt{6}}) > 0$ (krummer nedover) og

dermed er $x = \frac{15}{\sqrt{6}}$ er minimumspunkt og gir minste tiden.

4: Vurdering av løsningen

$x \simeq 6$ ser fornuftig ut, da $t(6) = \frac{140 - 6}{5} + \sqrt{6^2 + 900} = \frac{\frac{15}{\sqrt{6}}}{\sqrt{225/6 + 900}} \simeq 57,4$. Legg merke til at vi brukte her tilnærmet verdi for $x = 6$ og tiden blir litt mer enn det vi har regnet ut. Uansett blir ca. 1 min, og vi ser at denne tiden ser fornuftig liten ut.

Med programmering leses AD og CD fra skjermen, og det settes opp: $t = \frac{AD - x}{5} + \frac{\sqrt{x^2 + CD^2}}{1}$.

Deretter setter opp deriverte til t med hensyn til x

```

1 x = symbols('x')
2 t = (AD-x)/5 + np.sqrt(x**2+CD**2)/1
3 dt = diff(t, x)

```

og finner nullpunktet til $t'(x)$.

Man kunne sette opp $t''(x)$ og sjekke om $t''(\text{nullpunkt}) > 0$.

Fasit 6.2.9:

a)

b)

```

1 import math
2
3 def taylorrex(x, n):
4     f_estimert = 0
5     for i in range(n):
6         f_estimert += x**i/math.factorial(i)
7     return f_estimert
8
9 x=float(input("Tyalor-rekken skal beregnes for x=:"))
10 n=int(input("Skriv inn antall ledd i Taylor-rekken n:"))
11 print()
12
13 for i in range(1,n+1):
14     f_estimert = taylorrex(x,i)
15     f_eksakt = math.exp(x)
16     e_error = abs(f_estimert - f_eksakt)
17
18 print(f'{i} ledd: Taylor-rekken estimerer e^{x}= {f_estimert}')
19 print('Eksaktverdi e^{x}= {f_eksakt}')
20 print('Estimeringsfeil = {e_error}')

```

Ved å kjøre programmet og skrive inn $x = 2$, $n = 7$ i første utregning og $n = 10$ i det neste utregning får vi:

```

1 7 ledd: Taylor-rekken estimerer e^2.0= 7.355555555555555,
2 Eksaktverdi e^2.0= 7.38905609893065
3 Estimeringsfeil = 0.033500543375095226
4
5 10 ledd: Taylor-rekken estimerer e^2.0= 7.3887125220458545
6 Eksaktverdi e^2.0= 7.38905609893065
7 Estimeringsfeil = 0.0003435768847959153

```

```

1 import math
2
3 def taylorrex2(x, n):
4     f_estimert = 0
5     for i in range(n):
6         f_estimert += (x**2)**i/math.factorial(i)
7     return f_estimert
8
9 x=float(input("Tyalor-rekken skal beregnes for x=:"))
10 n=int(input("Skriv inn antall ledd i Taylorrekken n:"))
11 print()
12 for i in range(1,n+1):

```

```

13     f_estimert = taylor2(x,i)
14     f_eksakt = math.exp(x**2)
15     e_error = abs(f_estimert - f_eksakt)
16
17 print(f'{i} ledd: Taylor rekken estimerer e^{x**2}= {f_estimert}')
18 print('Eksaktverdi e^{x**2}= {f_eksakt}')
19 print('Estimeringsfeil = {e_error}')

```

Ved å kjøre programmet og skrive inn $x = 2$, $n = 10$ i første utregning og $n = 15$ i det neste utregning får vi:

```

1
2 10 ledd: Taylor-rekken estimerer e^4.0= 54.15414462081129
3 Eksaktverdi e^4.0= 54.598150033144236
4 Estimeringsfeil = 0.4440054123329489
5
6 15 ledd: Taylor-rekken estimerer e^4.0= 54.59706179769672
7 Eksaktverdi e^{x}= {f_eksakt}
8 Estimeringsfeil = {e_error}

```

c) Fasit 6.3.1:

L'Hopitalsregel: $\lim_{x \rightarrow \infty} \frac{\ln x}{x} \stackrel{\left[\frac{\infty}{\infty} \right]}{=} \lim_{x \rightarrow \infty} \frac{1/x}{1} = 0.$

Python-program:

```

1 from sympy import *
2 x = symbols('x')
3 y = ln(x)/x;
4
5 print("Utrykket er: {}".format(y))
6
7 x0 = input("Skriv inn x0: ")
8 L = limit(y, x, x0)
9 print("Grenseverdien når x går mot",x0, "er lik: {}".format(L))

```

Ved å kjøre programmet, får vi:

```

1 Skriv inn x0: oo
2 oo
3 Grenseverdien når x går mot oo er lik: 0

```

I Python-språket er oo uendelig

Fasit 6.3.2:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def ff(x):
5     """ Funksjon """
6     f=x*np.exp(2*x)
7     return f
8
9 def EksaktD(x):
10    """Den deriverte"""
11    f = (2*x+1)*np.exp(2*x)

```

```

12     return f
13
14 def Dforlengs(x,h):
15     f = (ff(x+h)-ff(x))/h
16     return f
17
18 def Dbaklengs(x,h):
19     f = (ff(x)-ff(x-h))/h
20     return f
21
22 def DMidt(x,h):
23     f = (ff(x+h)-ff(x-h))/(2*h)
24     return f
25
26 def error_Dforlengs(y):
27     f= abs((Dforlengs(1,y)-EksaktD(1))/EksaktD(1))
28     return f
29
30 def error_Dbaklengs(y):
31     f= abs((Dbaklengs(1,y)-EksaktD(1))/EksaktD(1))
32     return f
33
34 def error_DMidt(y):
35     f= abs((DMidt(1,y)-EksaktD(1))/EksaktD(1))
36     return f
37
38 print("Eksakt f'(1)= ",EksaktD(1))
39 print("Forlengs:", Dforlengs(1,0.01), error_Dforlengs(0.01))
40 print("Baklengs:", Dbaklengs(1,0.01),error_Dbaklengs(0.01))
41 print("Midtpunkt:", DMidt(1,0.01),error_DMidt(0.01))
42
43 x=np.arange(0.001,1,0.001);
44
45 plt.xlabel('steglengde h')
46 plt.ylabel('Relativt estimeringsavvik')
47 plt.title('Midtpunktsmetoden konvergerer raskere')
48 plt.loglog(x,error_Dforlengs(x),label='Forlengsmetoden')
49 plt.loglog(x,error_Dbaklengs(x),label='Baklengsmetoden')
50 plt.loglog(x,error_DMidt(x),label='Midtpunktsmetoden')
51 plt.show()

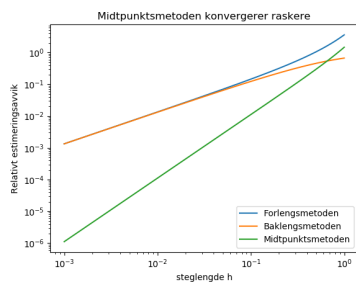
```

Ved å kjøre programmet får vi:

```

1 Eksakt f'(1)= 22.16716829679195
2 Forlengs: 22.465208406789117 0.01344511423411261
3 Baklengs: 21.874054362124884 0.013222885789588433
4 Midtpunkt: 22.169631384457 0.00011111422226200907

```



Fasit 6.3.3:

a) `from numpy import exp`

```
# Definerer funksjonen f(x) som vi skal finne nullpunktet til
# Funksjonen må være kontinuerlig på intervallet [a,b]

def f(x):
    return x - 2*exp(-x)+ 1

print("Vi skal bestemme nullpunktet til f(x)= x - 2*exp(-x)+ 1 nær 1")

# e: Presisjonen for funksjonsverdi |x-m|<e

N = 100 # Maks antal iterasjoner

a = float(input("Skriv a: ")) # Nedre grense for intervallet
b = float(input("Skriv b: ")) # Øvre grense for intervallet
e = float(input("Skriv presisjonen (e): "))

if f(a)*f(b) < 0:
    n = 0 # antall iterasjoner
    m = (a+b)/2 # Finn m, midtpunktet mellom a og b.
    while abs(f(m)) > e and n < N:
        n +=1

        if f(m) == 0: # Vi har funnet et nullpunkt og kan avslutte
            break # break tilrettelegger at vi går ut av løkken
        elif f(m)*f(b) < 0: # Det er minst et nullpkt mellom m og b,
            a = m # så vi flytter a til midten
        else: # Det er minst et nullpkt mellom a og m,
            b = m # så vi flytter b til midten

        m = (a+b)/2 # Regner ut m igjen

    print(f'Nullpunktet med presisjonen {e} er: ', m)
    print("Antall iterasjoner:",n)
else:
    print("Feil: f(a) og f(b) skal ligge på hver si side av x-aksen!")
```

Ved å kjøre programmet får vi:

```

1 Skriv a: 0
2 Skriv b: 2
3 Skriv presisjonen (e): 0.001
4
5 Nullpunktet med presisjonen 0.001 er: 0.375
6 Antall iterasjoner: 3

```

```

1 def f(x):
2     return 2*x*math.cos(x)-1
3
4 a = 0
5 b = 1
6 n = 0
7 deltax = .001
8
9 while f(a) * f(a + deltax) > 0:
10     a = a + deltax
11     n = n+1
12
13 print(a)
14 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet får vi:

```

1 0.61000000000000004
2 Antall iterasjoner: 610

```

b)

```

1 import random
2 import math
3
4 def f(x):
5     return 2*x*math.cos(x)-1
6
7 a = 1
8 b = 2
9 N = 1000
10 n= 0
11 Beste_y = f(a)
12 Beste_x = a
13 for i in range(N):
14     x = random.uniform(a, b)
15     y = f(x)
16     if abs(y) < abs(Beste_y):
17         Beste_y = y
18         Beste_x = x
19     n= n+1
20 print(Beste_x)
21 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet får vi forskjellige verdier, noe som er forventet, da vi bruker tilfeldig søk blant 1000 tall i intervallet.

```

1 1.097861902832455
2 Antall iterasjoner: 9
3 1.0986945954448402
4 Antall iterasjoner: 4

```

Fasit 6.3.4:

```
1 '''Derivert ved hjelp av definisjonen'''
2 def derivert(f,x):
3     h = 1e-5
4     return (f(x+h) - f(x))/h
5
6 '''Newtonsmetode'''
7 def newtonsmetode(f, x):
8     epsilon = 1e-4
9     while True:
10         x1 = x - f(x)/derivert(f,x)
11         print(x1)
12         e = abs(x1 - x)
13         if e < epsilon:
14             break
15         x = x1
16
17     return x
18
19 '''Test and results'''
20
21 def f(x):
22     return x**3-x-1
23
24 r = newtonsmetode(f, 1)
25 print('Løsningen er', r)
```

Ved å kjøre programmet, får vi:

```
1 1.4999925000863985
2 1.3478254914987762
3 1.3252005800527265
4 1.3247181786548083
5 1.324717957246855
6 Løsningen er 1.3247181786548083
```

Fasit 7.0.1:

a) $\int_0^5 \frac{1}{x^2+1} dx = [\arctan x]_0^5 = \arctan 5 \simeq 1.37340077$

b) Python-program:

```
1 import scipy.integrate as integrate
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 a = 0
6 b = 5
7 N = int(input("Skriv inn antall patisjoner N: "))
8
9 h = (b - a) / N
10 x = np.linspace(a,b,N+1)
11 f = 1/(1+x**2)
12 ff = lambda x: 1/(1+x**2)
13
14 eksakt,error=integrate.quad(ff, a, b)
```

```

15 print(f'Eksaktverdien til integrasjonen er: {eksakt}')
16 I_riemannL = h * sum(f[:N])
17 err_riemannL =abs(eksakt - I_riemannL)
18
19 print(f'Venstre Riemann-integrasjon med n={N} gir: {I_riemannL}')
20 print(f'Estimeringsavvik ved venstre Riemann-integrasjon med n={N}
    er: {err_riemannL}')
21
22
23 x = np.linspace(a,b,N)
24 y = ff(x)
25
26 plt.figure(figsize=(15,5))
27
28 plt.plot(x,y,'b')
29 x_right = x[1:] # Venstre endepunkt
30 y_right = y[1:]
31 plt.plot(x_right,y_right,'b.',markersize=10)
32 plt.bar(x_right,y_right,width=-(b-a)/N,alpha=0.2,align='edge',
    edgcolor='b')
33 plt.title('Nedre Riemann Sum, N = {}'.format(N))
34
35 plt.show()

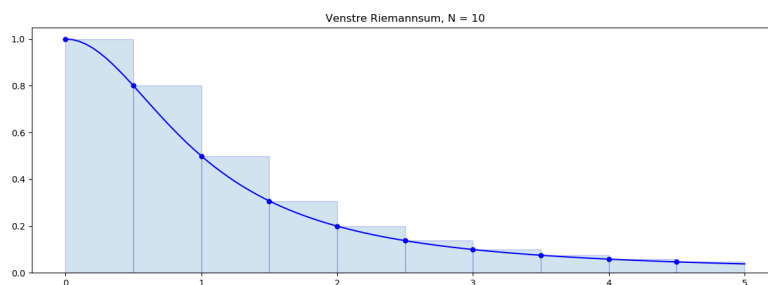
```

Løsning:

```

1
2 Skriv inn antall patisjoner N: 10
3 Eksaktverdien til integrasjonen er: 1.3734007669450166
4 Venstre Riemann-integrasjon med n=10 gir: 1.6134886966147248
5 Estimeringserror 0.4807692307692308

```



Fasit 7.0.2:

a) $\int_0^5 \frac{1}{x^2 + 1} dx = [\arctan x]_0^5 = \arctan 5 \simeq 1.37340077$

b) Python program:

```

1 import scipy.integrate as integrate
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 a = 0
6 b = 5
7 N = int(input("Skriv inn antall patisjoner N: "))
8 n=N

```

```

9  h = (b - a) / N
10 x = np.linspace(a,b,N+1)
11 f = 1/(1+x**2)
12 ff = lambda x: 1/(1+x**2)
13
14 eksakt,error=integrate.quad(ff, a, b)
15 print(f'Eksaktverdien til integrasjonen er: {eksakt}')
16 I_riemannL = h * sum(f[:N])
17
18
19 I_riemannR = h * sum(f[1:])
20
21 print(f'Venstre Riemann-integrasjon med n={N} gir: {I_riemannL}')
22
23
24 I_mid = h*sum(ff((x[:N]+ x[1:])/2))
25 err_mid = abs(eksakt- I_mid)
26
27
28 x = np.linspace(a,b,N+1)
29 y = ff(x)
30
31 X = np.linspace(a,b,n*N+1)
32 Y = ff(X)
33
34 plt.figure(figsize=(15,5))
35
36 plt.subplot(1,3,1)
37 plt.plot(X,Y,'b')
38 x_left = x[:-1] # Left endpoints
39 y_left = y[:-1]
40 plt.plot(x_left,y_left,'b.',markersize=10)
41 plt.bar(x_left,y_left,width=(b-a)/N,alpha=0.2,align='edge',edgecolor
         ='b')
42 plt.title('Venstre Riemannsum, N = {}'.format(N))
43
44 plt.subplot(1,3,2)
45 plt.plot(X,Y,'b')
46 x_mid = (x[:-1] + x[1:])/2 # Midpoints
47 y_mid = ff(x_mid)
48 plt.plot(x_mid,y_mid,'b.',markersize=10)
49 plt.bar(x_mid,y_mid,width=(b-a)/N,alpha=0.2,edgecolor='b')
50 plt.title('Midpunkt Riemannsum, N = {}'.format(N))
51
52 plt.subplot(1,3,3)
53 plt.plot(X,Y,'b')
54 x_right = x[1:] # Left endpoints
55 y_right = y[1:]
56 plt.plot(x_right,y_right,'b.',markersize=10)
57 plt.bar(x_right,y_right,width=-(b-a)/N,alpha=0.2,align='edge',
         edgecolor='b')
58 plt.title('Høyre Riemannsum, N = {}'.format(N))
59
60
61 print(f'Numerisk integrasjon ved midtpunktmetoden med n={N} gir: {I_

```

```

        mid}}')
62
63 print(f'Høyre Riemann-integrasjon med n={N} gir: {I_riemannR}')
64
65 err = abs(h*(ff(b)-ff(a)))
66
67 print('Estimeringserror',err)
68
69 plt.show()

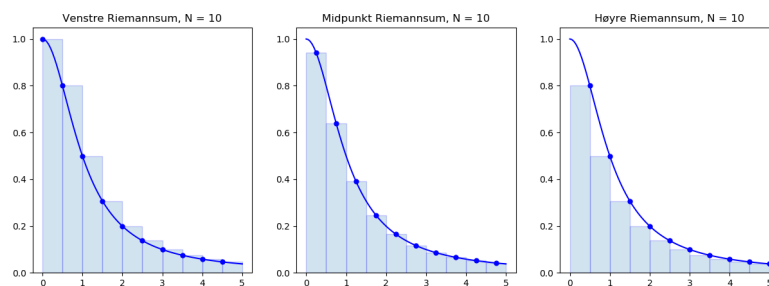
```

Løsning:

```

1 Skriv inn antall patisjoner N: 10
2 Eksaktverdien til integrasjonen er: 1.3734007669450166
3 Venstre Riemann-integrasjon med n=10 gir: 1.6134886966147248
4 Numerisk integrasjon ved midtpunktmetoden med n=10 gir:
  1.373543428316664
5 Høyre Riemann-integrasjon med n=10 gir: 1.1327194658454942
6 Estimeringserror 0.4807692307692308
7
8 skriv inn antall patisjoner N: 100
9 Eksaktverdien til integrasjonen er: 1.3734007669450166
10 Venstre Riemann-integrasjon med n=100 gir: 1.3974361466841378
11 Estimeringserror 0.04807692307692308

```



Fasit 7.0.3:

a)

```

1 from math import *
2 import scipy.integrate as integrate
3 import numpy as np
4
5 a = 0
6 b = 2
7 N = 10
8 h = (b - a) / N
9 x = np.linspace(a, b, N+1)
10 f = 1/(1+x**2)
11 ff = lambda x: 1/(1+x**2)
12 #ff = lambda x: np.sin(x)
13 eksakt,error=integrate.quad(ff, a, b)
14
15 I_riemannL = h * sum(f[:N])
16 err_riemannL =abs(eksakt - I_riemannL)
17
18 I_riemannR = h * sum(f[1::])

```

```

19 err_riemannR = abs(eksakt - I_riemannR)
20
21 print(f'Venstre Riemann-integrasjon med n={N} gir: {I_riemannL}')
22 print(f'Estimeringsavvik ved nedre Riemann-integrasjon med n={N} er:
    {err_riemannL}')
23
24 print(f'Høyre Riemann-integrasjon med n={N} gir: {I_riemannR}')
25 print(f'Estimeringsavvik ved Nedre Riemann-integrasjon med n={N} er:
    {err_riemannR}')
26
27 I_mid = h*sum(ff((x[:N]+ x[1:])/2))
28
29 err_mid = abs(eksakt- I_mid)
30
31 print(f'Eksaktverdien er: {eksakt}')
32 print(f'Numerisk integrasjon ved midtpunktmetoden med n={N} gir: {I_
    mid}')
33 print(f'Estimeringsavvik ved midtpunktmetoden med n={N} er: {err_mid
    }')
34
35
36 I_trap = (h/2)*(f[0] + \
37               2 * sum(f[1:N]) + f[N])
38
39 err_trap = abs(eksakt- I_trap)
40
41 print(f'Numerisk integrasjon ved Trapesmetoden med n={N} gir: {I_
    trap}')
42 print(f'Estimeringsavvik ved Trapesmetoden med n={N} er: {err_trap}'
    )
43
44 I_simp = (h/3) * (f[0] + 2*sum(f[2:N-1:2]) \
45                 + 4*sum(f[1:N:2]) + f[N])
46
47 err_simp = abs(eksakt - I_simp)
48
49 print(f'Numerisk integrasjon ved Simpsonsметoden med n={N} gir: {I_
    simp}')
50 print(f'Estimeringsavvik ved Simpsonsметoden med n={N} er: {err_simp}
    ')

```

Ved å kjøre programmet får vi:

```

1 Eksaktverdien er: 1.1071487177940904
2 Numerisk integrasjon ved midtpunktmetoden med n=10 gir:
    1.107414937162371
3 Estimeringsavvik ved midtpunktmetoden med n=10 er:
    0.00026621936828052917
4 Numerisk integrasjon ved Trapesmetoden med n=10 gir:
    1.1066158957367225
5 Estimeringsavvik ved Trapesmetoden med n=10 er:
    0.0005328220573679321
6 Numerisk integrasjon ved Simpsonsметoden med n=10 gir:
    1.107146526974766
7 Estimeringsavvik ved Simpsonsметoden med n=10 er: 2.190819324399129e
    -06

```

Fasit 7.0.4:

a)

```
1 from math import *
2 import scipy.integrate as integrate
3 import numpy as np
4
5 a = 0
6 b = 2
7 N = 10
8 h = (b - a) / N
9 x = np.linspace(a, b, N+1)
10 f = 1/(1+x**2)
11 ff = lambda x: 1/(1+x**2)
12 #ff = lambda x: np.sin(x)
13 eksakt,error=integrate.quad(ff, a, b
14
15 I_riemannL = h * sum(f[:N])
16 err_riemannL =abs(eksakt - I_riemannL)
17
18 I_riemannR = h * sum(f[1::])
19 err_riemannR = abs(eksakt - I_riemannR)
20
21 print(f'Venstre Riemann-integrasjon med n={N} gir: {I_riemannL}')
22 print(f'Estimeringsavvik ved nedre Riemann-integrasjon med n={N} er:
23       {err_riemannL}')
24
25 print(f'Høyre Riemann-integrasjon med n={N} gir: {I_riemannR}')
26 print(f'Estimeringsavvik ved Nedre Riemann-integrasjon med n={N} er:
27       {err_riemannR}')
28
29 I_mid = h*sum(ff((x[:N]+ x[1:])/2))
30
31 err_mid = abs(eksakt- I_mid)
32
33 print(f'Eksaktverdien er: {eksakt}')
34 print(f'Numerisk integrasjon ved midtpunktmetoden med n={N} gir: {I_
35       mid}')
36 print(f'Estimeringsavvik ved midtpunktmetoden med n={N} er: {err_mid
37       }')
38
39 I_trap = (h/2)*(f[0] + \
40               2 * sum(f[1:N]) + f[N])
41
42 err_trap = abs(eksakt- I_trap)
43
44 print(f'Numerisk integrasjon ved Trapesmetoden med n={N} gir: {I_
45       trap}')
46 print(f'Estimeringsavvik ved Trapesmetoden med n={N} er: {err_trap}'
47       )
48
49 I_simp = (h/3) * (f[0] + 2*sum(f[2:N-1:2]) \
50                 + 4*sum(f[1:N:2]) + f[N])
51
52 err_simp = abs(eksakt - I_simp)
```

```

49 print(f'Numerisk integrasjon ved Simpsonsметoden med n={N} gir: {I_
    simp}')
50 print(f'Estimeringsavvik ved Simpsonsметoden med n={N} er: {err_simp}
    ')

```

Ved å kjøre programmet får vi:

```

1 Eksaktverdien er: 1.1071487177940904
2 Numerisk integrasjon ved midtpunktметoden med n=10 gir:
  1.107414937162371
3 Estimeringsavvik ved midtpunktметoden med n=10 er:
  0.00026621936828052917
4 Numerisk integrasjon ved Trapesметoden med n=10 gir:
  1.1066158957367225
5 Estimeringsavvik ved Trapesметoden med n=10 er:
  0.0005328220573679321
6 Numerisk integrasjon ved Simpsonsметoden med n=10 gir:
  1.107146526974766
7 Estimeringsavvik ved Simpsonsметoden med n=10 er: 2.190819324399129e
  -06

```

Fasit 7.0.5:

```

1 a = 0
2 b = 2
3 N = 10
4 h = (b - a) / N
5 x = np.linspace(a, b, N+1)
6 f = x**2*np.exp(x)
7 ff = lambda x: x**2*np.exp(x)
8 #ff = lambda x: np.sin(x)
9 eksakt,error=integrate.quad(ff, a, b)
10
11
12 I_riemannL = h * sum(f[:N])
13
14 I_riemannR = h * sum(f[1:])
15
16 print(f'Venstre Riemann-integrasjon med n={N} gir: {I_riemannL}')
17
18 print(f'Høyre Riemann-integrasjon med n={N} gir: {I_riemannR}')
19
20
21
22 I_trap = (h/2)*(f[0] + \
23               2 * sum(f[1:N]) + f[N])
24
25 print(f'Numerisk integrasjon ved Trapesметoden med n={N} gir: {I_trap}')
26
27
28 I_simp = (h/3) * (f[0] + 2*sum(f[2:N-1:2]) \
29                 + 4*sum(f[1:N:2]) + f[N])
30
31 print(f'Numerisk integrasjon ved Simpsonsметoden med n={N} gir: {I_simp}')

```

Ved å kjøre programmet får vi:

```

1 Venstre Riemann-integrasjon med n=10 gir: 0.9800072469099018
2 Høyre Riemann-integrasjon med n=10 gir: 0.7836703746876488
3 Numerisk integrasjon ved Trapesmetoden med n=10 gir: 0.8818388107987754
4 Numerisk integrasjon ved Simpsonsметoden med n=10 gir:
  0.882074876854494

```

Ved å endre tll $N = 100$ i programmet får vi:

```

1 Venstre Riemann-integrasjon med n=100 gir: 0.8918957924511557
2 Estimeringsavvik ved nedre Riemann-integrasjon med n=100 er:
  0.009814401688734153
3 Høyre Riemann-integrasjon med n=100 gir: 0.8722621052289304
4 Estimeringsavvik ved Nedre Riemann-integrasjon med n=100 er:
  0.009819285533491096
5 Eksaktverdien er: 0.8820813907624215
6 Numerisk integrasjon ved midtpunktметoden med n=100 gir:
  0.8820826116625586
7 Estimeringsavvik ved midtpunktметoden med n=100 er: 1.2209001370999673e
  -06
8 Numerisk integrasjon ved Trapesmetoden med n=100 gir: 0.882078948840043
9 Estimeringsavvik ved Trapesmetoden med n=100 er: 2.4419223785265842e-06
10 Numerisk integrasjon ved Simpsonsметoden med n=100 gir:
  0.8820813901111865
11 Estmeringsavvik ved Simpsonsметoden med n=100 er: 6.512350658738342e-10

```

Fasit 7.0.6:

```

1 import matplotlib.pyplot as plt
2
3 def f(x):
4     return x**3
5 xstart = 0
6 xslutt = 10
7 xsteg = .1
8 N = 10
9 Deltax = (xslutt - xstart)/N
10 xverdier = []
11 yverdier = []
12 x = xstart
13 y = f(x)
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x += xsteg
18     y = f(x)
19 x = xstart
20 y = f(x)
21 for i in range(N):
22     xkoord = [x, x + Deltax, x + Deltax, x, x]
23     ykoord = [0, 0, y, y, 0]
24     plt.fill(xkoord, ykoord)
25     x += Deltax
26     y = f(x)
27 plt.plot(xverdier, yverdier)
28 plt.show()

```

Fasit 7.0.7:

Ved trapesmetoden: 0,22070, med estimeringsavvik: $|e_T| < \frac{12}{12 \cdot 4^2}(1-0)^3 = 0,0625$

Ved Simpsons formel: 0,20052, med estimeringsavvik: $|e_T| < \frac{24}{180 \cdot 4^2}(1-0)^3 \simeq 0,0083$

Fasit 7.0.8:

- a) 0,69455 med estimeringsavvik 0,28105.
- b) 0,67564 med estimeringsavvik 0,000105.
- c) Simpsons regel benytter 3 punkt fra kurven i hvert estimeringssteg mens trapesmetoden bruker 2 punkt. Dermed gir Simpsons regel en mer presis tilnærming.

Fasit 7.0.9:

- a) 20,64455905 med estimeringsavvik 4,19193129.
- b) 17.35362645 med estimeringsavvik 0,90099869.
- c) Simpsons regel benytter 3 punkt fra kurven i hvert estimeringssteg mens trapes metoden bruker 2 punkt. Dermed gir Simpsons metode en mer presis tilnærming. Legg merke til at man får bedre tilnærming ved å bruke 6 delintervaller i Simpsons formel ($n = 6$, 16.6919 som er mer presis)

Fasit 7.1.1:

- a) Metoden starter med å dele inn intervallet $[a, b]$ i en mengde delintervaller (som vi her lar være like store for enkelhets skyld). Hvis vi vil ha n delintervaller, betyr det at vi øker x fra a til b med steg som har størrelse $\Delta x = (b - a)/n$. For hver av disse delintervallene beregnes arealet til trapeser, og den første, og siste sidelengde ($f(a)$ og $f(b)$) brukes kun én gang, mens de andre brukes to ganger.

$$f(x_0) = f(0) = 0$$

$$2f(x_1) = 2f\left(\frac{\pi}{6}\right) = 1$$

$$2f(x_2) = 2f\left(\frac{\pi}{3}\right) = \sqrt{3} \approx 1.732050807568877$$

$$2f(x_3) = 2f\left(\frac{\pi}{2}\right) = 2$$

$$2f(x_4) = 2f\left(\frac{2\pi}{3}\right) = \sqrt{3} \approx 1.732050807568877$$

$$2f(x_5) = 2f\left(\frac{5\pi}{6}\right) = 1$$

$$f(x_6) = f(\pi) = 0$$

b)

Dermed: $\pi/12(0+1+1.732050807568877+2+1.732050807568877+1+0) = 1.954097233313707$

Den eksakte verdien:

$$\int_0^\pi \sin(x) dx = [\cos x]_0^\pi = 2$$

c)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def Func_f(x):
```

```

5     return np.sin(x)
6
7 def Func_g(x):
8     return np.sin(x**2)
9
10 x_start = 0
11 x_slutt = np.pi
12 N = 1000
13
14 x = np.linspace(x_start, x_slutt, num=N+1)
15 f = [Func_f(i) for i in x]
16 g = [Func_g(i) for i in x]
17
18 plt.plot(x,f,'k',x,g,'r')
19
20 int_f = f[0] + f[N]
21 for i in range(1,N):
22     int_f = int_f + 2*f[i]
23
24 int_f = (x_slutt - x_start)*(2*(N))**(-1) *int_f
25 print(int_f)
26
27 int_g = g[0] + g[N]
28 for i in range(1,N):
29     int_g = int_g + 2*g[i]
30
31 int_g = (x_slutt - x_start)*(2*(N))**(-1) *int_g
32 print(int_g)

```

Ved å kjøre programmet får vi:

```

1 1.9999983550656624
2 0.7726470478388956

```

Ved å bruke $N = 6$ får vi:

```

1 1.9540972333137065
2 0.6086224277543034

```

som tilsvarer svaret vi fikk i del b)

Fasit 9.1.1:

```

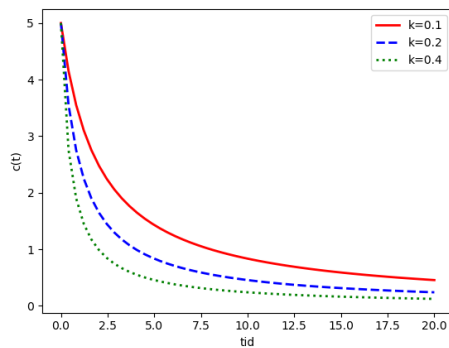
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # funksjonen setter opp dx/dt
6 def model(c,t,k):
7     dcdt = -k * c**2
8     return dcdt
9
10 # startverdi
11 c0 = 5
12 # tidslinjen
13 t = np.linspace(0,20)
14

```

```

15 # Løser ODEs
16 k = 0.1
17 c1 = odeint(model, c0, t, args=(k,))
18 k = 0.2
19 c2 = odeint(model, c0, t, args=(k,))
20 k = 0.4
21 c3 = odeint(model, c0, t, args=(k,))
22
23 # Tegner løsningskurvene
24 plt.plot(t, c1, 'r-', linewidth=2, label='k=0.1')
25 plt.plot(t, c2, 'b--', linewidth=2, label='k=0.2')
26 plt.plot(t, c3, 'g:', linewidth=2, label='k=0.4')
27 plt.xlabel('tid')
28 plt.ylabel('c(t)')
29 plt.legend()
30 plt.show()

```



Fasit 9.1.2:

```

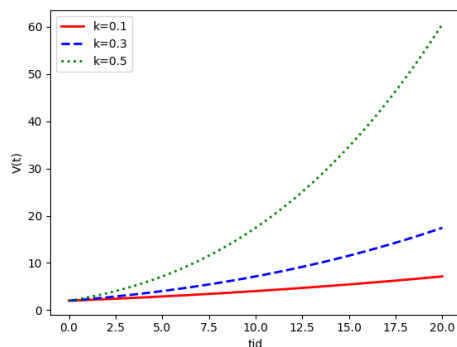
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 # funksjonen setter opp dx/dt
6 def model(V,t,k):
7     dVdt = k * V**(2/3)
8     return dVdt
9
10 # startverdi
11 V0 = 2
12 # tidslinjen
13 t = np.linspace(0,20)
14
15 # Løser ODEs
16 k = 0.1
17 V1 = odeint(model, V0, t, args=(k,))
18 k = 0.3
19 V2 = odeint(model, V0, t, args=(k,))
20 k = 0.5
21 V3 = odeint(model, V0, t, args=(k,))
22
23 # Tegner løsningskurvene
24 plt.plot(t, V1, 'r-', linewidth=2, label='k=0.1')

```

```

25 plt.plot(t,V2,'b--',linewidth=2,label='k=0.3')
26 plt.plot(t,V3,'g:',linewidth=2,label='k=0.5')
27 plt.xlabel('tid')
28 plt.ylabel('V(t)')
29 plt.legend()
30 plt.show()

```



Fasit 9.1.3:

- a) $x_0 = 0$ og $y(0,2) = y_1 = y_0 + hy'_0 = 2 + 0,2(0) = 2$
 $x_1 = 0,2$, $y_2 = y(0,4) = y_1 + hy'_1 = 2 + 0,2(3 \cdot 0,2e^{-2}) = 2,01640$
 $x_2 = 0,4$, $y_3 = y(0,6) = y_2 + hy'_2 = 2,016 + 0,2(3 \cdot 0,2e^{-2,016}) = 2,0482$
- b) $\int e^y dy = \int 3x dx$ gir $e^y = 3/2x^2 + C$ eller $y = \ln(3/2x^2 + C)$, der $y(0) = 2$, og dermed $C = e^2$, og $y(0,6) = \ln(3/2(0,6)^2 + e^2) = 2,075$. Svarene numerisk og eksakt er 2,0482 og 2,075, og er nære hverandre. Hvis man velger mindre h -verdi, får vi en bedre presisjon.
- c)

```

1 from numpy import exp
2
3 def f(x,y):
4     return 3*x*exp(-y)
5
6 # or
7 # f = lambda x: 3xexp(-y)
8
9 # Euler metode
10 def euler(x0,y0,xn,n):
11
12     # Steg lengde
13     h = (xn-x0)/n
14
15     print('\n-----Løsning-----')
16     print('-----')
17     print('x0\ty0\tstigning\tyn')
18     print('-----')
19     for i in range(n):
20         stigning= f(x0, y0)
21         yn = y0 + h * stigning
22         print('% .4f\t% .4f\t%0.4f\t% .4f'% (x0,y0,stigning,yn) )
23         print('-----')
24         y0 = yn
25         x0 = x0+h

```

```

26
27     print('\nAt x=%.4f, y=%.4f' %(xn,yn))
28
29 print('Skriv startverdiene:')
30 x0 = float(input('x0 = '))
31 y0 = float(input('y0 = '))
32
33 print('Skriv inn y-verdi som skal beregnes: ')
34 xn = float(input('xn = '))
35
36 print('Skriv antall steg:')
37 Steg = int(input('Antall steg = '))
38
39 # Euler metode call
40 euler(x0,y0,xn,Steg)

```

Ved å kjøre programmet, får vi:

```

1  Skriv startverdiene:
2  x0 = 0
3  y0 = 2
4  Skriv inn y-verdi som skal beregnes:
5  xn = 0.6
6
7  Skriv antall steg:
8  Antall steg = 3
9
10 -----Løsning-----
11 -----
12 x0      y0      stigning      yn
13 -----
14 0.0000   2.0000   0.0000   2.0000
15 -----
16 0.2000   2.0000   0.0812   2.0162
17 -----
18 0.4000   2.0162   0.1598   2.0482
19 -----
20
21 At x=0.6000, y=2.0482

```

Fasit 9.1.4:

- c) Løsningen til $\frac{dy}{dt} = -ay(y - \frac{b}{a})$ er $y = \frac{b/a}{1 + Ce^{-bt}}$
 Python-program:

```

1  import numpy as np
2  from scipy.integrate import odeint
3  import matplotlib.pyplot as plt
4
5  # funksjonen for dy/dt
6  def model(y,t,a,b):
7      dydt = -a * y*y+b*y
8      return dydt
9
10 # starverdi
11 y0 = 5

```

```

12
13 # deler opp tidsaksen
14 t = np.linspace(0,20)
15
16 # solve ODEs
17 a= 0.1
18 b=0.2
19 y1 = odeint(model,y0,t,args=(a,b,))
20 a= 0.1
21 b=0.01
22 y2 = odeint(model,y0,t,args=(a,b,))
23 a = 0.01
24 b=0.2
25 y3 = odeint(model,y0,t,args=(a,b,))
26
27 # plot results
28 plt.plot(t,y1,'r-',linewidth=2,label='a=0.1, b=0.2')
29 plt.plot(t,y2,'b--',linewidth=2,label='a=0.2, b=0.1')
30 plt.plot(t,y3,'g:',linewidth=2,label='a=0.01, b=0.2')
31 plt.xlabel('time')
32 plt.ylabel('y(t)')
33 plt.legend()
34 plt.show()

```

Fasit 9.1.5:

Alternativ 1: Eulersmtode

```

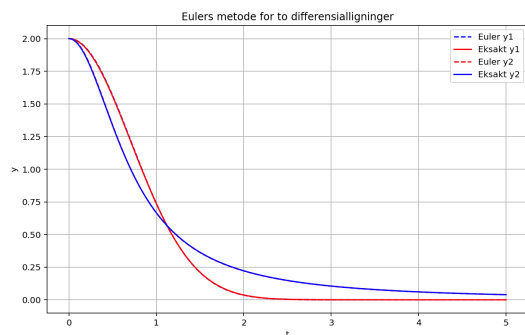
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parametre for Eulers metode
5 h = 0.01 # tidssteg
6 T = 5    # sluttid
7 n = int(T/h)
8 t = np.linspace(0, T, n+1)
9
10 # Initialbetingelser
11 y1 = np.zeros(n+1)
12 y2 = np.zeros(n+1)
13 y1[0] = 2
14 y2[0] = 2
15
16 # Eulers metode
17 for i in range(n):
18     y1[i+1] = y1[i] + h * (-2 * t[i] * y1[i])
19     y2[i+1] = y2[i] + h * (-2 * t[i] * y2[i]**2)
20
21 # Eksakte løsninger for sammenligning
22 y1_exact = 2 * np.exp(-t**2) # løsning av dy/dt = -2ty
23 y2_exact = 2 / (1 + 2 * t**2) # løsning av dy/dt = -2ty^2
24
25 # Plotting
26 plt.figure(figsize=(10, 6))
27 plt.plot(t, y1, label='Euler y1', color='blue', linestyle='--')
28 plt.plot(t, y1_exact, label='Eksakt y1', color='blue')

```

```

29 plt.plot(t, y2, label='Euler y2', color='red', linestyle='--')
30 plt.plot(t, y2_exact, label='Eksakt y2', color='red')
31 plt.xlabel('t')
32 plt.ylabel('y')
33 plt.title('Eulers metode for to differensialligninger')
34 plt.legend()
35 plt.grid(True)
36 plt.show()

```

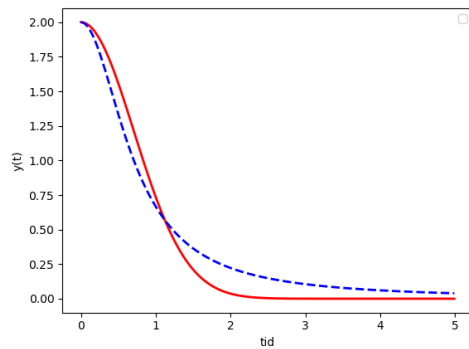


Alternativ 1: Odeint

```

1 from scipy.integrate import odeint
2 import matplotlib.pyplot as plt
3 import numpy as N
4
5 def f(y, t):
6     return -2*y*t
7
8 def g(y, t):
9     return -2 *t*y**2
10
11 y0 = 2          # startverdi
12 a = 0           # integrasjonsgrense
13 b = 5
14
15 t = N.arange(a, b, 0.01) # tidslinjen for løsningen
16
17 y1 = odeint(f, y0, t) # Løser f gitt y0
18 y2 = odeint(g, y0, t) # Løser g gitt y0
19
20
21 plt.plot(t, y1, 'r-', linewidth=2)
22 plt.plot(t, y2, 'b--', linewidth=2)
23 plt.xlabel('tid')
24 plt.ylabel('y(t)')
25 plt.legend()
26 plt.show()

```

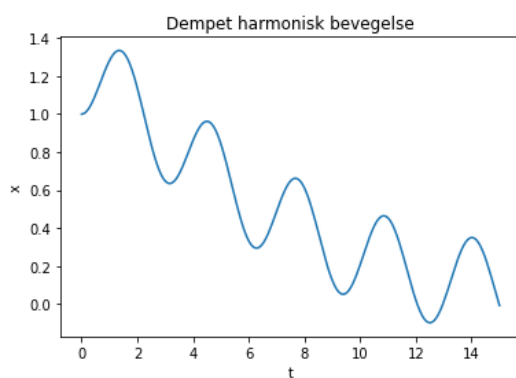


Fasit 9.1.6:

```

1 # Vi ønsker å løse  $x'' + 0.4x' + 0.05x = \cos(2x)$  gitt  $x(0)=1$  og  $x'(0)=0$ 
2 # Vi omskriver 2. orden differensial ligningen til et lineært system
3 #  $x'=y$  gir  $y' = -0.4y + 0.05x$ 
4
5 from scipy.integrate import odeint
6 import numpy as np
7 import matplotlib.pyplot as plt
8
9 def dydt(y,t):
10     # Her kan vi omskrive diff. ligningen som et system  $x=y[0]$  and  $x'=y$ 
11     # [1].
12     return [y[1], -0.4*y[1] - 0.05*y[0] + np.cos(2*t)]
13
14 y0 = [1, 0]
15 ts = np.linspace(0, 15, 200)
16 ys = odeint(dydt, y0, ts)
17 xs = ys[:,0]
18
19 plt.xlabel("t")
20 plt.ylabel("x")
21 plt.title("Dampet harmonisk bevegelse")
22 plt.plot(ts,xs);

```



Fasit 9.1.7:

```

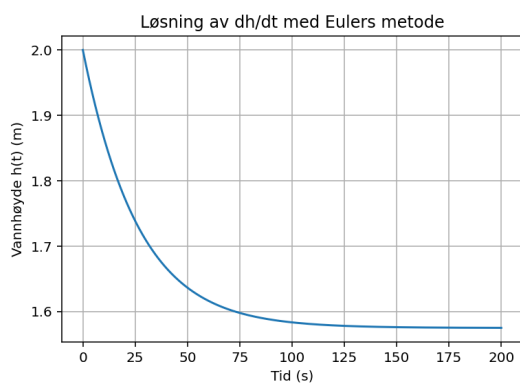
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Konstanter

```

```

5 v = 0.125
6 k = 0.0225
7 A = 1.0
8 g = 9.8
9
10 # Eulers metode innstillinger
11 h0 = 2.0      # Initialverdi
12 t0 = 0.0      # Starttid
13 tslutt = 200  # Sluttid
14 dt = 0.1      # Tidssteg
15
16 # Antall steg
17 antallsteg = int((tslutt - t0) / dt)
18
19 # Lagring for tid og løsning
20 tvalues = np.linspace(t0, tslutt, antallsteg)
21 hvalues = np.zeros(antallsteg)
22 hvalues[0] = h0
23
24 # Eulers metode
25 for i in range(1, antallsteg):
26     h = hvalues[i - 1]
27     dhdt = (1 / A) * (v - k * np.sqrt(2 * g * h))
28     hvalues[i] = h + dt * dhdt
29
30 # Plotting
31 plt.plot(tvalues, hvalues)
32 plt.xlabel("Tid (s)")
33 plt.ylabel("Vannhøyde h(t) (m)")
34 plt.title("Løsning av dh/dt med Eulers metode")
35 plt.grid(True)
36 plt.show()

```



Fasit 10.3.1:

```

1 import numpy as np
2 from scipy import linalg as lg
3 import scipy.linalg as la
4
5 from sympy import *
6
7 #Matrise A
8 A = np.array([ [0, 3] , [-3, 0] ])

```

```

9
10 #Print Matrix A
11 print('A=',A)
12
13 #Defining matrix B
14 B = np.array([ [1, 3], [3, 1] ])
15
16 #Print Matrix B
17 print('B=',B)
18
19 sum1 = A+B
20 #Print A+B
21
22 print('A+B=', sum1)
23
24 diff = A-B
25 #Print A-B
26 print('A-B=',diff)
27
28 prod = A.dot(B)
29 #Transpose
30
31
32 #Print A*B
33 print('AB=',prod)
34
35 transpose = A.T
36
37 #Print A'
38 print('Transponerte til A=',transpose)
39
40 #Determinant
41 determinantB = lg.det(B)
42 #Print det(B)
43 print('detB=',determinantB)
44
45 #Inverse (if non-singular)
46 invB = lg.inv(B)
47 #Eigenvalues, Eigenvectors of square matrix
48 print('Inversen til B=',invB)
49 egenverdi, egenvektor = lg.eig(A)
50
51 print('Egenverdiene til B er',egenverdi)
52
53 print('Egenvektorene til B er',egenvektor)

```

Ved å kjøre programmet, får vi:

```

1 A= [[ 0  3]
2   [-3  0]]
3 B= [[1 3]
4   [3 1]]
5 A+B= [[1 6]
6   [0 1]]
7 A-B= [[-1  0]
8   [-6 -1]]

```

```

9 AB= [[ 0  9]
10      [-9  0]]
11 Transponerte til A= [[ 0 -3]
12                      [ 3  0]]
13 detB= -8.0
14 Inversen til B= [[-0.125  0.375]
15                  [ 0.375 -0.125]]
16 Egenverdier til B er [0.+3.j 0.-3.j]
17 Egenvektorene til B er [[0.70710678+0.j          0.70710678-0.j          ]
18                          [0.          +0.70710678j 0.          -0.70710678j]]

```

Fasit 10.3.2:

```

b)
1 from sympy import *
2 # Definerer matrisen M
3 M=Matrix([[3, -2,  4, -2], [5,  3, -3, -2], [5, -2,  2, -2], [5, -2,
4               -3,  3]])
5 # sjekker om M er diagonaliserbar
6 print(M.is_diagonalizable())
7
8 # Diagonaliserer
9 P, D = M.diagonalize()
10
11 print('D=',D)
12 print('P=',P)
13
14 # Sjekker
15 A= P*D*P**-1
16 # undersøker om A er samme som M
17 print(A==M)
18
19 k = 20
20 Dk = D**k
21 Ak= P*Dk*P**-1
22 print('A^',k, 'er lik', Ak)
23 # sjekker om A^k=P*D**k*P**-1 er lik M^k
24 print(Ak==M**k)

```

Ved å kjøre programmet, får vi:

```

1 True
2 D= Matrix([[-2, 0, 0, 0], [0, 3, 0, 0], [0, 0, 5, 0], [0, 0, 0, 5]])
3 P= Matrix([[0, 1, 1, 0], [1, 1, 1, -1], [1, 1, 1, 0], [1, 1, 0, 1]])
4 True
5 A^ 20 er lik Matrix([[3486784401, -95363944856224, 190727889712448,
6               -95363944856224], [3485735825, 3486784401, 95360459120399,
7               -95363944856224], [3485735825, -95363944856224, 190727890761024,
8               -95363944856224], [3485735825, -95363944856224, 95360459120399,
9               3486784401]])
10 True

```

Fasit 10.3.3:

$$\text{a)} \quad \begin{pmatrix} x_{n+1} \\ y_{n+1} \end{pmatrix} = \begin{pmatrix} 0,86 & 0,08 \\ 0,15 & 1,15 \end{pmatrix} \begin{pmatrix} x_n \\ y_n \end{pmatrix}.$$

$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} 110 \\ 330 \end{pmatrix} \text{ og } \begin{pmatrix} x_2 \\ y_2 \end{pmatrix} = \begin{pmatrix} 121 \\ 363 \end{pmatrix}.$$

$$\text{b)} \quad \lambda_1 = 1,1, \vec{v}_1 = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \lambda_2 = 0,91, \vec{v}_2 = \begin{pmatrix} 8 \\ 5 \end{pmatrix}.$$

$$\vec{x}_n = C_1 \vec{v}_1 (\lambda_1)^t + C_2 \vec{v}_2 (\lambda_2)^t = C_1 \begin{pmatrix} 1 \\ 3 \end{pmatrix} (1,1)^t + C_2 \begin{pmatrix} 8 \\ 5 \end{pmatrix} (0,91)^t.$$

```

1 from sympy import *
2 import numpy as np
3 from scipy.linalg import eig
4 from scipy.linalg import inv
5
6 init_printing(use_unicode=True)
7
8 # X: antall ulver
9 # Y: antall rådyr
10 # Overgangsmatrise
11 # X(k+1) = M X(k) = M^k X(0)
12 M=Matrix([[0.86, 0.08], [-0.15, 1.15]])
13
14 M.is_diagonalizable()
15
16 egenverdi= M.eigenvals()
17 egenvektor = M.eigenvects(M)
18 print('egenverdiene er', egenverdi)
19 print('egenvektorene er', egenvektor)
20
21 P, D = M.diagonalize()
22
23 k = 4
24 Dk = D**k
25 Ak= P*Dk*P**-1
26
27 X0 = np.array([[100], [300]])
28
29 Akx=P*Dk*P**-1*X0

```

Med å kjøre programmet får vi:

```

1 egenverdiene er {11/10: 1, 91/100: 1}
2 egenvektorene er [(0.9100000000000000, 1, [Matrix([
3 [1.6],
4 [1.0]]))], (1.1000000000000000, 1, [Matrix([
5 [0.3333333333333333],
6 [1.0]])))]
7 X_ 4 er lik Matrix([[146.410000000000], [439.230000000000]])
8
9 XX_ 4 er lik Matrix([[146.410000000000], [439.230000000000]])

```

Fasit 10.3.4:

```

1 import numpy as np

```

```

2 from scipy import linalg as lg
3 import scipy.linalg as la
4
5 from sympy import *
6
7 #Matrise A
8 A = np.array([ [0, 3] , [-3, 0] ])
9
10 #Print Matrix A
11 print('A=',A)
12
13 #Defining matrix B
14 B = np.array([ [1, 3], [3, 1] ])
15
16 #Print Matrix B
17 print('B=',B)
18
19 sum1 = A+B
20 #Print A+B
21
22 print('A+B=', sum1)
23
24 diff = A-B
25 #Print A-B
26 print('A-B=',diff)
27
28 prod = A.dot(B)
29 #Transpose
30
31
32 #Print A*B
33 print('AB=',prod)
34
35 transpose = A.T
36
37 #Print A'
38 print('Transponerte til A=',transpose)
39
40 #Determinant
41 determinantB = lg.det(B)
42 #Print det(B)
43 print('detB=',determinantB)
44
45 #Inverse (if non-singular)
46 invB = lg.inv(B)
47 #Eigenvalues, Eigenvectors of square matrix
48 print('Inversen til B=',invB)
49 egenverdi, egenvektor = lg.eig(A)
50
51 print('Egenverdiene til B er',egenverdi)
52
53 print('Egenvektorene til B er',egenvektor)

```

Ved å kjøre programmet, får vi:

```

1 A= [[ 0  3]

```

```

2  [-3  0]]
3  B= [[1 3]
4      [3 1]]
5  A+B= [[1 6]
6         [0 1]]
7  A-B= [[-1  0]
8         [-6 -1]]
9  AB= [[ 0  9]
10       [-9  0]]
11 Transponerte til A= [[ 0 -3]
12                      [ 3  0]]
13 detB= -8.0
14 Inversen til B= [[-0.125  0.375]
15                  [ 0.375 -0.125]]
16 Egenverdiene til B er [0.+3.j 0.-3.j]
17 Egenvektorene til B er [[0.70710678+0.j          0.70710678-0.j          ]
18 [0.          +0.70710678j 0.          -0.70710678j]]

```

Fasit 10.3.5:

a)

b)

```

1  from sympy import *
2  # Definerer matrisen M
3  M=Matrix([[3, -2,  4, -2], [5,  3, -3, -2], [5, -2,  2, -2], [5, -2,
4             -3,  3]])
5  # sjekker om M er diagonaliserbar
6  print(M.isdiagonalizable())
7
8  # Diagonaliserer
9  P, D = M.diagonalize()
10
11 print('D=',D)
12 print('P=',P)
13
14 # Sjekker
15 A= P*D*P**-1
16 # undersøker om A er samme som M
17 print(A==M)
18
19 k = 20
20 Dk = D**k
21 Ak= P*Dk*P**-1
22 print('A^',k, 'er lik', Ak)
23 # sjekker om A^k=P*D**k*P**-1 er lik M^k
24 print(Ak==M**k)

```

Ved å kjøre programmet, får vi:

```

1  True
2  D= Matrix([[ -2,  0,  0,  0], [ 0,  3,  0,  0], [ 0,  0,  5,  0], [ 0,  0,  0,  5]])
3  P= Matrix([[ 0,  1,  1,  0], [ 1,  1,  1, -1], [ 1,  1,  1,  0], [ 1,  1,  0,  1]])
4  True
5  A^ 20 er lik Matrix([[3486784401, -95363944856224, 190727889712448,
                        -95363944856224], [3485735825, 3486784401, 95360459120399,

```

```

-95363944856224], [3485735825, -95363944856224, 190727890761024,
-95363944856224], [3485735825, -95363944856224, 95360459120399,
3486784401]])
6 True

```

Fasit 10.3.6:

a) $\begin{pmatrix} x_{n+1} \\ y_{n+1} \end{pmatrix} = \begin{pmatrix} 0,86 & 0,08 \\ 0,15 & 1,15 \end{pmatrix} \begin{pmatrix} x_n \\ y_n \end{pmatrix}.$
 $\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} 110 \\ 330 \end{pmatrix}$ og $\begin{pmatrix} x_2 \\ y_2 \end{pmatrix} = \begin{pmatrix} 121 \\ 363 \end{pmatrix}.$

b) $\lambda_1 = 1,1, \vec{v}_1 = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \lambda_2 = 0,91, \vec{v}_2 = \begin{pmatrix} 8 \\ 5 \end{pmatrix}.$

$$\vec{x}_n = C_1 \vec{v}_1 (\lambda_1)^t + C_2 \vec{v}_2 (\lambda_2)^t = C_1 \begin{pmatrix} 1 \\ 3 \end{pmatrix} (1,1)^t + C_2 \begin{pmatrix} 8 \\ 5 \end{pmatrix} (0,91)^t.$$

```

1 from sympy import *
2 import numpy as np
3 from scipy.linalg import eig
4 from scipy.linalg import inv
5
6 init_printing(use_unicode=True)
7
8 # X: antall ulver
9 # Y: antall rådyr
10 # Overgangsmatrise
11 # X(k+1) = M X(k) = M^k X(0)
12 M=Matrix([[0.86, 0.08], [-0.15, 1.15]])
13
14 M.is_diagonalizable()
15
16 egenverdi= M.eigenvals()
17 egenvektor = M.eigenvects(M)
18 print('egenverdiene er', egenverdi)
19 print('egenvektorene er', egenvektor)
20
21 P, D = M.diagonalize()
22
23 k = 4
24 Dk = D**k
25 Ak= P*Dk*P**-1
26
27 X0 = np.array([[100], [300]])
28
29 Akx=P*Dk*P**-1*X0

```

Med å kjøre programmet får vi:

```

1 egenverdiene er {11/10: 1, 91/100: 1}
2 egenvektorene er [(0.9100000000000000, 1, [Matrix([
3 [1.6],
4 [1.0]]))], (1.1000000000000000, 1, [Matrix([
5 [0.3333333333333333],
6 [1.0]]))]]

```

```

7 X_4 er lik Matrix([[146.410000000000], [439.230000000000]])
8
9 XX_4 er lik Matrix([[146.410000000000], [439.230000000000]])

```

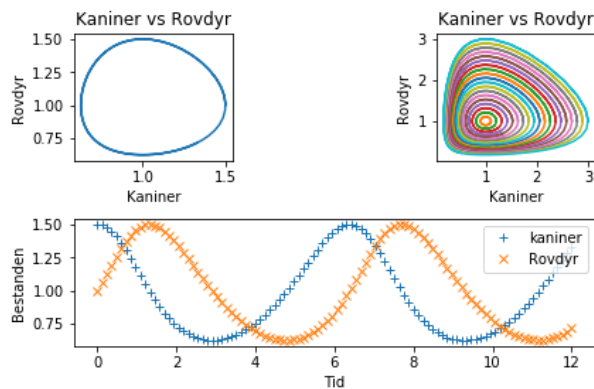
Fasit 10.4.1:

b)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 a,b,c,d = 1,1,1,1
6
7 def dN_dt(N, t):
8     return [N[0]*(a - b*N[1]), -N[1]*(c - d*N[0])]
9
10 ts = np.linspace(0, 12, 100)
11 N0 = [1.5, 1.0]
12 Ns = odeint(dN_dt, N0, ts)
13 prey = Ns[:,0]
14 predators = Ns[:,1]
15
16 plt.close()
17 fig = plt.figure()
18
19 plt.subplot(212)
20 plt.plot(ts, prey, "+", label="kaniner")
21 plt.plot(ts, predators, "x", label="Rovdyr")
22 plt.xlabel("Tid")
23 plt.ylabel("Bestanden")
24 plt.legend();
25
26 plt.subplot(231)
27 plt.plot(prey, predators, "-")
28 plt.xlabel("Kaniner")
29 plt.ylabel("Rovdyr")
30 plt.title("Kaniner vs Rovdyr");
31
32 fig.tight_layout(pad=1.0)
33
34 plt.subplot(233)
35 ix = np.linspace(1.0, 3.0, 20)
36 for r in ix:
37     N0 = [r, 1.0]
38     Ns = odeint(dN_dt, N0, ts)
39 plt.plot(Ns[:,0], Ns[:,1], "-")
40 plt.xlabel("Kaniner")
41 plt.ylabel("Rovdyr")
42 plt.title("Kaniner vs Rovdyr");

```



Fasit 10.4.2:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Definer antall delintervaller og del opp t-aksen
5 n = 2000
6 t = np.linspace(0,20, num = n+1)
7 # Finn steglengden
8 delta_t = t[1]-t[0]
9
10 # Initialiser løsningsvektorene
11 x = np.zeros(n+1)
12 y = np.zeros(n+1)
13
14 # Sett initialbetingelsene
15 x[0] = 1
16 y[0] = 0
17
18 # Definer konstantene
19 M = 1
20 alpha = 2
21 k = 1
22
23 # Los vha Eulers metode
24 for i in range(n):
25     x[i+1] = x[i] + y[i]*delta_t
26     y[i+1] = y[i] - ((alpha/M)*y[i] + (k/M)*x[i])*delta_t
27
28 plt.plot(t,x)

```

Fasit 12.0.1:

- $\nabla f_P = [2x - 6, 2y + 4]_P = [-4, 6]$.
 $D_{\mathbf{v}} f_P = \nabla f_P \cdot \frac{\mathbf{v}}{|\mathbf{v}|} = [-4, 6] \cdot \frac{1}{\sqrt{(-3)^2 + 4^2}} [-3, 4] = 48/5 = 9.6$
- Koden løser optimeringsproblemet $\min_x f(x)$, der $f(x, y) = x^2 + y^2 - 6x + 4y$ og $\nabla f = [2x - 6, 2y + 4]$. Det er brukt *Gradient Descent*-metoden. $HVA1 = 2x - 6$ og $HVA2 = 2y + 4$.
- I koden defineres betingelsen for god nok:

```

1 if np.abs(cost[-1] - cost[-2]) < tol:

```

Det vil si programmet stopper når kostfunksjonen (*cost*) endrer seg veldig lite. Denne kan endres til

```
1 for i in range(n):
2     grad_Fx, grad_Fy = gradF(x, y)
3     x = x - gamma * gradFx
4     y = y - gamma * gradFy
5     cost.append(F(x,y))
6     t+= 1
7     if np.linalg.norm(gradF(x, y)) < tol:
8         break
```

- d) Ved å kjøre programmet forventer man å få med *print(x, y)*.
p teller antall iterasjoner.

```
1 3.0    -2.0
```

$2x - 6 = 0$ gir $x = 3$ og $2y + 4 = 0$ gir $y = -2$.

Fasit 12.0.2:

- a) Programmet beregner numerisk differensialligningen $y' = f(x, y) = 0.5y$ gitt $y(0) = 1$ i intervallet $[0, 10]$ ved å dele opp intervallet i 4 like deler ($\Delta x = 2.5$)
b) Programmet beregner numerisk differensialligningen $y' = f(x, y) = y$ gitt $y(0) = 1$ ved hjelp av Eulers metode i intervallet $[0, 10]$ ved å dele opp intervallet i 4 like deler ($\Delta x = 2.5$).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Fra oppgaven
5 def f(x,y):
6     return 0.5*y
7
8 xmin = 0
9 xmax = 10
10
11 # Definerer antall delintervaller
12 N = 4
13
14 # Deler inn x-aksen
15 x = np.linspace(xmin, xmax, num = N+1)
16
17 deltax = (xmax - xmin)/N
18 # Kan eventuelt definerer deltax, og la N = (xmax - xmin)/deltax
19
20 # Eulers metode
21 y = np.zeros(N+1)
22
23 # Initialbetingelsen
24 y[0] = 1
25
26 print('\tx[n]\ty[n]\t dy/\dx \ty[n+1]')
27 # Løkke
28 for i in range(N):
29     y[i+1] = y[i] + f(x[i],y[i])*deltax
30
```

```

31     print('n=', i, '%.4f\t%.4f\t%.4f\t%.4f'%(x[i], y[i], f(x[i], y[i]), y
      [i+1]))
32
33 plt.plot(x, y, '--', label='Numerisk løsning')
34 plt.xlabel('x')
35 plt.ylabel('y')
36
37 """ Vi kan sjekke opp mot ekte løsning """
38 def yeksakt(x):
39     return np.exp(0.5*x)
40
41 xpen = np.linspace(xmin, xmax, 100)
42 plt.plot(xpen, yeksakt(xpen), label='ekte løsning')
43 plt.legend()
44
45 print('Den numeriske løsningen gir at y(', xmax, ') er', y[N])
46 print('Den eksakte løsningen er at y(', xmax, ') er', yeksakt(x[N]))

```

c) i) $xx=10$ og $xxxxx=25.6289$ (programmet skriver egentlig ut: 25.62890625)

ii) $y(2.5) = y(0) + f(0, 1) \cdot 2.5 = 1 + 0.5 \cdot 2.5 = 1 + 1.25 = 2.25$

$y(5) = y(2.5) + f(2.5, 2.25) \cdot 2.5 = 2.25 + 0.5 \cdot 2.25 \cdot 2.5$
 $= 2.25 + 2.8125 = 5.0625.$

d) Man kan øke N til en større verdi.

Fasit 12.0.3:

a) Minst én løsning: $f(a) \cdot f(b) < 0$.

Kun en løsning $f(a) \cdot f(b) < 0$ og f er monoton i intervallet (da er f en-entydig).

$$f(1) = \cos 1 > 0$$

$$f(2) = -1 + \cos 2 > 0$$

Dermed har ligningen minst én løsning på intervallet $(1, 2)$.

$f'(x) = -1 - \sin x < 0$, det vil si funksjonen er stigende i dette intervallet og dermed ligningen har kun én løsning på intervallet.

b) Newtons metode:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$f(x) = x - 1 - \cos(x)$ og $f'(x) = 1 + \sin(x)$ dermed:

$$x_{n+1} = x_n - \frac{x_n - 1 - \cos(x_n) - x_n}{1 + \sin(x_n)}$$

```

1 x0 = 1.5
2 x1= 1.2850994364385926
3 x2= 1.2834289428542498

```

Løsningen er 1.283. Presisjonen var ikke presisert, og her er det nok med to steg slik det sto i oppgaven.

Fasit 12.0.4:

a) Programmet beregner $\int_0^\pi \sin(x) dx$ ved å dele integrasjonsintervallet $[0, \pi]$ i 10 like store deler.

- b) Metoden starter ved å dele inn intervallet $[a, b]$ i en mengde delintervaller (som vi her lar være like store for enkelhets skyld). Hvis vi vil ha n delintervaller, betyr det at vi øker x fra a til b med steg som har størrelse $\Delta x = (b - a)/n$. For hver av disse delintervallene beregner arealet til trapeser, og den første og siste sidelengde ($f(a)$ og $f(b)$) brukes kun én gang mens de andre brukes to ganger.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # definerer funksjonen
5 def f(x):
6     return np.sin(x)
7
8 # Integrasjonsgrenser
9 a = 0
10 b = np.pi
11 #Antall delintervaller
12 N = 10
13 # Oppdeling til integrasjonspunkter (N+1 punkter for N deler)
14 x = np.linspace(a, b, num=N+1)
15 # Integrasjons steglengde
16 deltax = (b-a)/N
17
18 #Trapezmetoden: Første og siste funksjonsverdi brukes en gang i
    beregning av arealet til trapesene mens de andre
    funksjonsverdiene brukes to ganger:
19 sum = f(a) + f(b)
20 for i in range(1,N):
21     sum = sum + 2*f(x[i])
22 sum = sum*deltax/2

```

Med å endre N til en større verdi for å øke presisjonen.

c) $\Delta x = \frac{5-1}{4} = 1$

$$\int_1^5 \frac{1}{x^3} dx = \frac{1}{2} \left(\frac{1}{1^3} + \frac{2}{2^3} + \frac{2}{3^3} + \frac{2}{4^3} + \frac{1}{5^3} \right)$$

$$= 1/2 (1 + 0.25 + 0.074074074074074 + 0.03125 + 0.008) \simeq 0.681662037$$

Fasit 12.0.5:

- a) $\nabla f_P = [2x - 6, 2y + 4](1, 1) = [-4, 6]$.
 $D_{\mathbf{v}} f_P = \nabla f_P \cdot \frac{\mathbf{v}}{|\mathbf{v}|} = [-4, 6] \cdot \frac{1}{\sqrt{(-3)^2 + 4^2}} [-3, 4] = 48/5 = 9.6$
- b) Koden løser optimeringsproblemet $\min_x f(x)$ der $f(x, y) = x^2 + y^2 - 6x + 4y$ og $\nabla f = [2x - 6, 2y + 4]$. Det er brukt **Gradient Descent**-metoden. $HVA1 = 2x - 6$ og $HVA2 = 2y + 4$.
- c) I koden defineres betingelsen som god nok:

```

1 if np.abs(cost[-1] - cost[-2]) < tol:

```

Det vil si at programmet stopper når kostfunksjonen ($cost$) endrer seg veldig lite. Denne kan endres til

```

1 for i in range(n):
2     grad_Fx, grad_Fy = gradF(x, y)
3     x = x - gamma * grad_Fx
4     y = y - gamma * grad_Fy

```

```

5     cost.append(F(x,y))
6     t+= 1
7     if np.linalg.norm(gradF(x, y)) < tol:
8         break

```

- d) Ved å kjøre programmet forventer man å få med $\text{print}(x, y)$?
 p teller antall iterasjoner

```

1 3.0   -2.0

```

$2x - 6 = 0$ gir $x = 3$ og $2y + 4 = 0$ gir $y = -2$.

Fasit 12.0.6:

- a) Programmet beregner numerisk differensialligningen $y' = f(x, y) = 0.5y$ gitt $y(0) = 1$ i intervallet $[0, 10]$ ved å dele opp intervallet i 4 like deler ($\Delta x = 2.5$)
b) Programmet beregner numerisk differensialligningen $y' = f(x, y) = y$ gitt $y(0) = 1$ ved hjelp av Eulers metode i intervallet $[0, 10]$ ved å dele opp intervallet i 8 like deler ($\Delta x = 1.25$).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Fra oppgaven
5 def f(x,y):
6     return -0.5*y
7
8 x_min = 0
9 x_maks = 10
10
11 # Definerer antall delintervaller
12 N = 8
13
14 # Deler inn x-aksen
15 x = np.linspace(x_min, x_maks, num = N+1)
16
17 deltax = (x_maks - x_min)/N
18 # Kan eventuelt definerer deltax, og la N = (x_maks - x_min)/deltax
19
20 # Eulers metode
21 y = np.zeros(N+1)
22
23 # Initialbetingelsen
24 y[0] = 2
25
26
27 print('\t x[n]\t y[n]\t dy/dx \t y[n+1]')
28 # Løkke
29 for i in range(N):
30     y[i+1] = y[i] + f(x[i],y[i])*deltax
31
32     print('n=',i, '%.4f\t%.4f\t%.4f\t%.4f'%(x[i],y[i],f(x[i],y[i]),y[i+1]))
33
34
35 plt.plot(x,y,'--',label='Numerisk løsning')
36 plt.xlabel('x')
37 plt.ylabel('y')

```

```

38
39 """ Vi kan sjekke opp mot ekte løsnings """
40 def y_eksakt(x):
41     return 2*np.exp(-0.5*x)
42
43 x_pen = np.linspace(x_min,x_maks,100)
44 plt.plot(x_pen,y_eksakt(x_pen),label = 'ekte løsnings')
45 plt.legend()
46
47 print('Den numeriske løsningen gir at y(',x_maks,') er',y[N])
48 print('Den eksakte løsningen er at y(',x_maks,') er',y_eksakt(x[N]))

```

- c) i) $xx=10$ og $xxxxx=0.013475893998170934$
 ii) $y(1.25) = y(0) + f(0, 2) \cdot 1.25 = 2 + (-1) \cdot 1.25 = 2 - 1.25 = 0.75$
 $y(2.5) = y(1.25) + f(1.25, 0.75) \cdot 2.5 = 0.75 - 1.5 \cdot 1.25 \cdot -1.125.$
- d) Man kan øke N til en større verdi.

Fasit 12.0.7:

- a) Minst én løsning: $f(a) \cdot f(b) < 0$.
 Kun én løsning $f(a) \cdot f(b) < 0$ og f er monoton i intervallet (da er f en-entydig).

$$f(1) = \cos 1 > 0$$

$$f(2) = -1 + \cos 2 > 0$$

Dermed ligningen har minst

én løsning på intervallet $(1, 2)$.

$f'(x) = -1 - \sin x < 0$, det vil si at funksjonen er stigende i dette intervallet og dermed har ligningen kun én løsning på intervallet.

- b) Newtons metode:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

$f(x) = x - 1 - \cos(x)$ og $f'(x) = 1 + \sin(x)$, dermed:

$$x_{n+1} = x_n - \frac{x_n - 1 - \cos(x_n) - x_n}{1 + \sin(x_n)}$$

```

1 x0 = 1.5
2 x1= 1.2850994364385926
3 x2= 1.2834289428542498

```

Løsningen er 1.283. Det var ikke presisert presisjonen, og her er det nok med to steg slik det sto i oppgaven.

Fasit 12.0.8:

- a) Programmet beregner $\int_0^\pi \sin(x)dx$ ved å dele integrasjonsintervallet $[0, \pi]$ i 10 like store deler.
- b) Metoden starter ved å dele inn intervallet $[a, b]$ i en mengde delintervaller (som vi her lar være like store for enkelhets skyld). Hvis vi vil ha n delintervaller, betyr det at vi øker x fra a til b med steg som har størrelse $\Delta x = (b - a)/n$. For hver av disse delintervallene beregner arealet til trapeser og den første og siste sidelengde ($f(a)$ og $f(b)$) brukes kun én gang mens de andre brukes to ganger.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # definerer funksjonen
5 def f(x):
6     return np.sin(x)
7
8 # Integrasjonsgrenser
9 a = 0
10 b = np.pi
11 #Antall delintervaller
12 N = 10
13 # Oppdeling til integrasjonspunkter (N+1 punkter for N deler)
14 x = np.linspace(a, b, num=N+1)
15 # Integrasjons steglengde
16 deltax = (b-a)/N
17
18 #Trapezmetoden: Første og siste funksjonsverdi brukes en gang i
    beregning av arealet til trapesene mens de andre
    funksjonsverdiene brukes to ganger:
19 sum = f(a) + f(b)
20 for i in range(1,N):
21     sum = sum + 2*f(x[i])
22 sum = sum*deltax/2

```

Ved å endre N til en større verdi kan man å øke presisjonen.

c) $\Delta x = \frac{5-1}{4} = 1$

$$\int_1^5 \frac{1}{x^3} dx = \frac{1}{2} \left(\frac{1}{1^3} + \frac{2}{2^3} + \frac{2}{3^3} + \frac{2}{4^3} + \frac{1}{5^3} \right)$$

$$= 1/2 (1 + 0.25 + 0.074074074074074 + 0.03125 + 0.008) \simeq 0.681662037$$

Bibliografi

- [1] Tor Gulliksen, Amir M. Hashemi and Arne Hole (2022). Matematikk i praksis, 7. utgave. Universitetsforlaget 2022, ISBN 9788215062778.
- [2] BBC Bitesize (2019). KS3 Computer Science. Introduction to computational thinking. Hentet fra <https://www.bbc.co.uk/bitesize/topics/z7tp34j>
- [3] Udir (2019). Matematikk fellesfag 1-10 – oppsummering av høringen. Hentet fra <https://www.udir.no/laring-og-trivsel/lareplanverket/fagfornyelsen/oppsummeringer-av-7000-horingssvar-om-nye-lareplaner/matematikk-grunnskole-oppsummering-av-horingen/>
- [4] Biologiske fagtermer fra A til Å. Institutt for biovitenskap ved matematisk-naturvitenskapelig fakultet ved UiO. Hentet fra <https://www.mn.uio.no/ibv/tjenester/kunnskap/plantefys/leksikon/>.
- [5] Collins Online Dictionary. Hentet fra <https://www.collinsdictionary.com/dictionary/english>
- [6] Jakobsen, Renate E. (2019) Programmering i matematikk –muligheter og utfordringer (Masteroppgave, UiA). Henter fra [https://uia.brage.unit.no/uia-xmlui/bitstream/handle/11250/2646381/Renate 20Elise 20Jakobsen.pdf](https://uia.brage.unit.no/uia-xmlui/bitstream/handle/11250/2646381/Renate%20Elise%20Jakobsen.pdf)
- [7] Kunnskapsdepartementet (2013). Læreplan i matematikk fellesfag. Hentet fra <https://www.udir.no/kl06/mat1-04>
- [8] Kunnskapsdepartementet (2018). Kjerneelementer i fag. Hentet fra <https://www.regjeringen.no/no/aktuelt/forny-er-innholdet-i-skolen/id2606028/>
- [9] Papert, Seymour. Mindstorms: Children, computers, and powerful ideas. Basic Books, Inc., 1980. Hentet fra [http://worrydream.com/refs/Papert20 – 20Mindstorms201st20ed.pdf](http://worrydream.com/refs/Papert20-20Mindstorms201st20ed.pdf)
- [10] Udir, Notat nr. 2 - Programmering i skolen. November 2016104. Programmering som kompetanse for det 21. århundre. <https://www.udir.no/kvalitet-og-kompetanse/profesjonsfaglig-digital-kompetanse/notat-om-programmering-i-skolen/>
- [11] Utdanningsdirektoratet (2013). Læreplan i matematikk fellesfag. Hentet fra <https://www.udir.no/kl06/MAT1-04>
- [12] Utdanningsdirektoratet (2017). Kjerneelementer i matematikk, men hvorfor programmering? Hentet fra <https://udirbloggen.no/kjerneelementer-i-matematikk-men-hvorfor-programmering/>
- [13] Utdanningsdirektoratet (2019). Læreplan i matematikk 1.-10. trinn. Hentet fra <https://www.udir.no/lk20/mat01-05>

- [14] Utdanningsdirektoratet (2019). Læreplan i matematikk fellesfag Vg1 teoretisk. Hentet fra <https://www.udir.no/lk20/mat09-01?lang=nob>
- [15] Wing, Jeannette (april 2008). «Computational thinking and thinking about computing». 2008 IEEE International Symposium on Parallel and Distributed.

Dette heftet vil fremme læreglede for å lære seg algoritmisk tenkning, modellering og problemløsning, og vil bruke programmering til å vise hvordan man kan løse oppgaver og problemer fra realfag og andre fag i ungdomstrinnet og videregående skole trinnet.

Struktur og progresjon i oppgavene skal forberede Deltagrene til å kunne løse utfordrende problemstillinger og tilegne seg utholdenhet, kreativitet, selvtillit, og vil bidra til matematisk tenkning.

Problem: Regn ut $\sqrt{1,06}$ uten å bruke kalkulator.

Matematisk modell: tangentmetoden:

$$f(x) = \sqrt{x+1}, f(0) = 1, \quad f'(x) = \frac{1}{2\sqrt{x+1}}, f'(0) = \frac{1}{2}.$$

$$\sqrt{x+1} \simeq 1 + \frac{1}{2}x \quad \text{når } x \text{ er nær } 0 \ (x \simeq 0).$$

$$\sqrt{1,06} \simeq 1 + \frac{0,06}{2} = 1,03.$$

